

# shAnalysis Workflow & Theory Guide

GRACE / GRACE-FO spherical-harmonic processing in base MATLAB — theory, best practices, and step-by-step recipes

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Toolbox version | shAnalysis v3.6.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Edition         | 5 (adds: ICGEM time-series download, the standard chain and custom filter design as single entry points, and what real provider files actually look like — FORTRAN D-exponents, the ICGEM 2.0 column order, ragged record groups)                                                                                                                                                                                                                                                     |
| Scope           | ICGEM I/O, spectral diagnostics, synthesis & analysis, destriping, Gaussian / DDK / tvANS filtering, VCE, basin averages & deconvolution, climatology & AR(1) significance, GIA, degree-1/C20 chain, Slepian localization, SINEX covariances, mascon comparison, load deformation, gradient tensor, multi-center combination, sea-level fingerprints, Love-number I/O incl. frame conversion, ICGEM series download, pole-tide conventions, pointwise synthesis at satellite altitude |
| Dependencies    | base MATLAB only (no toolboxes); Java optionally for gz streaming                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Validation      | every numerical method independently validated in Python (numpy/scipy) before MATLAB implementation; 169 unit tests                                                                                                                                                                                                                                                                                                                                                                   |
| License         | MIT (see LICENSE; bundled DDK3 Wbd file MIT from its upstream repository; ITSG/GFZ fixtures remain their providers' property)                                                                                                                                                                                                                                                                                                                                                         |
| Provenance      | Developed by Matthias Weigelt with the help of Claude (Fable 5), 2026-08-07                                                                                                                                                                                                                                                                                                                                                                                                           |

**How to read this guide.** Part I develops the theory exactly as implemented (equation forms match the code, including storage conventions and numerically validated tolerances). Part II condenses the best practices the implementation enforces or enables. Part IV is the complete API reference — every public function and method with typed, dimensioned inputs/outputs, defaults and a real-data example, generated from the source arguments blocks at build time. Part III gives copy-paste step-by-step recipes for the standard products. The HTML reference (doc shAna`l`ysis) documents every function signature; this guide explains why and in which order.

# Part I — Theory as implemented

## 1. Spherical harmonics, normalization, storage

The disturbing potential is expanded in fully  $(4\pi)$ -normalized real spherical harmonics:

$$V(r, \vartheta, \lambda) = \frac{GM}{R} \sum_{n=0}^N \left(\frac{R}{r}\right)^{n+1} \sum_{m=0}^n [\bar{C}_{nm} \cos m\lambda + \bar{S}_{nm} \sin m\lambda] \bar{P}_{nm}(\cos \vartheta)$$

with the normalization  $\int |Y_{nm}|^2 d\Omega = 4\pi$ , the ICGEM convention. Throughout the toolbox coefficients live in lower-triangular matrices with 1-based indexing  $C(n+1, m+1)$ ; vectorized orderings are always mediated by `shLowLevel.shIndex` (fields `n`, `m`, `cs`, `P`; `MinDegree` defaults to 2 for GRACE work, 0 for analysis). `GM` and `R` ride with the data (read from the `gfc` header); the defaults (3.986004415e14, 6378136.3) are overridable, never baked-in physics. Load Love numbers  $k_n$  for water-storage kernels must be supplied explicitly — the toolbox refuses to guess them.

## 2. Legendre functions: scaled Holmes–Featherstone recursion

Fully normalized associated Legendre functions are computed by the standard order-diagonal seed plus degree recursion. The naive diagonal seed underflows near the poles at high degree; `shLowLevel.legendreALF` therefore implements the globally scaled Holmes–Featherstone variant (scale  $10^{-280}$ , rescaling on the fly), verified stable to degree 2190: the sum rule and parity

$$\sum_{m=0}^n \bar{P}_{nm}^2(\cos \vartheta) = 2n + 1 \quad (\sim 10^{-11} \text{ at } n = 2190), \quad \bar{P}_{nm}(-\varphi) = (-1)^{n+m} \bar{P}_{nm}(\varphi)$$

hold across all latitudes; the parity relation halves the recursion cost on hemispherically symmetric grids (the parity trick).

## 3. Synthesis

Spatial synthesis evaluates, per latitude, the order sums

$$A_m(\varphi) = \sum_{n=m}^N f_n \bar{C}_{nm} \bar{P}_{nm}(\varphi), \quad B_m(\varphi) \text{ analogously with } \bar{S}_{nm},$$

then the longitude sum

$$g(\varphi, \lambda) = \sum_{m=0}^N [A_m(\varphi) \cos m\lambda + B_m(\varphi) \sin m\lambda].$$

For uniform full-circle longitude grids this last sum is an inverse FFT after folding aliased orders into  $\text{mod}(m, n_{\text{lon}})$  bins — exact, validated against the direct product to 3e-13 (`Method="auto"|"fft"|"direct"`).

**Memory model (v2.2).** The Legendre stack  $(n_{\text{max}}+1)^2 \times n_{\text{lat}} \times 8$  bytes is the memory driver: 7 GB at  $n_{\text{max}} = 2190$  over 181 latitudes. Above the 'MaxMemGB' budget (default 4), synthesis streams latitude bands: compute the Legendre block, fold into  $A_m/B_m$ , discard. Chunked equals monolithic exactly (Python-validated, max deviation 0.0); the class-level Legendre cache is bypassed automatically when the stack would not fit.

## 4. Quantity kernels

A single degree factor  $f_n$  maps Stokes coefficients to the target quantity (`shLowLevel.kernelFactors`):

| quantity            | $f_n$                                                    | unit                    |
|---------------------|----------------------------------------------------------|-------------------------|
| geoid               | $R$                                                      | m                       |
| potential           | $GM/R$                                                   | $\text{m}^2/\text{s}^2$ |
| gravity disturbance | $\frac{GM}{R^2} (n+1)$                                   | $\text{m}/\text{s}^2$   |
| gravity anomaly     | $\frac{GM}{R^2} (n-1)$                                   | $\text{m}/\text{s}^2$   |
| EWB (water)         | $R \frac{\rho_{\text{ave}}}{3\rho_w} \frac{2n+1}{1+k_n}$ | m                       |
| surface density *   | $R \frac{\rho_{\text{ave}}}{3} \frac{2n+1}{1+k'_n}$      | $\text{kg}/\text{m}^2$  |

| quantity                    | $f_n$                                                                                 | unit             |
|-----------------------------|---------------------------------------------------------------------------------------|------------------|
| bottom pressure *           | $g_0 R^{\frac{\rho_{\text{ave}}}{3} \frac{2n+1}{1+k'_n}}, \quad g_0 = \frac{GM}{R^2}$ | Pa               |
| vertical deformation *      | $R \frac{h'_n}{1+k'_n}$                                                               | m                |
| gravity gradient $T_{rr}$ * | $\frac{GM}{R^3} (n+1)(n+2)$                                                           | 1/s <sup>2</sup> |

(\* new in v2.3.) The 'Height' option continues potential-type kernels upward by  $(R/r)^p$  ( $p = n+1$  potential,  $n+2$  anomaly/disturbance,  $n+3$   $T_{rr}$ ; validated against  $-d/dr$  and  $d^2/dr^2$  of the continued potential) — evaluate directly at satellite altitude. Surface quantities reject Height by contract.

The EWH kernel divides by  $(1+k_n)$ : the observed potential contains the solid-Earth loading response, which must be removed to recover surface mass.  $k_n$  is always user-supplied (e.g. Wahr et al. 1998 or your loading-theory set of choice).

## 5. Elastic load deformation (v2.3)

A surface load deforms the elastic Earth; the displacement at the surface, expressed through the load-induced (residual) Stokes coefficients, is (Wahr et al. 1998):

$$u_{\text{up}} = R \sum_{n,m} \frac{h'_n}{1+k'_n} \Delta \bar{C}_{nm} \bar{Y}_{nm}, \quad u_{\text{north}} = R \sum_{n,m} \frac{l'_n}{1+k'_n} \Delta \bar{C}_{nm} \frac{\partial \bar{Y}_{nm}}{\partial \varphi}, \quad u_{\text{east}} = R \sum_{n,m} \frac{l'_n}{1+k'_n} \Delta \bar{C}_{nm} \frac{1}{\cos \varphi} \frac{\partial \bar{Y}_{nm}}{\partial \lambda}$$

The vertical is a pure degree factor ('deformation\_up'); the horizontals need the exact spherical-harmonic gradient. `shLowLevel.legendreALFDeriv` provides  $d\text{Pbar}/d\phi$  through a frozen identity — the  $(n,m)$  coefficient pattern (with  $\sqrt{2}$  corrections at  $m=0$  and  $m=1$  from the normalization of  $\text{Pbar}_{n0}/\text{Pbar}_{n1}$ ) was calibrated by least squares against numerical differentiation and then frozen; residuals sit at the finite-difference floor ( $\sim 1\text{e-}9$ ).

`shLowLevel.shSynthesisDeformation / g.deformation` evaluate all three components on grids or GNSS station lists (Mode="points", LatType="geodetic"); north/east validated point-wise against brute-force gradients to  $\sim 5\text{e-}10$ . Degree 0 is excluded; degree 1 is opt-in and only meaningful with consistent geocenter (CF/CE frame) handling; all three Love-number sets must come from ONE loading model. East is NaN at the poles (undefined direction).

## 6. Analysis (the inverse problem)

`shLowLevel.shAnalysisGrid / shCoefficients.analysis` estimate coefficients from data. Two regimes:

- **Ring grids** (uniform full-circle longitudes, arbitrary latitude rings): an FFT along longitude decouples the normal equations per order; each order solves a small least-squares over latitude rings. Exact recovery of band-limited input to  $\sim 8.7\text{e-}15$  when the sampling theorem is met ( $n_{\text{lat}} \geq n_{\text{max}} + 1$ ,  $n_{\text{lon}} \geq 2n_{\text{max}} + 1$ ).
- **Scattered points**: chunked accumulation of full normal equations. Under-determined or ill-conditioned samplings (polar gaps, regional patches) need Kaula regularization: a diagonal prior

$$\sigma_{\text{prior}}^2(n) = \left(\frac{K}{n^2}\right)^2$$

stabilizes the inversion at the cost of damping unconstrained coefficients toward zero. For regional data the mathematically cleaner route is Slepian localization (Section 12) — estimate only the  $\sim N$  concentrated linear combinations the data can actually see.

## 7. Destriping and Gaussian smoothing

GRACE monthly solutions carry correlated north–south stripes: within one order and parity, coefficients of successive degrees correlate. The classic Swenson–Wahr destriper (`shLowLevel.shDestripe`) fits and removes a low-order polynomial across same-parity degree series per order (defaults: orders  $\geq 8$ , polynomial order 2, window 5, all configurable). Gaussian smoothing (`shLowLevel.shGaussianFilter`) multiplies degree-wise by Jekeli weights from the recursion

$$b = \frac{\ln 2}{1 - \cos(r/R)}, \quad W_0 = 1, \quad W_1 = \frac{1 + e^{-2b}}{1 - e^{-2b}} - \frac{1}{b}, \quad W_{n+1} = -\frac{2n+1}{b} W_n + W_{n-1}$$

with the half-response radius  $r$  as the single tuning knob. Both are provided primarily as the baseline against which tvANS and DDK are judged.

## 8. DDK anisotropic decorrelation (v2.2)

The DDK filters (Kusche 2007; Kusche et al. 2009: DDK1–DDK8, ordered strong→weak) replace the scalar Gaussian weight by a full matrix per (order, C/S) block, derived from a GRACE error covariance and a signal prior — anisotropic, order-dependent smoothing that suppresses stripes with less signal attenuation than an equivalent Gaussian. `shLowLevel.readDDK` parses the released binary `Wbd` files natively (Rietbroek/Kusche BIN format; endian autodetect; verified to 4.4e-16 against the repository-documented DDK3 reference values; DDK1..8 = `a_1d14p_4`, 1d13, 1d12, 5d11, 1d11, 5d10, 1d10, 5d9; `Nmax`= truncates an `Lmax`-120 filter to `n96` series), plus `.mat` containers and an ASCII exchange layout; `applyDDK` applies block-wise `c → M·c`, passing uncovered degrees through unchanged. Formal errors are invalidated by the filter (coefficients become correlated); propagate uncertainties with `shLowLevel.mcPropagate`.

**Note.** The real DDK3 file (`Wbd_2-120.a_1d12p_4`, MIT-licensed, from [github.com/strawpants/GRACE-filter](https://github.com/strawpants/GRACE-filter)) ships in `tests/test_data`; the unit tests pin the parser to the repository-documented pack values. Degrees below the filter's `Lmin` (2) pass through unfiltered, as in the reference implementation.

## 9. The tvANS filter (time-variable anisotropic noise/signal Wiener filter)

The toolbox's flagship decorrelation filter. Model per month  $t$ :  $x = \text{signal} + \text{noise}$  with signal covariance  $S$  (estimated from the data themselves) and noise covariance  $s_t N$  (a shape  $N$ , e.g. from a SINEX covariance or the built-in order/parity model, scaled monthly by VCE factors  $s_t$ ). The Wiener estimate of the residual field is

$$\hat{x}_t = W_t x_t, \quad W_t = S(S + s_t N)^{-1}$$

Implementation: one generalized symmetric eigendecomposition gives, for every month simultaneously,

$$SU = NUA \Rightarrow W_t = V \text{diag}\left(\frac{\lambda_i}{\lambda_i + s_t}\right) U^T, \quad V = U^{-T}$$

—  $O(P^2)$  per month instead of a fresh  $P^3$  solve. The deterministic model (bias/trend/annual/semi-annual) is removed first and restored after filtering; VCE factors are estimated from high-degree residuals via a robust median  $\chi^2$  estimator (`shLowLevel.vceRescale`). With a block-diagonal  $N$  (order/parity blocks), the whole machinery runs per block (`Blocks="auto"`), tractable to `Lmax`  $\approx$  120. External noise models join the block path when they are block-diagonal in the (order, C/S, parity) partition (verified, not assumed — v2.2.2); dense covariances fall back to the full path (auto) or error loudly (on).

**Per-order-band VCE (v2.2).** One  $s_t$  per month is coarse: striping strength varies with order. `VCEBands`=[0 16 33 61] estimates independent monthly factors per order band (block path only — band-wise scaling breaks the single global eigendecomposition, which the code asserts). The per-block factors are honored consistently in the filter application, the posterior sigmas, and the basin deconvolution covariance (identity of the banded block filter with the directly constructed  $W$  validated to 5.6e-16).

**Posterior uncertainty.** The filtered-residual variance

$$\text{diag}((I - W_t)S) = (V \circ V) \frac{s_t \lambda}{\lambda + s_t}$$

(validated 3.5e-16) is exposed as per-coefficient/month sigma stacks (`tsF.sigmaCs/Ss`). With hard constraints (v2.5) the EXACT diagonal of  $(W-I)S(W-I)' + s \cdot \text{WNW}$  is used ( $O(P^2q)$  in the eig basis, validated 2e-15 against the brute-force covariance; the constrained-direction identity  $Ac' \cdot \text{Cov} \cdot Ac = s \cdot Ac' N Ac$  holds exactly). Honest correction: the earlier formula UNDERESTIMATED along constrained directions (ratios down to  $\sim 0.72$ ) — the pre-v2.5 'upper bound' note had the direction wrong.

## 10. Basin averages and deconvolution

A basin kernel  $b$  (Section 11) gives the filtered basin series  $a_t = b^T x_t / (b^T b)$ . Because filtering attenuates and leaks, `shLowLevel.basinDeconvolve` solves the small linear system that undoes the filter's action on a set of basin kernels  $B$ :

$$c_t = A^{-1} B^T \hat{x}_t, \quad A = B^T W_t B \quad (K \times K); \quad \text{cov}(c_t) = A^{-1} [G_v^T \text{diag}(s_{\text{vec}} g^2) G_v] A^{-T}$$

where  $g$  are the spectral gains and  $s_{\text{vec}}$  the (possibly band-wise) monthly noise factors — the v2.2 form is exactly the v2.1 formula when bands are off. Ill-conditioned kernel sets accept a relative ridge ( $\lambda = \text{Ridge} \cdot \|A\|$ ), with the covariance consistently using the regularized inverse. Since v2.5 the basin sigma also carries the OLS parameter uncertainty of the restored deterministic part (design leverage  $h_t$   $\times$  per-coefficient residual variance, carried on the operator; Monte-Carlo: `emp/pred` 1.18 without the term, 1.00 with it, 1.26  $\rightarrow$  1.08 at seasonal leverage peaks), and with constraints the deconvolution noise covariance uses the exact constrained operator. The deterministic-part errors remain correlated across epochs through the shared fit (pointwise sigma; documented).

## 11. Basin kernels: buffering and tapering (v2.2)

`shLowLevel.basinKernel` builds  $b = Y^T(w \cdot \text{mask})$  by exact Gauss–Legendre quadrature from a polygon, mask, or  $f(\text{lat}, \text{lon})$  handle. Two leakage controls:

- **BufferKm**: grow/shrink the region by exact great-circle distance before quadrature (trade leakage-in against leakage-out; computed in latitude chunks, practical to  $L_{\text{max}} \sim 96$ ).
- **TaperKm**: multiply the kernel spectrally by Jekeli Gaussian weights — soft edges suppress the ringing of the truncated indicator. Rule of thumb: taper radius  $\approx$  the coefficient filter's half-response radius.

The quadrature area fraction is returned;  $b(1)$  equals it exactly ( $Y_{00} = 1$ ), which the tests pin down.

## 12. Slepian localization (v2.2)

For a region  $R$ , the concentration problem maximizes the energy ratio over band-limited  $g = Yx$ :

$$\lambda = \frac{\int_R g^2 d\Omega}{\int_\Omega g^2 d\Omega} \rightarrow \max, \quad K = Y^T \text{diag}(w \cdot \text{mask}) Y, \quad N_{\text{Shannon}} = \sum_i \lambda_i = \frac{A_R}{4\pi} P$$

Eigenvectors (Slepian tapers) are orthonormal; eigenvalues are concentrations in  $[0,1]$ ; the Shannon number (exact by the addition theorem — the trace identity the tests verify) counts the tapers the region supports. Regional analysis then estimates  $\sim N$  Slepian coefficients  $a = G^T x$  instead of  $P$  Stokes coefficients — well-posed by construction, no Kaula prior needed. Validated on the  $30^\circ$  polar cap:  $\lambda$  bounds to  $4e-16$ , orthonormality  $3e-15$ .

## 13. Climatology, alias periods, AR(1) significance (v2.2)

`ts.climatology()` fits bias, trend, annual and semi-annual terms (plus optional extra periods) per coefficient. GRACE tidal aliasing puts S2 at 161 d, K2 at 3.66 yr, K1 at 7.48 yr: `Periods=[161/365.25, 3.66, 7.48]` keeps these out of trend and semi-annual estimates. Per-coefficient 1-sigma significance comes from the OLS covariance (weighted and robust variants available) and rides on every component accessor, enabling significance-masked maps.

**AR(1) correction.** GRACE residuals are temporally correlated; white-noise sigmas are optimistic. `ARCorrect=true` estimates the lag-1 residual autocorrelation  $r_1$  per coefficient and inflates sigmas by

$$\sigma_{\text{AR}(1)} = \sigma \sqrt{\frac{1+r_1}{1-r_1}}$$

— the standard first-order effective-sample-size correction, since v2.5 applied to the Kendall bias-corrected  $r_1' = r_1 + (1+3r_1)/T$ . Monte-Carlo validation: uncorrected sigmas underestimate the empirical trend scatter by  $2.00 \times (\phi = 0.6, T = 120)$ ; with the raw- $r_1$  inflation the ratio is 1.065, with the bias-corrected  $r_1$  1.028 ( $1.152 \rightarrow 1.079$  at  $T = 60$ ). Still first-order and slightly conservative — the residual few percent stems from the regression absorbing low-frequency noise.

## 14. GIA correction (v2.2)

Secular Stokes trends mix present-day mass change with glacial isostatic adjustment. `ts.removeGIA(gia)` subtracts  $\text{rate} \cdot (t - T_0)$  per epoch; `clim.removeGIA(gia)` corrects a fitted trend directly — both routes agree identically (tested to  $1e-13$ ). GIA models (ICE-6G\_D, Caron et al., A/W13, ...) are user-supplied gfc rate files, never embedded. The model is treated as exact: its uncertainty, often the dominant regional trend error (Laurentia, Fennoscandia, Antarctica), is not propagated — difference several models as a spread estimate.

## 15. Geocentric vs. geodetic latitude (v2.2)

All SH mathematics here runs in geocentric latitude; maps, GIS products and mascon grids are geodetic:

$$\tan \varphi' = (1 - f)^2 \tan \varphi$$

with the WGS84 flattening as overridable default. The difference peaks at  $0.192^\circ$  ( $\sim 21$  km) near  $45^\circ$  — large enough to visibly bias basin averages and point comparisons. Synthesis, analysis and `shAnalysisGrid` accept `LatType="geodetic"` and convert internally.

## 16. The degree-1 / C20 completion chain

GRACE does not observe geocenter motion (degree 1), and its C20 is weak: the standard product chain is GSM + TN-14 (C20/C30 from SLR) + TN-13 (degree-1). `shLowLevel.readTN14` and `shLowLevel.readTN13` parse the official technical notes (TN-13 verified against the real GFZ RL06 file: 256 paired months 2002-04 to 2026-04, including the 2017.4–2018.4 gap); `ts.addDegree1` and the C20/C30 replacement close the chain with epoch

matching and provenance notes in the history.

## 17. Normal field and (GM, R) conventions (v2.4)

Geoid maps need the DISTURBING field: subtract the ellipsoidal normal potential. `shLowLevel.normalFieldCS` computes the even zonals from the WGS84/GRS80 defining constants (GM, a, f,  $\omega$ ) via the Heiskanen–Moritz closed form ( $q_0$  series  $\rightarrow J_2 \rightarrow J_{2n}$ ) — no coefficient tables; validated against NIMA TR8350.2 to all published digits. Crucially, WGS84 GM (3.986004418e14) and a (6378137.0) DIFFER from the ICGEM conventions (3.986004415e14, 6378136.3): `g.subtractNormalField` rescales the ellipsoid field to the model's (GM, R) first via

$$\bar{C}'_{nm} = \bar{C}_{nm} \frac{GM_1}{GM_2} \left( \frac{R_1}{R_2} \right)^n$$

(`shLowLevel.rescaleGMR / g.toReference`; the physical field is invariant — verified by synthesizing the potential at a common radius from both representations). Skipping the rescaling costs millimetres. The arithmetic operators refuse GM/R-mismatched operands (`shCoefficients.constantsMismatch`) instead of silently mixing conventions. Permanent tide: the ellipsoid is tide-free by construction; the model's C20 tide system (ICGEM header) is NOT converted silently — zero-tide vs tide-free differ by  $\sim 4.2\text{e-}9$  in  $\text{Cbar}_{20}$ .

## 18. Multi-center combination (v2.4)

Several centers observe the same monthly field with different noise. One variance factor per (center, month) — the COST-G granularity — estimated by Foerstner iteration with PARTIAL redundancies:

$$\hat{x} = \left( \sum_c w_c N_c^{-1} \right)^{-1} \sum_c w_c N_c^{-1} x_c, \quad s_c^2 = \frac{v_c^T N_c^{-1} v_c}{r_c}, \quad r_c = P - \text{tr}(H w_c N_c^{-1})$$

Without  $r_c$  the factors bias low ( $v_c$  is correlated with  $x$ ). Noise shapes  $N_c$  come from each center's own climatology residuals as (order, C/S, parity) blocks; the whole iteration runs block-wise. `Robust=true` replaces the global ratio by the median of per-block unbiased estimates  $q_b/r_b$ . The empirical shapes are trace-normalized, so the factors carry each center's absolute noise power and are directly comparable (weights invariant; v2.4.1). Two limits are structural: common-mode errors (shared K-band data, AOD1B, tides) are invisible to inter-center VCE — posterior sigmas are a LOWER bound — and inter-center correlations violate independence; `info.interCenterCorr` quantifies the optimism. Combine GSM before TN-14/TN-13, then apply the replacements once.

## 19. Elastic sea-level fingerprints (v2.4)

A melting ice mass does not raise the ocean uniformly: the ocean surface follows the perturbed geoid minus the deformed crust, under global mass conservation (Farrell & Clark 1976, elastic limit):

$$S = \mathcal{O}(N - U + \kappa), \quad \int_{\text{ocean}} \rho_w S dA = - \int_{\text{land}} \sigma dA$$

`shLowLevel.seaLevelFingerprint` iterates this on the toolbox's exact Gauss–Legendre quadrature pair. Validation: mass conserved to machine precision,  $\sim 11$  iterations, and the classic pattern — sea level FALLS near the melting mass (lost self-attraction plus rebound; validation run: near field -0.3 mm vs far field +0.8 mm about a +0.6 mm eustatic mean per unit load). Elastic only, no rotational feedback, fixed coastlines.

## 20. Gravity gradient tensor (v2.4)

`shLowLevel.shSynthesisGradientTensor` synthesizes all six NEU components at altitude; second Legendre derivatives come from the frozen first-derivative stencil applied twice (exact identity). Every call self-checks the Laplace invariant  $\text{trace}(G) = 0$ , validated at  $7\text{e-}16$  of  $\max|G|$ ;  $G_{uu}$  is bit-consistent with the 'gravity\_gradient\_rr' kernel route.  $1 \text{ Eotvos} = 1\text{e-}9 \text{ s}^{-2}$ .

## Part II — Best practices

### Processing chain order

The order below is not arbitrary; each step assumes the previous ones:

- 1. Read GSM monthlies (shSeries.read, gz transparent, ICGEM 1.0/2.0 incl. gfct).
- 2. Replace C20 (and C30 in the accelerometer-degraded years) from TN-14; add degree-1 from TN-13 (addDegree1). Do this BEFORE filtering — filters see the stripes, not the low-degree completions.
- 3. Bridge the GRACE↔GRACE-FO gap explicitly: dropNaN / select; never fit a climatology across NaNs.
- 4. Filter: tvANS (or DDK for benchmarking; Gaussian as baseline).
- 5. Remove GIA (trend work only).
- 6. Fit climatology with tidal alias periods and ARCorrect=true; or form basin series with deconvolution.
- 7. Synthesize maps / averages; propagate sigmas (analytic where derived, mcPropagate everywhere else).

### Choosing a filter

| situation                       | recommendation                                                                           |
|---------------------------------|------------------------------------------------------------------------------------------|
| global EWH maps, standard use   | tvANS (Blocks="auto"); Gaussian 300 km only as sanity baseline                           |
| cross-paper comparability       | add DDK3-DDK6 variants via readDDK (binary Wbd, Nmax=96) + applyDDK                      |
| strong order-dependent striping | tvANS with VCEBands=[0 16 33 61] (v2.2)                                                  |
| small basins (< ~200,000 km²)   | expect leakage; taper kernels, consider basinDeconvolve with several neighboring kernels |
| regional studies                | Slepian basis instead of filtering a global field (Part III, G4)                         |

### Uncertainty accounting — what is honest and what is approximate

- Formal per-coefficient sigmas (readers) → exact as published.
- tvANS posterior sigmas → exact for the unconstrained AND (v2.5) the constrained filter.
- Basin sigmas → noise part consistent with (banded) VCE and the exact (constrained) operator; deterministic-fit parameter uncertainty included since v2.5 (pointwise; epoch-correlated through the shared fit).
- Climatology sigmas → OLS/weighted/robust; AR(1) correction first-order with Kendall-corrected  $r_1$  (v2.5; residual ~3% optimism from regression absorption).
- GIA → model treated exact; use model spread.
- Multi-center combination → posterior sigmas are a LOWER bound (common mode invisible; see info.interCenterCorr).
- DDK-filtered fields → no analytic sigmas; use mcPropagate.
- Anything else → mcPropagate: independent-sigma or full SINEX covariance sampling validates every analytic formula empirically (the toolbox's own tests do exactly this).

### Validation habits the toolbox is built around

- Every numerical method was validated in Python (numpy/scipy) before the MATLAB implementation; the validation numbers are quoted in the help texts. Keep the habit for your own extensions.
- Ring-grid analysis↔synthesis roundtrips are exact to ~1e-14: use them as a self-test on your grids (Part III, G1 step 5).
- Real provider files beat synthetic fixtures: the test suite auto-discovers any \*.gfc / \*.snx(.gz) you drop into tests/test\_data and sanity-checks them (large SINEX are streamed, v2.2).
- runAllTests is the gate: run it after every toolbox update; perf\_log.csv accumulates machine-local timing history.

### Common silent errors this toolbox guards against

| error                                           | guard                                          |
|-------------------------------------------------|------------------------------------------------|
| geodetic latitudes fed to SH math (~21 km bias) | LatType="geodetic" + conversion helpers (v2.2) |
| hardcoded Love numbers                          | kn is a required explicit input for EWH        |
| climatology across the 2017-2018 gap NaNs       | assertClean errors loudly; dropNaN/select      |

| error                                                   | guard                                                              |
|---------------------------------------------------------|--------------------------------------------------------------------|
| trend maps without GIA                                  | removeGIA one-liner (v2.2)                                         |
| optimistic trend sigmas                                 | ARCorrect=true (v2.2)                                              |
| tidal alias leakage into trends                         | Periods=[161/365.25, 3.66, 7.48]                                   |
| zero-variance rows collapsing the noise floor           | positive-diagonal floor + shLowLevel:buildNoiseCov:allZeroVariance |
| 8-column ICGEM 1.0 acos/asin ambiguity                  | EIGEN-style period read; documented in README                      |
| sine sectorals $S_{nn}$ blanked in difference triangles | fixed v2.4.2; rendered-CData regression test                       |

## Part III — Step-by-step guides

### G1. Monthly EWH anomaly maps from ITSG files

```
% 1. read the monthly GSM series (gz ok, ICGEM 1.0/2.0)
ts = shSeries.read("data/ITSG-Grace2018_n96_*.gfc");

% 2. low-degree completion BEFORE filtering
tn14 = shLowLevel.readTN14("TN-14_C30_C20_SLR_GSFC.txt");
ts = ts.applyTN14(tn14); % C20 (+C30 where flagged)
tn13 = shLowLevel.readTN13("TN-13_GEOC_GFZ_RL06.txt");
ts = ts.addDegree1(tn13);

% 3. bridge the mission gap explicitly
ts = ts.dropNaN();

% 4. anomalies w.r.t. a mean field, then filter
ts = ts - ts.mean(); % anomalies
tsF = ts.filter("tvANS", Blocks="auto"); % or .gaussian(300)

% 5. EWH synthesis (Love numbers user-supplied!)
kn = readmatrix("loadLoveNumbers.txt"); % your set, degrees 0..nmax
lat = -89.5:1:89.5; lon = 0.5:1:359.5;
ewh = tsF.at(24).synthesis(lat, lon, quantity="ewh", kn=kn); % [m]

% self-test: analysis roundtrip on a ring grid should be ~1e-14
gChk = shCoefficients.analysis(ewh, lat, lon, tsF.nmax, ...
    quantity='ewh', kn=kn);
```

Interpretation: values are equivalent water height anomalies relative to the removed mean, in meters. The FFT synthesis path engages automatically on this uniform grid; check `tsF.at(24).history` for the full provenance chain.

### G2. Basin time series with uncertainties

```
idx = shLowLevel.shIndex(ts.nmax, MinDegree=2);
% kernel from a polygon [lat lon], buffered + tapered (v2.2)
amazon = readmatrix("amazon_polygon.txt");
[b, info] = shLowLevel.basinKernel(idx, amazon, BufferKm=150, TaperKm=350);

% filtered series + filter operator
[tsF, op] = ts.filter("tvANS", Blocks="auto", VCEBands=[0 16 33 61]);

% deconvolved basin average with 1-sigma (banded-VCE-consistent, v2.2)
[avg, out] = shLowLevel.basinDeconvolve(b, op);
shLowLevel.plotBasinSeries(op.tYears, avg(1,:), out.sigma(1,:), Units="cm");
```

The deconvolution undoes the filter's attenuation of the kernel; the sigma comes from the posterior covariance with per-band noise factors. For several basins pass a kernel matrix `B` — the joint solve also redistributes mutual leakage. (Edition-2 erratum fixed: the signature is `basinDeconvolve(B, op)`, epochs live in `op.tYears`, averages in the first output.)

### G3. Significance-masked trend maps (GIA- and AR(1)-honest)

```
gia = shCoefficients.read("ICE-6G_D_rates.gfc"); % rate [1/yr]
tsG = tsF.removeGIA(gia);
clim = tsG.climatology(Periods=[161/365.25, 3.66, 7.48], ...
    ARCorrect=true);

tr = clim.trend(); % shCoefficients WITH sigmas
kn = readmatrix("loadLoveNumbers.txt");
rate = tr.synthesis(lat, lon, quantity="ewh", kn=kn); % [m/yr]
mc = shLowLevel.mcPropagate(@(g) g.synthesis(lat, lon, ...
    quantity="ewh", kn=kn), tr, N=300, Seed=1);
sigma = mc.sigma;
mask = abs(rate) > 2 * sigma; % 2-sigma significance
rate(~mask) = NaN; imagesc(lon, lat, rate);
```

Compare against a second GIA model (Caron) and show the difference as the model-spread contribution to the error budget — usually dominant over formal errors in formerly glaciated regions.

### G4. Regional analysis with Slepian functions

```

idx = shLowLevel.shIndex(60, MinDegree=0);
region = @(la,lo) double(la>-90 & la<-60); % Antarctica-ish belt
[G, lam, infoS] = shLowLevel.slepianBasis(idx, region);
fprintf("Shannon %.1f -> keeping %d tapers\n", infoS.shannon, numel(lam));

x = shLowLevel.vecFromCS(g.C, g.S, idx); % one month, idx order
a = G' * x; % Slepian coefficients
xR = G * a; % regional part of x
[Cr, Sr] = shLowLevel.csFromVec(xR, idx);
gR = shCoefficients(Cr, Sr, GM=g.GM, R=g.R);

```

Estimating a (dimension  $\approx$  Shannon number) instead of all P Stokes coefficients is well-posed on regional data by construction; concentrations  $\lambda$  near 1 tell you which tapers the region genuinely constrains.

## G5. Working with released covariances (SINEX)

```

idx = shLowLevel.shIndex(96, MinDegree=2);
% full read (monthly solutions to ~30 MB): covariance in idx order
snx = shLowLevel.readSINEX("ITSG-..._n96_2008-04.snx.gz", ...
    Output="covariance", Index=idx);
N = snx.M; % noise shape for tvANS
[tsF, op] = ts.filter("tvANS", NoiseCov=N, Blocks="off");

% multi-100-MB NEQ SINEX: stream just the estimates in seconds (v2.2)
est = shLowLevel.readSINEX("ITSG-..._operational_n96_2020-01.snx.gz", ...
    Only="estimate");

% propagate the full covariance through ANY functional (v2.2)
out = shLowLevel.mcPropagate(@(gs) b' * shLowLevel.vecFromCS(gs.C,gs.S,idx), ...
    g0, Cov=snx.M, Idx=idx, N=2000);

```

## G6. Comparing against mascons

```

mas = shLowLevel.readMascon("GRCTellus.JPL...MSCNv03CRI.nc");
% mascon lats are GEODETIC -> one switch avoids the 21-km bias (v2.2)
kn = readmatrix("loadLoveNumbers.txt");
k = find(abs(mas.epoch - 2010.29) < 0.05, 1); % match an epoch
ewhSH = tsF.at(k).synthesis(mas.lat', mas.lon', ...
    quantity="ewh", kn=kn, LatType="geodetic"); % [m]
d = 100*ewhSH - mas.ewh(:,k); % mascon units: cm
fprintf("RMS diff %.2f cm\n", sqrt(mean(d(:).^2, 'omitnan')));

```

Expect coastal and small-basin differences: mascons carry their own regularization. Compare basin-integrated series rather than pointwise fields for a fair statement; use the same GIA convention on both sides (JPL mascons ship GIA-corrected — remove GIA from the SH side too).

## G7. GNSS loading comparison (v2.3)

```

% residual monthly series (mean removed), filtered
ts = ts - ts.mean(); % anomalies
tsF = ts.filter("tvANS", Blocks="auto");

% load Love numbers k', h', l' - ONE consistent model, user-supplied;
% v2.5 reader: layouts, sparse pchip interpolation, CF-frame conversion
LN = shLowLevel.readLoveNumbers("loadLoveNumbers_PREM.txt", ...
    Columns="n k h l", MaxDegree=tsF.nmax, Interp="pchip", ...
    InFrame="CE", OutFrame="CF"); % match TN-13 (CF)
kn = LN.kn; hn = LN.hn; ln = LN.ln;

% GNSS stations (geodetic coordinates!)
sta = readmatrix("stations.txt"); % [lat lon]
T = tsF.nEpochs;
up = zeros(size(sta,1), T); no = up; ea = up;
for k = 1:T
    [up(:,k), no(:,k), ea(:,k)] = tsF.at(k).deformation( ...
        sta(:,1)', sta(:,2)', kn=kn, hn=hn, ln=ln, ...
        Mode="points", LatType="geodetic");
end
% compare up(i,:) against the GNSS vertical of station i (same epochs);
% typical GRACE-elastic amplitudes: mm to ~1 cm vertical, ~1/3 horizontal

```

Degree 1 is excluded by default; include it only when both the GRACE side (TN-13 in the CF frame) and the Love numbers use the same frame convention. The elastic prediction excludes GIA and poroelastic effects — detrend both series or remove GIA explicitly before comparing trends.

## G7b. Data management (v2.4.1, temporal catalogue v2.4.2)

```
shLowLevel.dataFolder("D:/geodata/shLowLevel");      % once; persists (getpref)
shLowLevel.fetchDDK(1:8);                            % all released DDK filters
W5 = shLowLevel.readDDK("DDK5");                     % by name, from the data folder
T = shLowLevel.listICGEM();                           % 180+ static models as a table
f = shLowLevel.fetchICGEM("EGM2008");                % -> dataFolder/icgem/EGM2008.gfc
g = shCoefficients.read(f);
shLowLevel.fetchITSG(2010:2016);                      % -> dataFolder/itsg_series
shLowLevel.fetchITSG("2008-04", Product="daily");    % Kalman daily n40 (v2.4.1)

% FULL temporal catalogue: all ~70+ series, all centers
Tt = shLowLevel.listICGEM(Type="temporal");
% fetch a WHOLE monthly series (v3.1.1) - one row of that catalogue
fs = shLowLevel.fetchICGEM(17, Type="temporal");
ts = shSeries.fromFolder(fileparts(fs(1)));          % straight into a series
```

The ICGEM static catalogue is parsed from [icgem.gfz.de](http://icgem.gfz.de) (fixture-tested offline). The temporal catalogue lists every series of every center — groups 01\_GRACE, 02\_COST-G, 03\_other, 04\_SLR — with columns group, center, series, path, url and zip. Since v3.1.1 a temporal row is not just browsable but fetchable: `fetchICGEM(idx, Type="temporal")` downloads the whole series into `<dataFolder>/icgem/series/<group>_<center>_<series>/`, which `shSeries.fromFolder` and `shLowLevel.standardChain` consume directly. For ITSG monthlies `shLowLevel.fetchITSG` remains the convenient route.

### Mode: one request, not three hundred

The default `Mode="auto"` takes the server's whole-series ZIP in a SINGLE request and unpacks it. This is not an optimization but a courtesy requirement: [icgem.gfz.de](http://icgem.gfz.de) rate-limits, and fetching a 283-file series one file at a time earns HTTP 429 and a tarpit (field-observed as "too many connections"). If the archive is unavailable the fetcher falls back to per-file mode automatically — resumable, throttled with `Pause=` and `Retries=` exponential backoff, each file verified by parse before it replaces anything. `Mode="archive"|"files"` force either path; `info.mode` reports which one actually ran, so a script can tell a fresh download from a no-op. `Files=` filters the series, and `FileList=` injects a catalogue table for offline mirrors and subsets.

```
% forced per-file, filtered to a window, gentle on the server
fs = shLowLevel.fetchICGEM(17, Type = "temporal", Mode = "files", ...
    Files = "*2008*.gfc", Pause = 3, Retries = 3);
[ts, rep] = shLowLevel.standardChain(fileparts(fs(1)), Filter = "DDK3");
```

## G8. Multi-center combination (v2.4)

```
shLowLevel.fetchITSG(2010:2016);                      % download once (websave)
tsI = shSeries.fromFolder("ITSG/");                  % GSM level, same nmax
tsC = shSeries.fromFolder("CSR/");
tsG = shSeries.fromFolder("GFZ/");
[tsComb, info] = shLowLevel.combineCenters({tsI, tsC, tsG}, Robust=true);

figure; plot(info.epochs, info.weights');             % weight time series
legend(info.centers); ylabel("relative weight");
figure; imagesc(info.interCenterCorr);               % honesty diagnostic
% THEN degree-1 / C20-C30 replacements, once:
tsComb = tsComb.applyTN14("TN-14.txt");
tsComb = tsComb.addDegree1("TN-13.txt");
```

Watch the weight series for regime changes (single-accelerometer era, GRACE-FO transition) and treat posterior sigmas as lower bounds — the common mode is invisible.

## G9. Sea-level fingerprint (v2.4)

```

idx = shLowLevel.shIndex(96, MinDegree=0);
LN = readmatrix("loadLoveNumbers.txt"); kn = LN(:,2); hn = LN(:,3);
greenland = @(la,lo) -280 * inGreenland(la, lo); % kg/m^2/yr, say
oceanF = @(la,lo) oceanMaskFun(la, lo); % 1 over ocean
[S, grid, info] = shLowLevel.seaLevelFingerprint(greenland, oceanF, idx, ...
    kn=kn, hn=hn);
shLowLevel.plotSHMap(info.S2D / info.eustatic, grid.latDeg, grid.lonDeg, ...
    Units="S / eustatic", Title="Greenland fingerprint");

```

Plot S normalized by the eustatic mean: near-field values drop below zero, far-field plateaus around 1.1–1.3 — if not, check the ocean mask and that idx uses MinDegree = 0.

## G10. The standard chain in one call

```

% the whole post-processing chain, in the ONE correct order
gia = shCoefficients.read("ICE-6G_D_trend.gfc"); % a TREND field [1/yr]
[ts, rep] = shLowLevel.standardChain("D:/grace/monthly", ...
    TN14File = "TN-14_C30_C20_SLR_GSFC.txt", ... % TN14 itself is a FLAG
    Degree1 = "CSR", ... % provider, not a path
    GIA = gia, Filter = "DDK3");
disp(rep.steps') % what was applied, in order, with provenance

```

Order is not a matter of taste here. The SLR C20/C30 replacement and the degree-1 restoration must happen on the UNFILTERED coefficients — a filter mixes degrees, so replacing a coefficient afterwards inserts an unfiltered value into a filtered field and the low degrees no longer mean anything consistent. GIA is a trend in the same (corrected) coefficients, so it comes next; filtering is last. `standardChain` encodes exactly that order and returns REP, a provenance record naming every step, the files it used and the toolbox version that ran — the thing you paste into a paper's data section. Every stage is optional: leave GIA= out and the report says so, rather than silently skipping a step you assumed had happened.

Two contracts worth reading before the first call: TN14 is a logical FLAG and the path goes in TN14File, while Degree1 names the TN-13 provider ("GFZ" | "CSR" | "JPL" | "none") with the path in Degree1File; and GIA takes an shCoefficients TREND field in [1/yr], not a filename — it is applied as  $(t - \text{GIAEpoch}) \times \text{GIA}$ . Left at their defaults the files come from the persistent data folder that `setup_shAnalysis` populates.

## G11. Designing your own anisotropic filter

```

% a DDK-class filter built from YOUR error covariance
g = shCoefficients.read("ITSG-Grace2018_n60_2008-04.gfc");
W = shLowLevel.designFilter(g.sigmaC, g.sigmaS, Kaula = 1e-5);
[Cf, Sf] = shLowLevel.applyDDK(g.C, g.S, W); % same block format as readDDK

```

The released DDK filters are built from a specific center's normal equations. If you have your own sigmas or a released covariance, `designFilter` builds the same kind of operator,  $W = (N + a S^{-1})^{-1} N$ , in the block format `readDDK` produces — so it drops into `applyDDK`, `g.applyDDK` and `standardChain(Filter=W)` unchanged. Note that `Noise=` expects a COVARIANCE, not sigmas: passing standard deviations where a variance is expected is the classic way to get a filter that looks plausible and smooths by the square root of what you intended.

## G12. Correcting leakage

```

% forward modelling: only the FILTER has to be known
ewh = g.synthesis(-89:89, 0:359, quantity = "ewh", kn = kn);
[m, info] = shLowLevel.leakageCorrect(ewh, -89:89, 0:359, ...
    Filter = "gauss300", Mask = basinMask, Gain = 2);

% or scaling factors, if you trust a model's spatial PATTERN
k = shLowLevel.gridScaling(modelEWH, -89:89, 0:359, Filter = "gauss300");
corrected = k .* ewh; % NaN where the model is blind

```

A filter removes real signal and spreads the rest across basin boundaries; a 6° disc under a 500 km Gaussian keeps about half its peak. The two corrections differ in what you have to BELIEVE, which is how to choose between them. Forward modelling (`leakageCorrect`) needs only the filter and, usefully, a mask saying where mass can exist — it then has to explain the observation with mass inside that region, so leakage outside is removed rather than redistributed. Scaling factors (`gridScaling`) need a model series, and inherit the correctness of its spatial pattern: the factors are invariant to the model's amplitude but not to its shape.

Two behaviours worth knowing before you read the output. Convergence is judged on the change of the SOLUTION, not on the residual: with a mask the problem is inconsistent, so the residual settles at a floor while the

solution is converged. And `gridScaling` returns NaN where the model carries no signal instead of a ratio of two numerical zeros — a model that does not reach your basin shows up as missing coverage rather than as noise multiplying your data.

### G13. What real provider files look like

Format specifications describe what files ought to contain. The reader is written against what they actually contain, and three separate field failures are worth knowing about, because they are silent — you get numbers, just not the right ones.

- **FORTTRAN D-exponents.** Providers switch from `1.23E-10` to `1.23D-10` partway through large files (EIGEN-6C4 does it above degree ~370). MATLAB's `str2double` returns NaN for those, so a line-by-line parser quietly corrupts the high degrees of exactly the models where the high degrees are the point. The reader normalizes D/d to E/e before conversion.
- **ICGEM 2.0 column order.** The real-world layout of an `acos/asin` line is `... sigC sigS t0 t1 period` — period LAST, verified against CNES/GRGS files. A period-first assumption mis-parses every such file and produces a time-variable model whose seasonal terms are wrong without ever raising an error.
- **Ragged record groups.** EIGEN-5S and 5C carry a single `gfc` line with a trailing epoch among thousands of uniform ones. Bulk `sscanf(txt, '%f', [nc Inf])` does not complain about that — it silently re-flows the whole group by one column and asks for a coefficient matrix indexed by a date. The reader checks `rows-out == lines-in` per group, subgroups by numeric width when ragged, and keeps an n/m sanity net.

The practical rule: when a new provider or a new release enters your pipeline, read one file and check `g.C(3,1)` and the highest degrees against the provider's own published values before you trust a thousand of them. Bulk parsing is fast — EIGEN-6C4 at 177.7 MB and degree 2190 in about 5 s, a 73.6 MB GRGS mean field with 674k variable terms in about 7 s — but fast and correct are different properties.

# Validation against GravIS

This chapter is a worked, reproducible comparison of the toolbox against a PUBLISHED product: the GravIS Greenland ice-mass basin averages (<https://gravis.gfz.de/gis>). It is written out in full because the numbers only mean something together with the choices behind them — and because two of those choices cost a factor that would otherwise have been mistaken for a bug in the toolbox.

## V1. The reference

GravIS publishes seven Greenland drainage basins as ASCII time series in Gt. Summing them and fitting bias, trend, annual and semi-annual gives the reference trend. It must be recomputed over the SPAN YOU ACTUALLY USE: over the full record COST-G RL02 gives  $-220.8$  Gt/yr, but over the shorter span for which the auxiliary correction tables were available it is  $-231.1$  Gt/yr. Comparing against the wrong span is a 5% error before any processing starts.

## V2. Matching the GravIS processing

GravIS Level-3 basin averages come from Level-2B coefficients, which are Level-2 plus four corrections (<https://gravis.gfz.de/corrections>). Applying anything less makes the comparison meaningless. All four are read with `shLowLevel.readSHM` or as plain tables:

```
% (1) their low-degree replacement: C20, C30, C21, S21
% (2) their geocenter (Swenson approximation, NOT TN-13)
% (3) their mean field, 2002/04-2020/03
% (4) GIA: ICE-6G_D (VM5a), a GRDOTA rate model in 1/yr
Mn = shLowLevel.readSHM("GRAVIS-2B_..._MEAN_2002095-2020091_NFIL.gz", Nmax = 90);
Gi = shLowLevel.readSHM("GRAVIS-2B_..._GIA_ICE-6G_D_VM5a.gz", Nmax = 90);

ts = shSeries.fromFolder(costgFolder);
ts = ts.select(ts.epochs <= maxTableEpoch + 0.05); % tables trail the data
% ... insert their C20/C30/C21/S21 and C10/C11/S11 per epoch ...
for k = 1:ts.nEpochs % mean field and GIA, per epoch
    Cs(:, :, k) = Cs(:, :, k) - Mn.C - (ts.epochs(k) - 2011) * Gi.C;
    Ss(:, :, k) = Ss(:, :, k) - Mn.S - (ts.epochs(k) - 2011) * Gi.S;
end
ts = ts.removeAlias(); % (5) the S2 161-day alias
```

## V3. The basin average

```
idx = shLowLevel.shIndex(ts.nmax, MinDegree = 0);
[b, info] = shLowLevel.basinKernel(idx, gisPolygon); % NaN-separated rings
area = info.areaFraction * 4*pi*6371^2; % NOT b(1)
kf = shLowLevel.kernelFactors("ewh", ts.nmax, GM, R, kn = kn);
avg = (b .* kf(idx.n+1))' * X / (b'*b); % EWH [m] per epoch
mass = avg(:) * area * 1e6 * 1000 / 1e12; % -> Gt
```

## V4. Results, and what each step is worth

| Method                                        | Gt/yr         | vs published |
|-----------------------------------------------|---------------|--------------|
| naive kernel average                          | -196.4        | -15%         |
| naive grid integral                           | -170.3        | -26%         |
| forward modelling, <b>Greenland-only</b> mask | -259.8        | +12%         |
| forward modelling, <b>union</b> mask          | -240.2        | +4%          |
| union mask + S2 alias removed                 | -240.0        | +4%          |
| <b>GravIS COST-G RL02, matching span</b>      | <b>-231.1</b> | —            |

### V4b. Describe the filter the FORWARD MODEL sees

The table above hides the single largest error I made. GravIS basin averages start from UNFILTERED Level-2B coefficients, so I passed `Filter="none"` to `LeakageCorrect`. But Dahle et al. (2025), Sect. 2.2.1, list the basin-average steps: spectral masking, then **low-pass filtering with a Wiener optimal filter, approximately a Gaussian of 4° latitude spatial half-width**, then the conversion to surface mass, then the least-squares adjustment. The forward model is filtered IDENTICALLY to the observations — that is the whole point of the method

— so the filter to declare is the Wiener one, not "none".

```
r = 4 * 111.195; % 4 deg latitude ~ 445 km
wG = shLowLevel.shGaussianWeights(nmax, r);
obs = shCoefficients(Ct .* wG(:), St .* wG(:)).synthesis( ...
    lat, lon, quantity = "ewh", kn = kn, nmin = 0);
m = shLowLevel.leakageCorrect(obs, lat, lon, Filter = "gauss445", ...
    Mask = unionMask, Gain = 2, MaxIter = 300);
```

| Declared filter                          | Gt/yr         | vs published |
|------------------------------------------|---------------|--------------|
| "none" (wrong)                           | –234.9        | +1.6%        |
| "gauss445" (Wiener equivalent)           | –227.4        | –1.6%        |
| <b>GravIS COST-G RL02, matching span</b> | <b>–231.1</b> | —            |

Declaring the filter correctly turns a 1.6% overshoot into a 1.6% undershoot, and the iteration also drifts less with continued iteration (2.0 Gt/yr from 300 to 900 steps, against 2.5 unfiltered) because the filter regularises the inverse problem. At that point the agreement is as close as this reproduction can meaningfully be read: the remaining difference is smaller than the spread between GravIS product versions and smaller than the effect of the stopping point.

The paper also specifies the mask GravIS uses, which is NOT the bare drainage-basin outline: the spectral mask is 1 until 200 km outside the grounding line and tapers smoothly to 0 by 600 km. That is `basinKernel(..., BufferKm = 200, TaperKm = 400)`, not a hard boundary.

## V4c. The regularised number, and the noise it needs

Every figure above was produced by stopping the iteration at a fixed count, which is not a criterion. Rerun with the discrepancy principle, two further things have to be right — and getting either wrong is worse than the leakage it was meant to fix.

- **The residual must be measured where the model is responsible.** With a Greenland mask the model describes one region while the data contains Antarctica, Alaska and global hydrology, so the GLOBAL residual never falls to the noise level: 0.0085 globally against 0.0008 near the mask, with a noise level of 0.0024. The principle silently never fires and the run stops at MaxIter. `ResidualRegion` (default: the mask) fixes this.
- **The noise level must match the quantity being inverted.** The open-ocean RMS of a TREND field is 0.0024 m/yr — but most of that is real barystatic sea-level rise, not error. Using it stops the iteration after 4 steps and gives –201.0 Gt/yr, 13% low. The trend's actual noise is the monthly noise propagated through the fit,  $\sigma_{\text{monthly}} / \sqrt{S_{xx}} = 0.0115 / \sqrt{8353} = 0.000125$  m/yr — nineteen times smaller.

```
% monthly noise from the residuals about the deterministic fit ...
sigMon = shLowLevel.oceanRMS(residGrid, lat, lon, openOcean);
% ... propagated to the TREND: sigma_trend = sigma_monthly / sqrt(Sxx)
sigTrend = sigMon / sqrt(sum((epochs - mean(epochs)).^2));
[m, info] = shLowLevel.leakageCorrect(trendGrid, lat, lon, ...
    Filter = "gauss445", Mask = unionMask, NoiseLevel = sigTrend);
info.stoppedBy % "discrepancy", after 82 iterations
```

| Stopping rule                                   | Gt/yr         | vs published |
|-------------------------------------------------|---------------|--------------|
| fixed 300 iterations (no criterion)             | –227.4        | –1.6%        |
| discrepancy, trend-field oceanRMS (wrong noise) | –201.0        | –13%         |
| <b>discrepancy, propagated trend noise</b>      | <b>–224.6</b> | <b>–2.8%</b> |
| <b>GravIS COST-G RL02, matching span</b>        | <b>–231.1</b> | —            |

The regularised answer is 2.8% from the published one, slightly further than the 1.6% obtained by stopping at an arbitrary iteration count. That is the honest outcome: the closer number was a coincidence of where the iteration happened to be, and a result that depends on an arbitrary choice is not a result. This one follows from a stated criterion and a noise estimate that can be recomputed.

## V5. The lessons

**The mask must cover every region that can hold mass, not only the one you are measuring.** A

Greenland-only mask tells the inversion that mass exists nowhere else, so Canadian Arctic, Iceland and Svalbard signal is forced into Greenland: +12% instead of +4%. In the union solution the neighbours absorb –112 Gt/yr —

mass the first mask had nowhere to put. The union mask also converges faster. This is a usage rule, not a bug: Mask= always accepted a union.

**The filter you declare must be the filter the FORWARD MODEL sees, not the filter the input file carries.** The inputs here are genuinely unfiltered, and declaring that was still wrong, because the method filters both sides. This cost more than the mask, the polygon resolution and the alias put together.

**The S2 alias removal is worth 0.2 Gt/yr on a twenty-year trend — that is, nothing.** A 161-day harmonic is very nearly orthogonal to a linear trend over two decades, so removing it barely moves the trend even though it is a real and necessary correction for the MONTHLY series. Reported here because it was expected to explain part of the residual and did not: a hypothesis removed is worth as much as one confirmed.

## V6. What the remaining 4% is, honestly

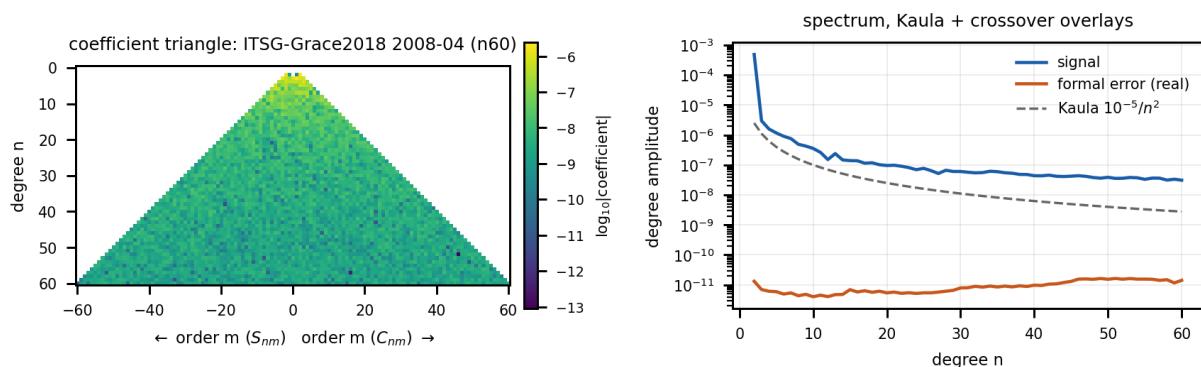
- GravIS inverts SEVEN drainage basins jointly (Sasgen et al. 2012); this recipe constrains one region against coarse neighbour boxes.
- The basin outline used here is decimated to 97.4% of the published area, and the neighbour outlines are rectangles.
- The iteration has not converged: step  $3.2\text{e-}4$  against Tol  $1\text{e-}4$  after 900 iterations, and the estimate was still drifting ( $-240.2$  at 300,  $-242.5$  at 900). Call it  $\pm 5$  Gt/yr.
- Nothing here was tuned to close the gap. A 4% agreement with an independently produced product, reached by matching its documented processing step for step, is the result; making it smaller by adjusting a mask until the number matched would not be.

## Demo gallery

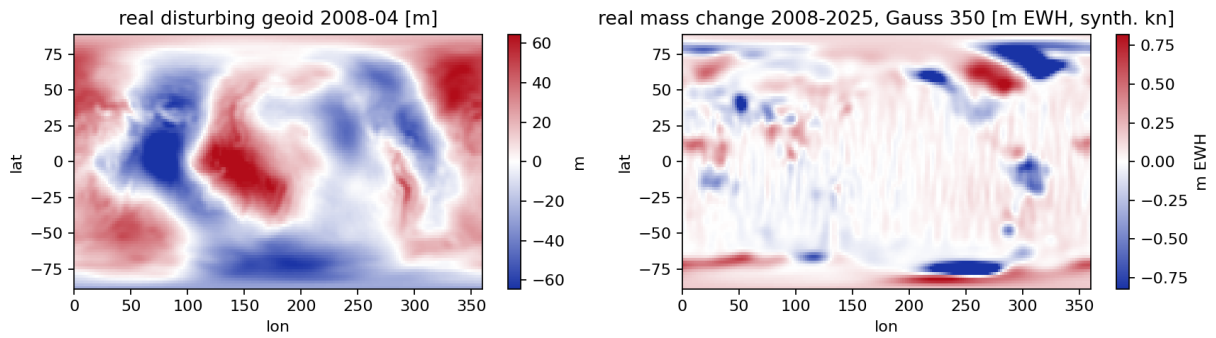
`demo_shAnalysis` is a case registry: `demo_shAnalysis("list")` prints the table below, `demo_shAnalysis("all")` runs everything, `demo_shAnalysis(["D04", "D15"])` runs a selection, and `Visible=false` keeps figures hidden for smoke tests. Real files in `tests/test_data` (ITSG month, DDK3 Wbd) are used when present; every case falls back to synthetic data. Love numbers in the demos are SYNTHETIC — real work requires a real loading model.

| ID  | Case                        | Exercises                                                                                                        |
|-----|-----------------------------|------------------------------------------------------------------------------------------------------------------|
| D01 | Read & spectral diagnostics | <code>shReadGFC</code> , <code>shDegreeRMS</code> , <code>triangle</code> , <code>spectrum</code>                |
| D02 | Synthesis quantities & maps | <code>shSynthesis</code> , <code>kernelFactors</code> , <code>plotSHMap</code> , <code>Height</code>             |
| D03 | Normal field → geoid        | <code>normalFieldCS</code> , <code>subtractNormalField</code> , <code>toReference</code>                         |
| D04 | Filter comparison           | <code>gaussian</code> , <code>fan</code> , <code>destripe</code> , <code>DDK</code> , <code>triangle diff</code> |
| D05 | Climatology & basin series  | <code>climatology</code> , <code>basinKernel</code> , <code>plotBasinSeries</code>                               |
| D06 | tvANS pipeline              | <code>buildNoiseCov</code> , <code>tvANSFilter</code> , <code>basinDeconvolve</code>                             |
| D07 | Analysis (inverse)          | <code>shAnalysisGrid</code> , <code>Kaula</code>                                                                 |
| D08 | Basin tools                 | <code>basinKernel taper</code> , <code>deconvolve vs scaling</code>                                              |
| D09 | Uncertainty                 | <code>errorMap</code> , <code>mcPropagate</code> , <code>plotCovariance</code>                                   |
| D10 | Load deformation            | <code>shSynthesisDeformation</code> , <code>Mode=points</code>                                                   |
| D11 | Gradient tensor             | <code>shSynthesisGradientTensor</code>                                                                           |
| D12 | EOF analysis                | <code>eofAnalysis</code>                                                                                         |
| D13 | Trend breakpoints           | <code>trendBreaks</code> , <code>F-test</code>                                                                   |
| D14 | Multi-center combination    | <code>combineCenters</code> , <code>VCE weights</code>                                                           |
| D15 | Sea-level fingerprint       | <code>seaLevelFingerprint</code> , <code>evalMask</code>                                                         |
| D16 | Export                      | <code>writeGrid netCDF</code> , <code>writeAnimation MP4</code>                                                  |

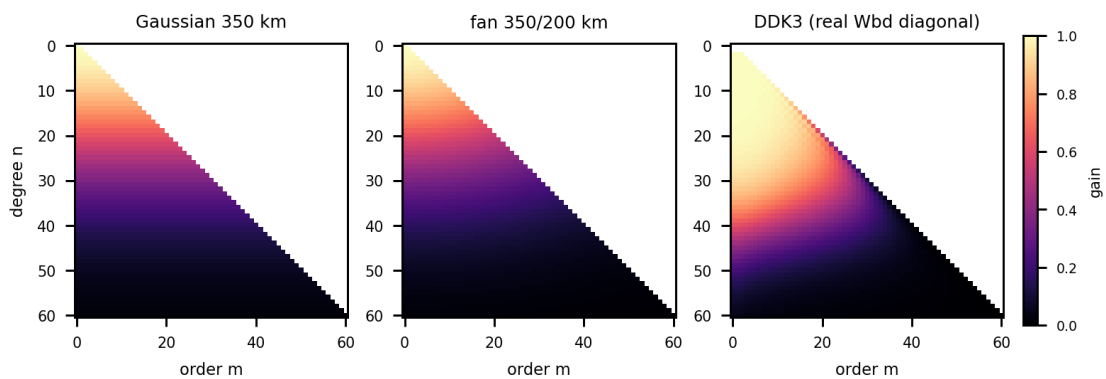
**Note.** Provenance of the figures below: rendered with the toolbox's Python validation port — the same algorithms that gate every MATLAB implementation, numerically equivalent by construction. D01/D02 use the REAL ITSG files shipped in `tests/test_data` (and D04 the real DDK3 Wbd binary); the remaining figures use synthetic data with known ground truth, because recovery demonstrations (VCE factors, EOF modes, F-test calibration) require a truth to recover. `shLowLevel.fetchITSG(years)` extends the real-data demos to full monthly series. The MATLAB demo cases produce the interactive originals.



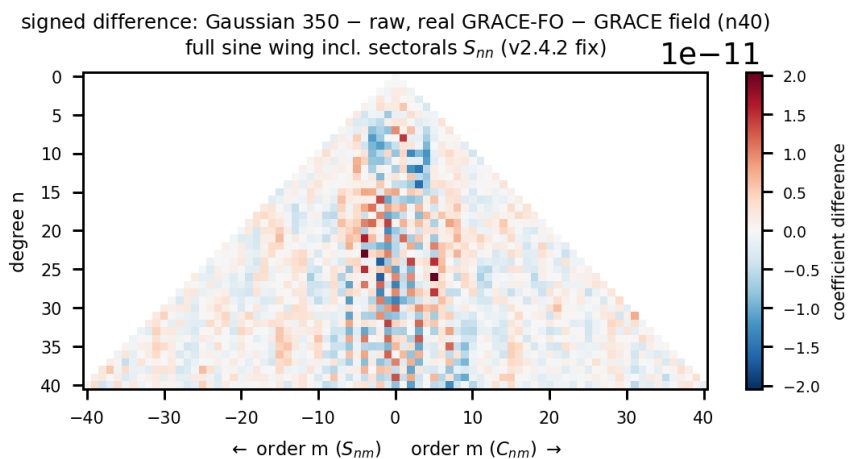
**D01 — REAL DATA:** the shipped ITSG-Grace2018 2008-04 month (rendered by the rebuilt Python port, v2.5). Coefficient triangle (apex up; C00/C20 removed for display) and the degree-amplitude spectrum with the REAL formal errors from the `gfc` file. At this n60 truncation the formal errors stay ~3 orders below the signal - the signal/error crossover (the resolution limit) lies beyond the truncation degree; Kaula decay as the yardstick.



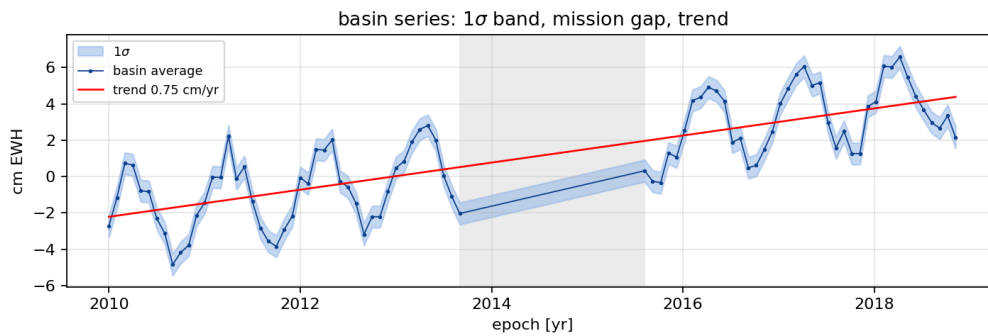
**D02 — REAL DATA.** Left: the disturbing geoid of 2008-04 after subtracting the WGS84 normal field computed from defining constants and rescaled to the ITSG (GM, R) — the classic  $\pm 100$  m map (Indian Ocean low, New Guinea high). Right: the real 2008-2025 mass change (GRACE-FO minus GRACE, Gaussian 350 km, EWH): Greenland and West Antarctic mass loss stand out; residual meridional stripes at low latitudes are the real GRACE error structure. Honest caveats: synthetic Love numbers, no GIA correction, degree 1 absent — patterns are real, amplitudes approximate.



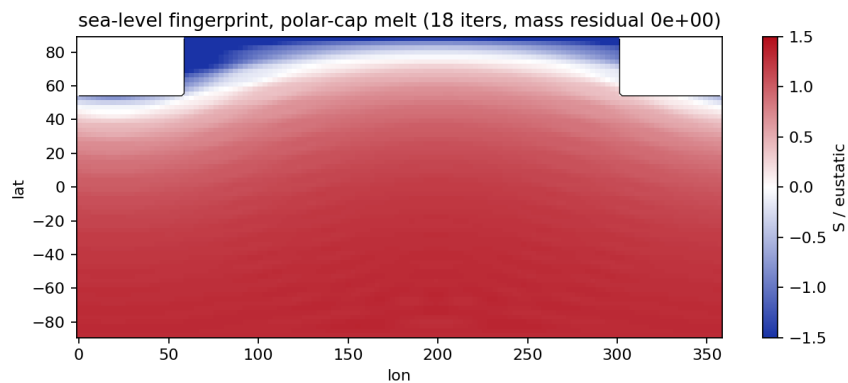
**D04 — Filter gain over the (degree, order) triangle.** Gaussian: isotropic in degree. Fan (Han): the order Gaussian additionally damps high orders — the striping direction. DDK3 (diagonal of the real Wbd blocks): anisotropic and order-dependent — near-zonal signal survives to  $n \sim 35$  while high orders are suppressed hard; exactly the structure a GRACE error covariance demands.



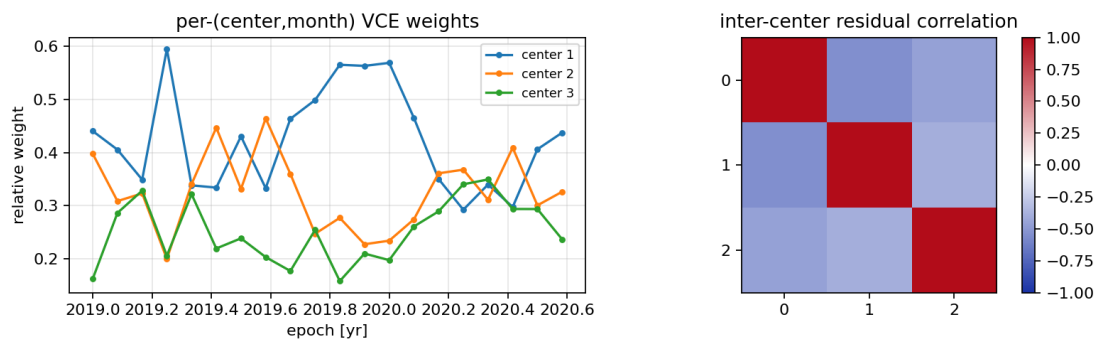
**D04b — REAL DATA, v2.4.2 fix:** the signed difference triangle (Gaussian 350 – raw) on the real GRACE-FO – GRACE field at  $n40$  — the removed signal concentrates at the low orders where the real stripes and secular signal live. The sine wing now spans the full  $n \geq m \geq 1$  support including the sectorals  $S_{nn}$ ; before v2.4.2 the difference mode blanked the sectoral edge and every diff triangle rendered the left wing one column narrower than the right (regression-tested on the rendered CData since).



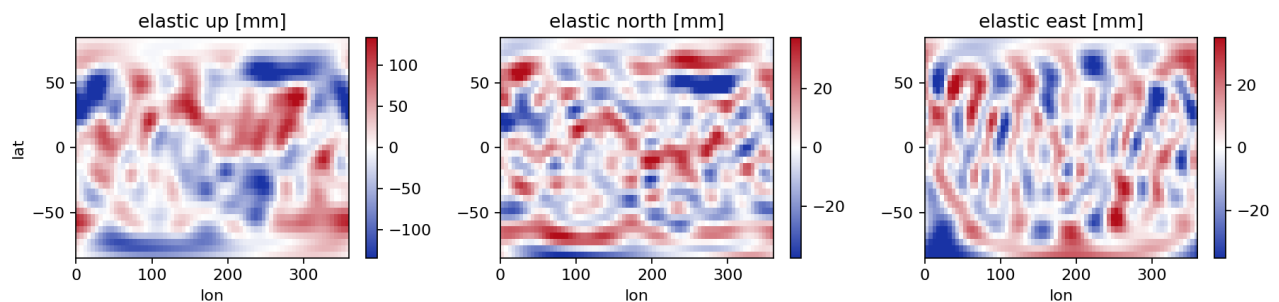
**D05/D06 — The standard basin figure** from `plotBasinSeries`: 1-sigma band, grey mission-gap patch, AR(1)-corrected trend annotation. With `tvANS` + `basinDeconvolve` the band comes from propagated posterior covariances instead of an empirical scatter.



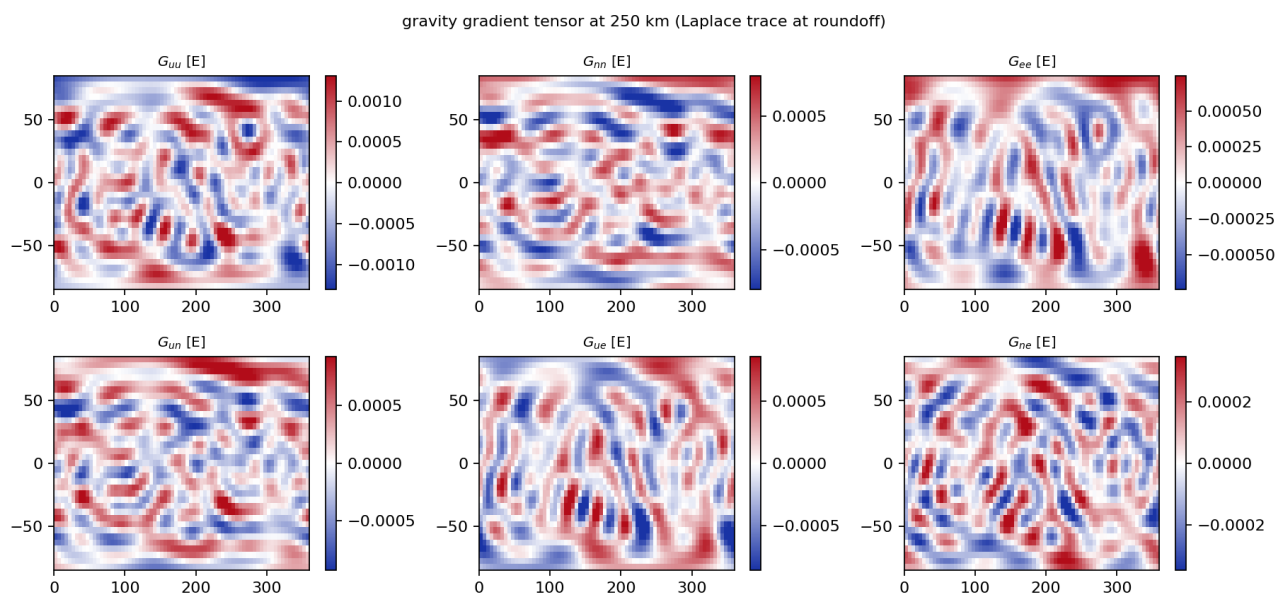
**D15 — Sea-level fingerprint** of a polar-cap melt, normalized by the eustatic mean: the near field FALLS (blue; lost self-attraction plus crustal rebound), the far field exceeds eustatic by ~20-30%. Mass conserved to machine precision — the printed residual is the built-in check.



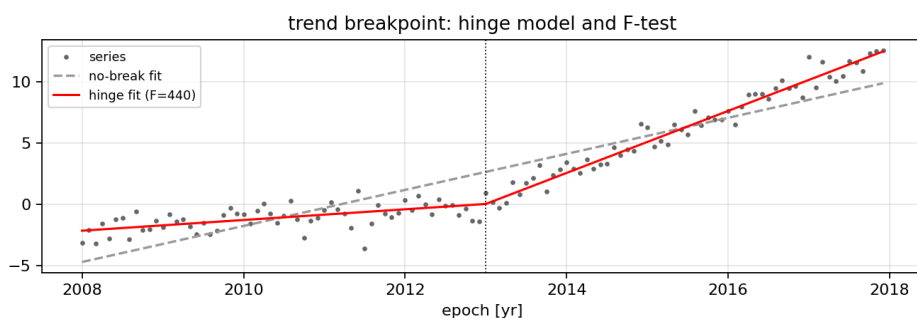
**D14 — Multi-center VCE**: per-(center, month) weights track the time-varying noise levels; the inter-center residual correlation matrix is the honesty diagnostic — VCE assumes independence, GRACE centers share data and background models.



**D10 — Elastic load deformation**: up, north, east from the same residual field. Horizontals are ~1/3 of the vertical (validation run: 0.38) and require the exact SH gradient (`legendreALFDeriv`); east is undefined at the poles.



**D11 — All six gradient tensor components** at 250 km in Eotvos. The Laplace invariant  $G_{nn} + G_{ee} + G_{uu} = 0$  holds at roundoff ( $4e-15$  of  $\max|G|$  here) — a full-chain check of first and second Legendre derivatives, trig derivatives and frame terms at once.



**D13 — Trend breakpoint:** continuous piecewise-linear hinge model vs the no-break fit; the F-test (p via betainc, base MATLAB) quantifies the break. Null rejection is calibrated at the nominal level (5.1% at 5%).



## Part IV — API reference

Generated from the source: the **arguments blocks are the ground truth** for input names, dimensions, classes and defaults (default 'required' = mandatory positional input); name-value options are passed as Name=Value. Conventions throughout: coefficient triangles are addressed C(n+1, m+1); vectors follow shLowLevel.shIndex ordering (MinDegree = 2 unless stated); latitudes are geocentric degrees; radii in km; epochs in decimal years. Every entry closes with a real-data example from the canonical GRACE chain; all examples are lint- and API-gated at build time.

### shCoefficients class

Spherical harmonic (Stokes) coefficient set - single epoch. The single point of access for one gravity field: a GRACE/GRACE-FO monthly solution (GSM), a dealiasing product (GAA/GAB/GAC/GAD), a static model, or any derived field (difference, filtered, climatology component). Immutable value class: every operation returns a NEW object and appends to .history (full provenance).

| property      | size  | type   | default           | notes |
|---------------|-------|--------|-------------------|-------|
| C             | —     | double | —                 |       |
| S             | —     | double | —                 |       |
| sigmaC        | —     | double | []                |       |
| sigmaS        | —     | double | []                |       |
| GM            | 1 x 1 | double | 3.986004415e14    |       |
| R             | 1 x 1 | double | 6378136.3         |       |
| epoch         | 1 x 1 | double | NaN               |       |
| productType   | 1 x 1 | string | "unknown"         |       |
| tideSystem    | 1 x 1 | string | "unknown"         |       |
| name          | 1 x 1 | string | ""                |       |
| header        | —     | struct | struct()          |       |
| variableTerms | —     | —      | []                |       |
| history       | —     | string | string.empty(0,1) |       |
| nmax          | —     | —      | —                 |       |

### shCoefficients.shCoefficients

```
obj = g.shCoefficients(C, S, Name=Value)
```

Construct from coefficient matrices.

| input         | size  | type   | default           | kind       |
|---------------|-------|--------|-------------------|------------|
| C             | —     | double | 0                 | positional |
| S             | —     | double | 0                 | positional |
| SigmaC        | —     | double | []                | name-value |
| SigmaS        | —     | double | []                | name-value |
| GM            | 1 x 1 | double | 3.986004415e14    | name-value |
| R             | 1 x 1 | double | 6378136.3         | name-value |
| Epoch         | 1 x 1 | double | NaN               | name-value |
| ProductType   | 1 x 1 | string | "unknown"         | name-value |
| TideSystem    | 1 x 1 | string | "unknown"         | name-value |
| Name          | 1 x 1 | string | ""                | name-value |
| Header        | —     | struct | struct()          | name-value |
| VariableTerms | —     | —      | []                | name-value |
| History       | —     | string | string.empty(0,1) | name-value |

| output | description                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------|
| obj    | (1 x 1) shCoefficients immutable coefficient set with GM (3.986004415e14), R (6378136.3), epoch, sigmas, history |

*Example*

```
g = shCoefficients(C, S, GM = 3.986004415e14, R = 6.3781363e6);
```

### shCoefficients.plus

```
out = g.plus(a, b)
```

Coefficient addition (e.g. GSM + GAD background restore).

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| a     | —    | —    | required | positional |
| b     | —    | —    | required | positional |

| output | description                                               |
|--------|-----------------------------------------------------------|
| out    | (1 x 1) shCoefficients sum; sigmas RSS; epochs must match |

*Example*

```
gsm = g + gad; % epoch-checked restore
```

### shCoefficients.minus

```
out = g.minus(a, b)
```

Coefficient difference (e.g. monthly change gB - gA).

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| a     | —    | —    | required | positional |
| b     | —    | —    | required | positional |

| output | description                                   |
|--------|-----------------------------------------------|
| out    | (1 x 1) shCoefficients difference; sigmas RSS |

*Example*

```
dg = g - gMean; % anomaly about the mean field
```

### shCoefficients.uminus

```
out = g.uminus(a)
```

Unary minus: negate all coefficients.

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| a     | —    | —    | required | positional |

| output | description                                               |
|--------|-----------------------------------------------------------|
| out    | (1 x 1) shCoefficients negated coefficients (sigmas kept) |

*Example*

```
gNeg = -g;
```

### shCoefficients.mtimes

```
out = g.mtimes(a, b)
```

Scalar scaling.

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| a     | —    | —    | required | positional |
| b     | —    | —    | required | positional |

| output | description                                              |
|--------|----------------------------------------------------------|
| out    | (1 x 1) shCoefficients scalar-scaled; sigmas scale by  a |

*Example*

```
gHalf = 0.5 * g; % scalar scaling, sigmas follow
```

### shCoefficients.compare

```
[rep, h] = g.compare(other, varargin)
```

Full comparison against another solution (v2.6.0). REP = g.compare(G2, ...) delegates to shLowLevel.compareSolutions with OBJ as the first solution; all options pass through (Plot=true for the 4-panel figure).

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| other    | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                                       |
|--------|---------------------------------------------------|
| rep    | (1 x 1) struct shLowLevel.compareSolutions report |
| h      | (1 x 1) graphics handle figure (Plot = true)      |

*Example*

```
rep = g.compare(gF, Names = ["raw", "G350"]);
```

### shCoefficients.times

```
out = g.times(a, b)
```

Elementwise scaling .\* (scalar or triangle matrix). W .\* G scales every coefficient by the matching entry of the (nmax+1 x nmax+1) matrix W (per-coefficient weighting or tapering); a scalar factor delegates to mtimes. Sigmas scale by |factor| elementwise (v2.5.1).

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| a     | —    | —    | required | positional |
| b     | —    | —    | required | positional |

| output | description                                            |
|--------|--------------------------------------------------------|
| out    | (1 x 1) shCoefficients elementwise-scaled coefficients |

*Example*

```
gW = w .* g; % per-coefficient weighting
```

### shCoefficients.truncate

```
out = g.truncate(nmaxNew)
```

Reduce the maximum degree.

| input   | size  | type   | default  | kind       |
|---------|-------|--------|----------|------------|
| obj     | —     | —      | required | positional |
| nmaxNew | 1 x 1 | double | required | positional |

| output | description                                      |
|--------|--------------------------------------------------|
| out    | (1 x 1) shCoefficients truncated to the new nmax |

*Example*

```
g60 = g.truncate(60);
```

### shCoefficients.setCoefficient

```
out = g.setCoefficient(n, m, Cval, Sval, Name=Value)
```

Replace a single (n,m) coefficient pair.

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| obj    | —     | —      | required | positional |
| n      | 1 x 1 | double | required | positional |
| m      | 1 x 1 | double | required | positional |
| Cval   | 1 x 1 | double | required | positional |
| Sval   | 1 x 1 | double | NaN      | positional |
| SigmaC | 1 x 1 | double | NaN      | name-value |
| SigmaS | 1 x 1 | double | NaN      | name-value |

| output | description                                             |
|--------|---------------------------------------------------------|
| out    | (1 x 1) shCoefficients copy with C/S(n+1, m+1) replaced |

*Example*

```
g = g.setCoefficient(2, 0, c20slr); % SLR C20 by hand
```

### shCoefficients.applyTN14

```
out = g.applyTN14(tn, Name=Value)
```

Replace C20 (and C30 where provided) from a TN-14 table. OUT = obj.applyTN14(TN) with TN from shLowLevel.readTN14 (or a filename). Requires a finite obj.epoch; the nearest TN epoch within opts.Tolerance (default 0.05 yr) is used. opts.ReplaceC30: "auto" (default: replace when the table has a non-NaN C30 for that month), "never", "always".

| input      | size  | type                               | default  | kind       |
|------------|-------|------------------------------------|----------|------------|
| obj        | —     | —                                  | required | positional |
| tn         | —     | —                                  | required | positional |
| Tolerance  | 1 x 1 | double                             | 0.05     | name-value |
| ReplaceC30 | 1 x 1 | string ∈ ["auto","never","always"] | "auto"   | name-value |

| output | description                                                                                    |
|--------|------------------------------------------------------------------------------------------------|
| out    | (1 x 1) shCoefficients C20 (and C30 when available) replaced by the SLR values, sigmas updated |

*Example*

```
g = g.applyTN14(tn14); % C20/C30 replacement
```

### shCoefficients.addDegree1

```
out = g.addDegree1(tn, Name=Value)
```

Insert TN-13 geocenter (degree-1) coefficients. OUT = obj.addDegree1(TN) with TN from shLowLevel.readTN13 (or a filename) sets C10, C11, S11 (with their sigmas) from the TN-13 record nearest to obj.epoch (within opts.Tolerance (0.05), default 0.05 yr). Without degree 1, EWH/mass grids are systematically biased - this completes the standard GSM + TN-14 + TN-13 processing chain.

| input     | size  | type   | default  | kind       |
|-----------|-------|--------|----------|------------|
| obj       | —     | —      | required | positional |
| tn        | —     | —      | required | positional |
| Tolerance | 1 x 1 | double | 0.05     | name-value |

| output | description                                                                        |
|--------|------------------------------------------------------------------------------------|
| out    | (1 x 1) shCoefficients degree-1 row set from the TN-13 record nearest to obj.epoch |

*Example*

```
g = g.addDegree1(tn13); % nearest epoch within 0.05 yr
```

### shCoefficients.destripe

```
out = g.destripe(Name=Value)
```

Swenson & Wahr (2006) / P3M6 decorrelation. OUT = obj.destripe(minOrder=6, polyOrder=3, windowLength=[])  
Formal errors are passed through UNCHANGED (the destripping operator is data-dependent; rigorous error propagation is not defined for it - use the tvANS filter on an shSeries for a filter with proper covariance semantics).

| input        | size  | type   | default  | kind       |
|--------------|-------|--------|----------|------------|
| obj          | —     | —      | required | positional |
| minOrder     | 1 x 1 | double | 6        | name-value |
| polyOrder    | 1 x 1 | double | 3        | name-value |
| windowLength | —     | double | []       | name-value |

| output | description                                      |
|--------|--------------------------------------------------|
| out    | (1 x 1) shCoefficients Swenson-Wahr decorrelated |

*Example*

```
g = g.destripe(minOrder = 6);
```

### shCoefficients.gaussian

```
out = g.gaussian(radiusKm)
```

Jekeli (1981) isotropic smoothing; sigmas scaled by Wn.

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| obj      | —     | —      | required | positional |
| radiusKm | 1 x 1 | double | required | positional |

| output | description                                                   |
|--------|---------------------------------------------------------------|
| out    | (1 x 1) shCoefficients Gaussian-smoothed; sigmas scaled by Wn |

*Example*

```
g = g.gaussian(350); % radius in km
```

### shCoefficients.degreeRMS

```
spec = g.degreeRMS(Name=Value)
```

Degree variance/RMS/amplitude spectrum (+ error spectrum).

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| n0    | 1 x 1 | double | 0        | name-value |

| output | description                                                          |
|--------|----------------------------------------------------------------------|
| spec   | struct: n, amp/rms/var, err degree spectrum incl. formal-error curve |

*Example*

```
spec = g.degreeRMS(); % degree amplitude + error spectrum
```

### shCoefficients.crossover

```
[n0, nInterp] = g.crossover(Name=Value)
```

Signal-vs-formal-error crossover degree.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| n0    | 1 x 1 | double | 2        | name-value |

| output  | description                               |
|---------|-------------------------------------------|
| nInterp | (1,1) double interpolated crossing degree |

*Example*

```
nc = g.crossover(); % needs formal sigmas
```

### shCoefficients.spectrum

```
h = g.spectrum(varargin)
```

Plot the degree-amplitude spectrum. h = obj.spectrum(...)

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| varargin | —    | —    | required | positional |

| output | description                           |
|--------|---------------------------------------|
| h      | (1 x 1) graphics handle spectrum plot |

*Example*

```
g.spectrum(Kaula = true, MarkCrossover = true);
```

### shCoefficients.triangle

```
h = g.triangle(varargin)
```

Plot the degree/order coefficient triangle. h = obj.triangle(...)

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| varargin | —    | —    | required | positional |

| output | description                                       |
|--------|---------------------------------------------------|
| h      | (1 x 1) graphics handle coefficient triangle plot |

*Example*

```
g.triangle(); % butterfly plot
```

### shCoefficients.synthesis

```
[grid, lat, lon] = g.synthesis(latVec, lonVec, Name=Value)
```

Spatial synthesis (geoid, gravity, potential, EWH). [GRID, LAT, LON] = obj.synthesis(LATVEC, LONVEC, quantity ("geoid")=..., kn ([])=..., nmin (0)=..., nmax (NaN)=..., UseCache (true)=true). The Legendre functions are served from a verified process-wide cache (shLowLevel.legendreCached), so monthly time series on a fixed grid pay the recursion only once - no manual 'P' passing. EWH requires explicitly supplied load Love numbers kn (never

hardcoded).

| input      | size  | type                               | default         | kind       |
|------------|-------|------------------------------------|-----------------|------------|
| obj        | —     | —                                  | required        | positional |
| latVec     | 1 x : | double                             | required        | positional |
| lonVec     | 1 x : | double                             | required        | positional |
| quantity   | 1 x 1 | string                             | "geoid"         | name-value |
| kn         | —     | double                             | []              | name-value |
| hn         | —     | double                             | []              | name-value |
| Height     | 1 x 1 | double                             | 0               | name-value |
| nmin       | 1 x 1 | double                             | 0               | name-value |
| nmax       | 1 x 1 | double                             | NaN             | name-value |
| rho_ave    | 1 x 1 | double                             | 5517            | name-value |
| rho_water  | 1 x 1 | double                             | 1000            | name-value |
| UseCache   | 1 x 1 | logical                            | true            | name-value |
| Method     | 1 x 1 | string ∈ ["auto","direct","fft"]   | "auto"          | name-value |
| LatType    | 1 x 1 | string ∈ ["geocentric","geodetic"] | "geocentric"    | name-value |
| Flattening | 1 x 1 | double                             | 1/298.257223563 | name-value |
| MaxMemGB   | 1 x 1 | double                             | 4               | name-value |

| output | description                            |
|--------|----------------------------------------|
| grid   | (nlat x nlon) double synthesized field |
| lat    | (1 x nlat) double geocentric latitudes |
| lon    | (1 x nlon) double longitudes           |

*Example*

```
ewh = g.synthesis(-89:89, 0:359, quantity = "ewh", kn = kn);
```

### shCoefficients.toReference

```
out = g.toReference(Name=Value)
```

Convert to another (GM (NaN), R (NaN)) reference (exact). OUT = g.toReference(GM=3.986004418e14, R=6378137.0) re-expresses the coefficients via shLowLevel.rescaleGMR; the physical field is invariant (Python-validated). Sigmas rescale identically; GM/R properties are updated.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| GM    | 1 x 1 | double | NaN      | name-value |
| R     | 1 x 1 | double | NaN      | name-value |

| output | description                                       |
|--------|---------------------------------------------------|
| out    | (1 x 1) shCoefficients rescaled to the given GM/R |

*Example*

```
g = g.toReference(GM = 3.986004415e14, R = 6.3781363e6);
```

### shCoefficients.subtractNormalField

```
out = g.subtractNormalField(Name=Value)
```

Remove the ellipsoidal normal field. OUT = g.subtractNormalField() % WGS84 OUT = g.subtractNormalField(System ("WGS84")="GRS80") Computes the even zonals from the DEFINING constants

(shLowLevel.normalFieldCS, Heiskanen-Moritz closed form; validated against NIMA TR8350.2 to all published digits), rescales them from the ellipsoid's own (GM (NaN), a (NaN)) to the object's (GM, R) - the WGS84 GM and a DIFFER from the ICGEM values, skipping this step costs millimetres - and subtracts them including the deg...

| input  | size  | type                       | default  | kind       |
|--------|-------|----------------------------|----------|------------|
| obj    | —     | —                          | required | positional |
| System | 1 x 1 | string ∈ ["WGS84","GRS80"] | "WGS84"  | name-value |
| GM     | 1 x 1 | double                     | NaN      | name-value |
| a      | 1 x 1 | double                     | NaN      | name-value |
| f      | 1 x 1 | double                     | NaN      | name-value |
| omega  | 1 x 1 | double                     | NaN      | name-value |

| output | description                                                                       |
|--------|-----------------------------------------------------------------------------------|
| out    | (1 x 1) shCoefficients disturbing field (even zonals of the normal field removed) |

*Example*

```
g = g.subtractNormalField(System = "GRS80");
```

### shCoefficients.map

```
h = g.map(latDeg, lonDeg, Name=Value)
```

Synthesize and plot in one call (shLowLevel.plotSHMap). H = g.map(-89:89, 0:359, quantity ("geoid")="ewh", kn ([])=kn) - all synthesis options plus the plotSHMap display options.

| input      | size  | type    | default  | kind       |
|------------|-------|---------|----------|------------|
| obj        | —     | —       | required | positional |
| latDeg     | 1 x : | double  | -89:2:89 | positional |
| lonDeg     | 1 x : | double  | 0:2:358  | positional |
| quantity   | 1 x 1 | string  | "geoid"  | name-value |
| kn         | —     | double  | []       | name-value |
| hn         | —     | double  | []       | name-value |
| Height     | 1 x 1 | double  | 0        | name-value |
| nmin       | 1 x 1 | double  | 0        | name-value |
| Projection | 1 x 1 | string  | "plate"  | name-value |
| Coast      | 1 x 1 | logical | true     | name-value |
| CLim       | —     | double  | []       | name-value |
| Units      | 1 x 1 | string  | ""       | name-value |
| Title      | 1 x 1 | string  | ""       | name-value |
| ax         | —     | —       | []       | name-value |

| output | description                                  |
|--------|----------------------------------------------|
| h      | (1 x 1) graphics handle synthesized map plot |

*Example*

```
g.map(quantity = "geoid", Title = "Geoid 2008-04 [m]");
```

### shCoefficients.fan

```
out = g.fan(rDegKm, rOrdKm)
```

Han fan filter (degree x order Gaussian) - shLowLevel.shFanFilter.

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| obj    | —     | —      | required | positional |
| rDegKm | 1 x 1 | double | required | positional |
| rOrdKm | 1 x 1 | double | required | positional |

| output | description                         |
|--------|-------------------------------------|
| out    | (1 x 1) shCoefficients fan-filtered |

*Example*

```
g = g.fan(350, 200);
```

### shCoefficients.deformation

```
[up, north, east] = g.deformation(latDeg, lonDeg, Name=Value)
```

Elastic load deformation up/north/east [m]. [UP, NORTH, EAST] = g.deformation(LAT, LON, kn=, hn=, ln=, Mode ("grid")="grid"|"points", nmin (1)=1, LatType ("geocentric")="geocentric") from the object's (residual!) coefficients - remove a mean field first. Love numbers are always user-supplied; see shLowLevel.shSynthesisDeformation for formulas and validation.

| input      | size  | type                               | default         | kind       |
|------------|-------|------------------------------------|-----------------|------------|
| obj        | —     | —                                  | required        | positional |
| latDeg     | 1 x : | double                             | required        | positional |
| lonDeg     | 1 x : | double                             | required        | positional |
| kn         | —     | double                             | required        | name-value |
| hn         | —     | double                             | required        | name-value |
| ln         | —     | double                             | required        | name-value |
| nmin       | 1 x 1 | double                             | 1               | name-value |
| Mode       | 1 x 1 | string ∈ ["grid","points"]         | "grid"          | name-value |
| LatType    | 1 x 1 | string ∈ ["geocentric","geodetic"] | "geocentric"    | name-value |
| Flattening | 1 x 1 | double                             | 1/298.257223563 | name-value |

| output | description                                          |
|--------|------------------------------------------------------|
| up     | (nlat x nlon   npts) double vertical deformation [m] |
| north  | same size north component [m]                        |
| east   | same size east component [m]                         |

*Example*

```
[up, no, ea] = g.deformation(latPts, lonPts, kn = LN.kn, hn = LN.hn, ...
    ln = LN.ln, Mode = "points");
```

### shCoefficients.applyDDK

```
out = g.applyDDK(W)
```

Apply a DDK (order-block-diagonal) decorrelation filter. OUT = obj.applyDDK(W) with W from shLowLevel.readDDK (struct, .mat, or documented ASCII exchange format). Degrees not covered by the filter pass through unchanged; sigmas are NOT propagated through the filter (set to NaN - use shLowLevel.mcPropagate for filtered uncertainties).

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| obj   | —    | —    | required | positional |
| W     | —    | —    | required | positional |

| output | description                             |
|--------|-----------------------------------------|
| out    | (1 x 1) shCoefficients DDK-decorrelated |

*Example*

```
g = g.applyDDK("DDK3");
```

### shCoefficients.evalAt

```
out = g.evalAt(epoch)
```

Evaluate gfct time-variable terms at an epoch.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| epoch | 1 x 1 | double | required | positional |

| output | description                                                                                             |
|--------|---------------------------------------------------------------------------------------------------------|
| out    | (1 x 1) shCoefficients gfct variable terms evaluated at the epoch; static models pass through unchanged |

*Example*

```
g2015 = g.evalAt(2015.5);           % gfct variable terms AT AN EPOCH
```

### shCoefficients.write

```
g.write(filename, Name=Value)
```

Export to an ICGEM .gfc file (round-trip tested).

| input    | size  | type    | default  | kind       |
|----------|-------|---------|----------|------------|
| obj      | —     | —       | required | positional |
| filename | —     | —       | required | positional |
| Comment  | 1 x 1 | string  | ""       | name-value |
| Sidecar  | 1 x 1 | logical | true     | name-value |

*Example*

```
g.write("out.gfc");
```

### shCoefficients.vec

```
x = g.vec(idx)
```

Pack into a coefficient vector in shLowLevel.shIndex ordering.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| idx   | 1 x 1 | struct | required | positional |

| output | description                                 |
|--------|---------------------------------------------|
| x      | (P x 1) double coefficients in idx ordering |

*Example*

```
x = g10.vec(idx);           % idx-ordered coefficient vector
```

### shCoefficients.disp

```
g.disp()
```

Compact display: size, epoch, GM/R, processing history. `disp(G)` prints one header line with the model name, one contract line (nmax, epoch, product type, tide system, GM, R), whether formal sigmas are attached and how many gfc variable terms are carried, followed by the processing history. Called automatically when an `shCoefficients` object is shown without a semicolon.

#### Example

```
disp(g) % degree, epoch, history
```

### shCoefficients.read (static)

```
obj = shCoefficients.read(filename, Name=Value)
```

Read an ICGEM .gfc/.gfct/.gfc.gz file into an `shCoefficients`. `G = shCoefficients.read(FILE)` parses product type and mid-epoch from standard GRACE L2 filenames (GSM/GAA/GAB/GAC/GAD-2\_...); override with `Epoch (NaN)=...`, `ProductType ("")=...` when needed.

| input       | size  | type   | default  | kind       |
|-------------|-------|--------|----------|------------|
| filename    | —     | —      | required | positional |
| Epoch       | 1 x 1 | double | NaN      | name-value |
| ProductType | 1 x 1 | string | ""       | name-value |

| output | description                                                                                                  |
|--------|--------------------------------------------------------------------------------------------------------------|
| obj    | (1 x 1) <code>shCoefficients</code> parsed gfc/gfct(.gz) with GM, R, sigmas, variable terms, Epoch= as given |

#### Example

```
g = shCoefficients.read("ITSG-Grace2018_n60_2008-04.gfc", Epoch = 2008.29);
```

### shCoefficients.fromVec (static)

```
obj = shCoefficients.fromVec(x, idx, like)
```

Unpack a `shLowLevel.shIndex`-ordered vector into an `shCoefficients`. `G = shCoefficients.fromVec(X, IDX, LIKE)` copies GM/R and other metadata from the template `LIKE` (an `shCoefficients`).

| input | size  | type                        | default  | kind       |
|-------|-------|-----------------------------|----------|------------|
| x     | : x 1 | double                      | required | positional |
| idx   | 1 x 1 | struct                      | required | positional |
| like  | 1 x 1 | <code>shCoefficients</code> | required | positional |

| output | description                                                                              |
|--------|------------------------------------------------------------------------------------------|
| obj    | (1 x 1) <code>shCoefficients</code> rebuilt from x with sizes/GM/R/epoch of the template |

#### Example

```
g2 = shCoefficients.fromVec(x, idx, g10); % sizes/GM/R from template
```

### shCoefficients.analysis (static)

```
[obj, info] = shCoefficients.analysis(grid, latVec, lonVec, nmax, Name=Value)
```

Spherical harmonic analysis: gridded/scattered data -> Stokes coefficients (the inverse of synthesis).

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| grid     | —     | double | required | positional |
| latVec   | —     | double | required | positional |
| lonVec   | —     | double | required | positional |
| nmax     | 1 x 1 | double | required | positional |
| quantity | 1 x 1 | string | "geoid"  | name-value |

| input      | size  | type                               | default         | kind       |
|------------|-------|------------------------------------|-----------------|------------|
| Method     | 1 x 1 | string                             | "auto"          | name-value |
| Weights    | 1 x 1 | string                             | "none"          | name-value |
| Kaula      | 1 x 1 | double                             | 0               | name-value |
| kn         | —     | double                             | []              | name-value |
| hn         | —     | double                             | []              | name-value |
| rho_ave    | 1 x 1 | double                             | 5517            | name-value |
| rho_water  | 1 x 1 | double                             | 1000            | name-value |
| GM         | 1 x 1 | double                             | 3.986004415e14  | name-value |
| R          | 1 x 1 | double                             | 6378136.3       | name-value |
| Epoch      | 1 x 1 | double                             | NaN             | name-value |
| Name       | 1 x 1 | string                             | "analysis"      | name-value |
| LatType    | 1 x 1 | string ∈ ["geocentric","geodetic"] | "geocentric"    | name-value |
| Flattening | 1 x 1 | double                             | 1/298.257223563 | name-value |

| output | description                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------------------|
| obj    | (1 x 1) shCoefficients Stokes coefficients estimated from the grid (exact on ring grids; Kaula for scattered points) |

*Example*

```
g2 = shCoefficients.analysis(grid, lat, lon, 10, Kaula = 1e-10);
```

## shSeries class

Time series of spherical harmonic coefficient sets. Wraps a stack of shCoefficients over epochs and provides everything with a time dimension: the mean field, the climatology (bias / trend / annual / semi-annual), GAX background restoration, the classic destripe+Gaussian chain applied per epoch, and the tvANS time-variable anisotropic Wiener filter with exact basin deconvolution. Immutable value class with provenance in .history.

| property    | size  | type   | default           | notes |
|-------------|-------|--------|-------------------|-------|
| Cs          | —     | double | —                 |       |
| Ss          | —     | double | —                 |       |
| sigmaCs     | —     | double | []                |       |
| sigmaSs     | —     | double | []                |       |
| epochs      | : x 1 | double | —                 |       |
| GM          | 1 x 1 | double | 3.986004415e14    |       |
| R           | 1 x 1 | double | 6378136.3         |       |
| productType | 1 x 1 | string | "unknown"         |       |
| names       | : x 1 | string | string.empty(0,1) |       |
| history     | —     | string | string.empty(0,1) |       |
| nmax        | —     | —      | —                 |       |
| nEpochs     | —     | —      | —                 |       |

### shSeries.shSeries

```
obj = ts.shSeries(in, Name=Value)
```

Construct from an shCoefficients array or raw stacks. ts = shSeries(objArray) ts = shSeries(Cs, Epochs ([])=..., Ss ([])=...) (raw-stack form via opts)

| input       | size  | type   | default           | kind       |
|-------------|-------|--------|-------------------|------------|
| in          | —     | —      | required          | positional |
| Ss          | —     | double | []                | name-value |
| Epochs      | : x 1 | double | []                | name-value |
| GM          | 1 x 1 | double | 3.986004415e14    | name-value |
| R           | 1 x 1 | double | 6378136.3         | name-value |
| ProductType | 1 x 1 | string | "unknown"         | name-value |
| Names       | : x 1 | string | string.empty(0,1) | name-value |
| SigmaCs     | —     | double | []                | name-value |
| SigmaSs     | —     | double | []                | name-value |
| History     | —     | string | string.empty(0,1) | name-value |

| output | description                                                         |
|--------|---------------------------------------------------------------------|
| obj    | (1 x 1) shSeries epoch-sorted monthly stack with sigmas and history |

*Example*

```
ts = shSeries(Cs, Ss = Ss, Epochs = tYears);
```

### shSeries.at

```
g = ts.at(k)
```

Extract epoch k as an shCoefficients.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| k     | 1 x 1 | double | required | positional |

| output | description                                          |
|--------|------------------------------------------------------|
| g      | (1 x 1) shCoefficients month k with epoch and sigmas |

*Example*

```
g = ts.at(37); % single month as shCoefficients
```

### shSeries.mean

```
g = ts.mean(Name=Value)
```

Mean field of the series. G = ts.mean (Omitnan=true default) returns an shCoefficients; its sigmas, when present, are the standard error of the mean.

| input   | size  | type    | default  | kind       |
|---------|-------|---------|----------|------------|
| obj     | —     | —       | required | positional |
| Omitnan | 1 x 1 | logical | true     | name-value |

| output | description                                                   |
|--------|---------------------------------------------------------------|
| g      | (1 x 1) shCoefficients temporal mean field (omits NaN months) |

*Example*

```
gM = ts.mean; % NaN-month tolerant
```

### shSeries.climatology

```
[clim, resid] = ts.climatology(Name=Value)
```

Bias/trend/annual/semi-annual (+extra periods) fit. [CLIM, RESID] = ts.climatology(Robust=false, T0 (NaN)=mean(epochs), Periods (double.empty(1,0))=[], Weights ([])=[]) CLIM is an shClimatology; RESID the residual shSeries. Periods: extra periodic terms [years] - e.g. the GRACE tidal alias periods [161/365.25, 3.66, 7.48] (S2, K2, K1); without them, trend and semi-annual estimates absorb tidal aliasing. Weights: T x 1 per-epoch weights (e.g. 1./info.vce from the tvANS filter). CLIM carries per-coefficient 1-sigm...

| input     | size  | type    | default           | kind       |
|-----------|-------|---------|-------------------|------------|
| obj       | —     | —       | required          | positional |
| Robust    | 1 x 1 | logical | false             | name-value |
| T0        | 1 x 1 | double  | NaN               | name-value |
| Periods   | 1 x : | double  | double.empty(1,0) | name-value |
| Weights   | : x 1 | double  | []                | name-value |
| ARCorrect | 1 x 1 | logical | false             | name-value |

| output | description                                                                                   |
|--------|-----------------------------------------------------------------------------------------------|
| clim   | (1 x 1) shClimatology fitted bias/trend/annual/semiannual (+Periods=) with coefficient sigmas |
| resid  | (1 x 1) shSeries residual series about the fit                                                |

#### Example

```
[clim, resid] = ts.climatology(Robust = true, ARCorrect = true);
```

### shSeries.trendBreaks

```
out = ts.trendBreaks(Name=Value)
```

Piecewise-linear trends with break significance. OUT = ts.trendBreaks(Breaks=[2016.0], Periods (double.empty(1,0))=[], T0 (NaN)=, ARCorrect (false)=false) fits the standard climatology design PLUS hinge terms max(0, t - tb) per break epoch (continuous piecewise-linear trend) to every coefficient, and tests the break's significance with an F-test against the no-break model (p-values via betainc - base MATLAB, no toolbox). Python-validated: 5.1% null rejection at the 5% level; F = 23 for a 0.02/yr break in 0.05-si...

| input     | size  | type    | default           | kind       |
|-----------|-------|---------|-------------------|------------|
| obj       | —     | —       | required          | positional |
| Breaks    | 1 x : | double  | required          | name-value |
| Periods   | 1 x : | double  | double.empty(1,0) | name-value |
| T0        | 1 x 1 | double  | NaN               | name-value |
| ARCorrect | 1 x 1 | logical | false             | name-value |

| output | description                                                                |
|--------|----------------------------------------------------------------------------|
| out    | struct: trends per segment (shCoefficients), F/p per break, segment epochs |

#### Example

```
out = ts.trendBreaks(Breaks = 2021.0); % F-test on the hinge
```

### shSeries.fan

```
out = ts.fan(rDegKm, rOrdKm)
```

Han fan filter on every epoch (degree x order Gaussian). OUT = ts.fan(RDEG, RORD) - see shLowLevel.shFanFilter.

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| obj    | —     | —      | required | positional |
| rDegKm | 1 x 1 | double | required | positional |
| rOrdKm | 1 x 1 | double | required | positional |

| output | description                             |
|--------|-----------------------------------------|
| out    | (1 x 1) shSeries fan-filtered per month |

*Example*

```
tsF = ts.fan(350, 200);
```

### shSeries.applyTN14

```
out = ts.applyTN14(tn, Name=Value)
```

Replace C20 (and flagged C30) per month from TN-14. OUT = ts.applyTN14("TN-14.txt") - series-level convenience around shCoefficients.applyTN14 (v2.4.1; the per-month epoch selects the TN entry). All options pass through.

| input      | size  | type   | default  | kind       |
|------------|-------|--------|----------|------------|
| obj        | —     | —      | required | positional |
| tn         | —     | —      | required | positional |
| Tolerance  | 1 x 1 | double | 0.05     | name-value |
| ReplaceC30 | 1 x 1 | string | "auto"   | name-value |

| output | description                                     |
|--------|-------------------------------------------------|
| out    | (1 x 1) shSeries C20/C30 replaced epoch-matched |

*Example*

```
ts = ts.applyTN14(tn14); % epoch-matched per month
```

### shSeries.addDegree1

```
out = ts.addDegree1(tn, Name=Value)
```

Insert TN-13 geocenter degree-1 per month. OUT = ts.addDegree1("TN-13.txt") - series-level convenience around shCoefficients.addDegree1 (v2.4.1).

| input     | size  | type   | default  | kind       |
|-----------|-------|--------|----------|------------|
| obj       | —     | —      | required | positional |
| tn        | —     | —      | required | positional |
| Tolerance | 1 x 1 | double | 0.05     | name-value |

| output | description                                       |
|--------|---------------------------------------------------|
| out    | (1 x 1) shSeries degree 1 completed epoch-matched |

*Example*

```
ts = ts.addDegree1(tn13);
```

### shSeries.removeAlias

```
out = ts.removeAlias(Name=Value)
```

Remove a tidal alias harmonic from every epoch. TS2 = ts.removeAlias() fits, per coefficient, a harmonic at the S2 tidal alias period (161 days) TOGETHER WITH bias, trend, annual and semi-annual, and subtracts only that harmonic. This is the correction Gravis applies to its Level-2B products (<https://gravis.gfz.de/corrections>): errors in the ocean-tide background model alias into the monthly fields at this period, and leaving them in puts a spurious ~161-day signal into every derived series.

| input  | size  | type   | default    | kind       |
|--------|-------|--------|------------|------------|
| obj    | —     | —      | required   | positional |
| Period | 1 x 1 | double | 161/365.25 | name-value |

| input       | size  | type   | default | kind       |
|-------------|-------|--------|---------|------------|
| PhaseOffset | 1 x 1 | double | 100     | name-value |
| SplitEpoch  | 1 x 1 | double | 2018.0  | name-value |
| T0          | 1 x 1 | double | NaN     | name-value |

| output | description                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------|
| out    | (1,1) shSeries copy with the alias harmonic removed from every epoch; the operation is appended to the history |

#### Example

```
ts = ts.removeAlias(); % S2 161-day alias, 100 deg offset
ts = ts.removeAlias(SplitEpoch = 2018.4);
```

### shSeries.removeGIA

```
out = ts.removeGIA(gia, Name=Value)
```

Subtract a glacial isostatic adjustment model. `OUT = ts.removeGIA(GIA, T0 (NaN)=mean(epochs))` subtracts the linear GIA signal `giaRate * (t - T0)` from every epoch. GIA is an `shCoefficients` object holding RATE coefficients [1/yr] - read your model (ICE-6G\_D, Caron et al., ...) from its `gfc` file with `shCoefficients.read`; models are ALWAYS user-supplied, never embedded. Trends computed after this are GIA-corrected; the model is treated as exact (its uncertainty, often the dominant trend error over Laurentia/Fennos...

| input | size  | type           | default  | kind       |
|-------|-------|----------------|----------|------------|
| obj   | —     | —              | required | positional |
| gia   | 1 x 1 | shCoefficients | required | positional |
| T0    | 1 x 1 | double         | NaN      | name-value |

| output | description                                                    |
|--------|----------------------------------------------------------------|
| out    | (1 x 1) shSeries GIA trend removed about the series mean epoch |

#### Example

```
ts = ts.removeGIA(giaModel); % e.g. ICE-6G trend gfc
```

### shSeries.applyDDK

```
out = ts.applyDDK(W)
```

Apply a DDK decorrelation filter to every epoch. `OUT = ts.applyDDK(W)`, `W` from `shLowLevel.readDDK`. Sigma stacks are invalidated (NaN) - the filter correlates coefficients.

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| obj   | —    | —    | required | positional |
| W     | —    | —    | required | positional |

| output | description                                 |
|--------|---------------------------------------------|
| out    | (1 x 1) shSeries DDK-decorrelated per month |

#### Example

```
ts = ts.applyDDK("DDK3");
```

### shSeries.compare

```
[rep, h] = ts.compare(others, varargin)
```

Temporal comparison against other series (v2.6.0). `REP = ts.compare(TS2)` or `ts.compare({TS2, TS3, ...})` delegates to `shLowLevel.compareSeries` with `OBJ` as the reference; all options pass through (`Basin=`, `Names=`, `Plot=`, ...).

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| others   | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                                    |
|--------|------------------------------------------------|
| rep    | (1 x 1) struct shLowLevel.compareSeries report |
| h      | (1 x 1) graphics handle figure (Plot = true)   |

#### Example

```
rep = ts.compare({tsGFZ, tsJPL}, Names = ["ref", "GFZ", "JPL"]);
```

### shSeries.restore

out = ts.restore(gax, Name=Value)

Add a background (GAA/GAB/GAC/GAD) series, epoch-matched. OUT = ts.restore(GAXSERIES, Tolerance=0.05, AllowMissing=false) Each epoch of OBJ is matched to the nearest GAX epoch within Tolerance [yr]. Missing matches error (or, with AllowMissing=true, leave those epochs unchanged with a note).

| input        | size  | type     | default  | kind       |
|--------------|-------|----------|----------|------------|
| obj          | —     | —        | required | positional |
| gax          | 1 x 1 | shSeries | required | positional |
| Tolerance    | 1 x 1 | double   | 0.05     | name-value |
| AllowMissing | 1 x 1 | logical  | false    | name-value |

| output | description                                                                 |
|--------|-----------------------------------------------------------------------------|
| out    | (1 x 1) shSeries background model added back epoch-matched (e.g. GSM + GAD) |

#### Example

```
ts = ts.restore(tsGAD); % GSM + GAD, epoch-matched
```

### shSeries.minus

out = ts.minus(a, b)

Subtract a field (shCoefficients) or an epoch-matched series.

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| a     | —    | —    | required | positional |
| b     | —    | —    | required | positional |

| output | description                                                    |
|--------|----------------------------------------------------------------|
| out    | (1 x 1) shSeries per-month difference (series or single field) |

#### Example

```
dts = ts - ts.mean;
```

### shSeries.destripe

out = ts.destripe(Name=Value)

Swenson & Wahr / P3M6, applied per epoch.

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| obj      | —     | —      | required | positional |
| minOrder | 1 x 1 | double | 6        | name-value |

| input        | size  | type   | default | kind       |
|--------------|-------|--------|---------|------------|
| polyOrder    | 1 x 1 | double | 3       | name-value |
| windowLength | —     | double | []      | name-value |

| output | description                          |
|--------|--------------------------------------|
| out    | (1 x 1) shSeries destriped per month |

*Example*

```
ts = ts.destripe(minOrder = 6);
```

### shSeries.gaussian

```
out = ts.gaussian(radiusKm)
```

Jekeli smoothing per epoch (vectorized over the stack).

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| obj      | —     | —      | required | positional |
| radiusKm | 1 x 1 | double | required | positional |

| output | description                                  |
|--------|----------------------------------------------|
| out    | (1 x 1) shSeries Gaussian-smoothed per month |

*Example*

```
ts = ts.gaussian(300);
```

### shSeries.truncate

```
out = ts.truncate(nmaxNew)
```

Reduce the maximum degree of the whole series.

| input   | size  | type   | default  | kind       |
|---------|-------|--------|----------|------------|
| obj     | —     | —      | required | positional |
| nmaxNew | 1 x 1 | double | required | positional |

| output | description                          |
|--------|--------------------------------------|
| out    | (1 x 1) shSeries truncated per month |

*Example*

```
ts8 = ts.truncate(8); % keep low degrees
```

### shSeries.dropNaN

```
out = ts.dropNaN()
```

Remove epochs containing any non-finite coefficient. OUT = ts.dropNaN() drops every epoch whose C or S stack has a NaN/Inf anywhere - typically placeholder months across the GRACE / GRACE-FO gap (2017.5-2018.5) or corrupt reads. The remaining series is irregularly sampled, which climatology and the tvANS filter handle natively (least-squares fit on the actual epochs; per-month VCE).

| output | description                         |
|--------|-------------------------------------|
| out    | (1 x 1) shSeries gap months removed |

*Example*

```
ts = ts.dropNaN; % remove gap months
```

## shSeries.select

```
out = ts.select(which)
```

Subset the series by index, mask, or time range. `OUT = ts.select(WHICH)` with `WHICH` one of logical (1,T) keep masked epochs indices keep listed epochs (in the given order) `[tMin tMax]` keep epochs with `tMin <= t <= tMax` (two-element non-integer-valued row => range; use explicit logical/index forms otherwise)

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| obj   | —    | —    | required | positional |
| which | —    | —    | required | positional |

| output | description                                            |
|--------|--------------------------------------------------------|
| out    | (1 x 1) shSeries months selected by logical/index mask |

### Example

```
ts = ts.select(ts.epochs > 2018.5);    % GRACE-F0 era only
```

## shSeries.filter

```
[out, op, info] = ts.filter(method, Name=Value)
```

Optimal time-variable filtering of the series. `[TSF, OP, INFO] = ts.filter("tvANS", Constraints ([])=..., NoiseCov ([])=..., SignalMode ("isotropic")="isotropic"/"inhomogeneous", NlterSignal (3)=3, Robust (false)=false, MinDegree (2)=2)` runs the tvANS chain (see `shLowLevel.tvANSFilter`): deterministic-model separation, empirical or supplied noise covariance, monthly VCE scaling, data-driven signal covariance, per-month anisotropic Wiener filter, optional hard constraints. `OP` is the stored linear operator - requir...

| input        | size  | type                         | default     | kind       |
|--------------|-------|------------------------------|-------------|------------|
| obj          | —     | —                            | required    | positional |
| method       | 1 x 1 | string                       | required    | positional |
| NoiseCov     | —     | double                       | []          | name-value |
| Constraints  | —     | double                       | []          | name-value |
| SignalMode   | 1 x 1 | string                       | "isotropic" | name-value |
| NlterSignal  | 1 x 1 | double                       | 3           | name-value |
| Robust       | 1 x 1 | logical                      | false       | name-value |
| MinDegree    | 1 x 1 | double                       | 2           | name-value |
| Blocks       | 1 x 1 | string ∈ ["auto","on","off"] | "auto"      | name-value |
| Shrinkage    | —     | double                       | []          | name-value |
| VCEMinDegree | —     | double                       | []          | name-value |
| VCEBands     | 1 x : | double                       | []          | name-value |

| output | description                                                                                                              |
|--------|--------------------------------------------------------------------------------------------------------------------------|
| out    | (1 x 1) shSeries tvANS-filtered series with sigmaCs/Ss posterior stacks (exact incl. constraints, v2.5)                  |
| op     | struct stored linear operator (V/Ut/lam/s or blocks, model, detLeverage/detResVar) for deconvolution and resolution maps |
| info   | struct: sigmaXfres (P x T), sigmaNote, VCE diagnostics                                                                   |

### Example

```
[tsF, op, info] = ts.filter("tvANS", Constraints = Ac, Blocks = "off");
```

## shSeries.basinAverage

```
[avg, out] = ts.basinAverage(B, Name=Value)
```

Basin averages, naive or exactly deconvolved.  $AVG = ts.basinAverage(B)$  - naive averages  $B \times x ./ \text{diag}(B \times B)$ ,  $K \times T$ , with  $B$  ( $P \times K$ ) basin kernels in `shLowLevel.shIndex` ordering (build from a grid mask via `shLowLevel.synthesisMatrix`:  $b = Y' * (w .* \text{mask})$ ).  $[AVG, OUT] = ts.basinAverage(B, \text{Deconvolve}(\text{false})=\text{true}, \text{Op}(\text{struct}())=\text{op})$  removes filter attenuation and inter-basin leakage exactly using the stored operator from `ts.filter` (see `shLowLevel.basinDeconvolve`). `OUT.sigma` ( $K \times T$ ) is the 1-sigma noise uncertainty of ...

| input      | size  | type    | default  | kind       |
|------------|-------|---------|----------|------------|
| obj        | —     | —       | required | positional |
| B          | —     | double  | required | positional |
| Deconvolve | 1 x 1 | logical | false    | name-value |
| Op         | —     | struct  | struct() | name-value |
| Ridge      | 1 x 1 | double  | 0        | name-value |

| output | description                                                                             |
|--------|-----------------------------------------------------------------------------------------|
| avg    | ( $K \times T$ ) double basin averages (deconvolved when <code>Deconvolve=true</code> ) |
| out    | struct: sigma ( $K \times T$ ), c, attn, condA (deconvolution path)                     |

*Example*

```
[avg, out] = tsF.basinAverage(B, Deconvolve = true, Op = op);
```

### shSeries.disp

`ts.disp()`

Compact display: months, span, gaps, processing history. `disp(TS)` prints the product type, the number of epochs, the maximum degree and the covered epoch range, followed by the processing history. Called automatically when an `shSeries` object is shown without a semicolon.

*Example*

```
disp(ts)
```

### shSeries.read (static)

`obj = shSeries.read(files, Name=Value)`

Read a series of gfc files (pattern or list), sorted by epoch. `TS = shSeries.read("GSM-2_*.gfc")` or `shSeries.read(fileList)`.

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| files    | —     | —      | required | positional |
| Truncate | 1 x 1 | double | NaN      | name-value |

| output | description                                        |
|--------|----------------------------------------------------|
| obj    | (1 x 1) shSeries wildcard-read, epoch-sorted stack |

*Example*

```
ts2 = shSeries.read("ITSG-Grace*_n60_*.gfc"); % wildcard, epoch-sorted
```

### shSeries.fromFolder (static)

`obj = shSeries.fromFolder(folder, Name=Value)`

Read every matching gfc file in a folder as a series. `TS = shSeries.fromFolder("itsg_series")` `TS = shSeries.fromFolder(dest, Pattern="*n96*.gfc", Truncate(NaN)=60)` Convenience wrapper around `shSeries.read` (sorted by epoch); pairs with `shLowLevel.fetchITSG`, which downloads ITSG monthly solutions into `tests/test_data/itsg_series` on demand.

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| folder   | 1 x 1 | string | required | positional |
| Pattern  | 1 x 1 | string | "*.gfc*" | name-value |
| Truncate | 1 x 1 | double | NaN      | name-value |

| output | description                                                   |
|--------|---------------------------------------------------------------|
| obj    | (1 x 1) shSeries all gfc(.gz) files of a folder, epoch-sorted |

*Example*

```
ts = shSeries.fromFolder(shLowLevel.dataFolder() + "/itsg_series");
```

## shClimatology class

Deterministic climatology of an SH series. Bias + trend + annual + semi-annual model per Stokes coefficient, as fitted by shSeries.climatology (least squares or Huber-robust):  $x(t) = \text{bias} + \text{trend} \cdot (t-t_0) + \cos A \cdot \cos(2\pi \cdot (t-t_0)) + \sin A \cdot \sin(\dots) + \cos SA \cdot \cos(4\pi \cdot (t-t_0)) + \sin SA \cdot \sin(4\pi \cdot (t-t_0)) + \sum_k \text{extra cos/sin terms of period } p_k$  (v2.1) All phases and the bias refer to the reference epoch  $t_0$ . Extra periods (Periods= option) typically hold the GRACE tidal alias terms S2 (161 d), K2 (3.66 yr), K1 (7.48 yr); access them via periodic(k). Per-coefficient 1-sigma significance from the fit is sto

| property  | size  | type   | default           | notes                                        |
|-----------|-------|--------|-------------------|----------------------------------------------|
| t0        | 1 x 1 | double | NaN               |                                              |
| GM        | 1 x 1 | double | 3.986004415e14    |                                              |
| R         | 1 x 1 | double | 6378136.3         |                                              |
| biasC     | —     | double | —                 |                                              |
| biasS     | —     | double | —                 |                                              |
| trendC    | —     | double | —                 |                                              |
| trendS    | —     | double | —                 |                                              |
| cosAnnC   | —     | double | —                 |                                              |
| cosAnnS   | —     | double | —                 |                                              |
| sinAnnC   | —     | double | —                 |                                              |
| sinAnnS   | —     | double | —                 |                                              |
| cosSemiC  | —     | double | —                 |                                              |
| cosSemiS  | —     | double | —                 |                                              |
| sinSemiC  | —     | double | —                 |                                              |
| sinSemiS  | —     | double | —                 |                                              |
| periods   | 1 x : | double | double.empty(1,0) | extra periods [yr] (v2.1)                    |
| extraCosC | —     | double | []                | n1 x n1 x K stacks for the extra periods     |
| extraCosS | —     | double | []                |                                              |
| extraSinC | —     | double | []                |                                              |
| extraSinS | —     | double | []                |                                              |
| coefSigma | —     | double | []                | (6+2K) x 2*Nc per-coefficient 1-sigma (v2.1) |
| history   | —     | string | string.empty(0,1) |                                              |
| nmax      | —     | —      | —                 |                                              |

### shClimatology.withNote

```
obj = clim.withNote(txt)
```

Append a provenance note to the history (v2.2). OBJ = clim.withNote(TXT) - the only supported way to annotate from outside the class (history is SetAccess=private).

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| obj   | —    | —    | required | positional |
| txt   | —    | —    | required | positional |

| output | description                                                      |
|--------|------------------------------------------------------------------|
| obj    | (1 x 1) shClimatology copy with the note appended to its history |

#### Example

```
clim = clim.withNote("GIA: ICE-6G_D removed");
```

### shClimatology.removeGIA

```
out = clim.removeGIA(gia)
```

Subtract a GIA rate model from the fitted trend. OUT = clim.removeGIA(GIA): GIA is an shCoefficients holding RATE coefficients [1/yr] (user-supplied model file via shCoefficients.read). Only trendC/trendS change; trend sigmas are kept - the model is treated as exact (documented limitation; GIA model spread often dominates regional trend error budgets - compare several models).

| input | size  | type           | default  | kind       |
|-------|-------|----------------|----------|------------|
| obj   | —     | —              | required | positional |
| gia   | 1 x 1 | shCoefficients | required | positional |

| output | description                                                                                    |
|--------|------------------------------------------------------------------------------------------------|
| out    | (1,1) shClimatology copy with the GIA trend removed from the trend component; history appended |

#### Example

```
clim = clim.removeGIA(giaModel);
```

### shClimatology.eval

```
g = clim.eval(epoch)
```

Evaluate the full climatology at an epoch -> shCoefficients.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| epoch | 1 x 1 | double | required | positional |

| output | description                                               |
|--------|-----------------------------------------------------------|
| g      | (1 x 1) shCoefficients model field evaluated at the epoch |

#### Example

```
g2020 = clim.eval(2020.0); % model field at an epoch
```

### shClimatology.bias

```
g = clim.bias()
```

Field at the reference epoch t0.

| output | description                                       |
|--------|---------------------------------------------------|
| g      | (1 x 1) shCoefficients bias component with sigmas |

### Example

```
g0 = clim.bias();
```

### shClimatology.trend

```
g = clim.trend()
```

Linear trend field, units of the coefficients per year.

| output | description                                                   |
|--------|---------------------------------------------------------------|
| g      | (1 x 1) shCoefficients trend component [units/yr] with sigmas |

### Example

```
gT = clim.trend(); % [units/yr], sigmas attached
```

### shClimatology.cosAnnual

```
g = clim.cosAnnual()
```

Cosine annual component (sigmas attached).

| output | description                                    |
|--------|------------------------------------------------|
| g      | (1 x 1) shCoefficients cosine annual component |

### Example

```
gc = clim.cosAnnual();
```

### shClimatology.sinAnnual

```
g = clim.sinAnnual()
```

Sine annual component (sigmas attached).

| output | description                                  |
|--------|----------------------------------------------|
| g      | (1 x 1) shCoefficients sine annual component |

### Example

```
gs = clim.sinAnnual();
```

### shClimatology.cosSemiannual

```
g = clim.cosSemiannual()
```

Cosine semiannual component (sigmas attached).

| output | description                                        |
|--------|----------------------------------------------------|
| g      | (1 x 1) shCoefficients cosine semiannual component |

### Example

```
gc2 = clim.cosSemiannual();
```

### shClimatology.sinSemiannual

```
g = clim.sinSemiannual()
```

Sine semiannual component (sigmas attached).

| output | description                                      |
|--------|--------------------------------------------------|
| g      | (1 x 1) shCoefficients sine semiannual component |

### Example

```
gs2 = clim.sinSemiannual();
```

### shClimatology.periodic

```
[gc, gs] = clim.periodic(k)
```

Cos/sin components of the k-th extra period (v2.1). [GC, GS] = clim.periodic(K) returns shCoefficients for the cos and sin terms of period obj.periods(K) [yr] - e.g. the S2/K2/K1 tidal alias terms when fitted via ts.climatology(Periods=[161/365.25, 3.66, 7.48]).

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| obj   | —     | —      | required | positional |
| k     | 1 x 1 | double | required | positional |

| output | description                                                      |
|--------|------------------------------------------------------------------|
| gc     | (1 x 1) shCoefficients cosine component of the k-th extra period |
| gs     | (1 x 1) shCoefficients sine component                            |

#### Example

```
[gc, gs] = clim.periodic(1); % extra Periods= component
```

### shClimatology.amplitudeMap

```
[A, lat, lon, phase] = clim.amplitudeMap(which, latVec, lonVec, Name=Value)
```

Pointwise seasonal amplitude (and phase) map. [A, LAT, LON, PHASE] = clim.amplitudeMap(WHICH, ... with WHICH "annual" | "semiannual" | k (index into clim.periods, v2.1), LATVEC, LONVEC, quantity ("geoid")=..., kn ([])=...) synthesizes the cos and sin component fields and combines them pointwise:  $A = \sqrt{fc.^2 + fs.^2}$  PHASE = atan2(fs, fc) [rad, relative to t0] (amplitude of a harmonic is NOT a linear functional of the coefficients, so it must be formed in the space domain).

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| obj      | —     | —      | required | positional |
| which    | 1 x 1 | —      | required | positional |
| latVec   | 1 x : | double | required | positional |
| lonVec   | 1 x : | double | required | positional |
| quantity | 1 x 1 | string | "geoid"  | name-value |
| kn       | —     | double | []       | name-value |

| output | description                                                              |
|--------|--------------------------------------------------------------------------|
| A      | (nlat x nlon) double amplitude of the harmonic in the requested quantity |
| lat    | (1 x nlat) double latitudes                                              |
| lon    | (1 x nlon) double longitudes                                             |
| phase  | (nlat x nlon) double phase [rad] of the harmonic                         |

#### Example

```
[A, lat, lon] = clim.amplitudeMap("annual", -89:89, 0:359, quantity = "ewh", kn = kn);
```

### shClimatology.disp

```
clim.disp()
```

Compact display: fitted components, reference epoch, note. disp(CLIM) prints the maximum degree and the reference epoch t0, the list of fitted components (bias, trend, annual and semi-annual cosine/sine, plus any extra Periods) and the processing history including the GIA note. Called automatically when an shClimatology object is shown without a semicolon.

#### Example

```
disp(clim)
```

### shClimatology.fromCoef (static)

```
obj = shClimatology.fromCoef(coef, t0, series, Name=Value)
```

Build from shLowLevel.fitDeterministicModel output (internal). OBJ = shClimatology.fromCoef(COEF, T0, SERIES, Periods (double.empty(1,0))=[], CoefSigma=[])) with COEF (6+2K) x (2\*Nc) over the flattened [C(:); S(:)] stack of SERIES; rows 7:end are the cos/sin pairs of the K extra Periods [yr].

| input     | size  | type     | default           | kind       |
|-----------|-------|----------|-------------------|------------|
| coef      | —     | double   | required          | positional |
| t0        | 1 x 1 | double   | required          | positional |
| series    | 1 x 1 | shSeries | required          | positional |
| Periods   | 1 x : | double   | double.empty(1,0) | name-value |
| CoefSigma | —     | double   | []                | name-value |

| output | description                                                                    |
|--------|--------------------------------------------------------------------------------|
| obj    | (1 x 1) shClimatology rebuilt from a coefficient table with the series' layout |

#### Example

```
clim = shClimatology.fromCoef(coef, t0, ts);
```

## Package +shLowLevel

### Readers, writers & data management

#### shLowLevel.shReadGFC

```
model = shLowLevel.shReadGFC(filename)
```

Read an ICGEM ASCII .gfc gravity field model file. MODEL = SHREADGFC(FILENAME) parses header key:value pairs and the gfc/gfct data block of a standard ICGEM-format file (GRACE/GRACE-FO monthly solutions, static models, GOCE, EGM2008, etc.). Gzip- compressed files (.gfc.gz) are transparently decompressed.

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| filename | —    | —    | required | positional |

| output | description                                                                                                                                                                 |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| model  | (1,1) struct fields: C/S/sigmaC/sigmaS (nmax+1 x nmax+1 double), GM/R (1,1 double), nmax (1,1 double), tide (string), variable terms (trend/annual/semiannual) when present |

#### Example

```
g = shLowLevel.shReadGFC("ITSG-Grace2018_n60_2008-04.gfc");
```

#### shLowLevel.shEvalGFCT

```
[Ct, St] = shLowLevel.shEvalGFCT(model, epoch)
```

Evaluate time-variable Stokes coefficients at a given epoch. [CT, ST] = SHEVALGFCT(MODEL, EPOCH) reconstructs  $C(t) = C_0 + \text{trnd}*(t-t_0) + \sum_k [\text{acos}_k*\cos(2*\pi*(t-t_0)) \dots + \text{asin}_k*\sin(2*\pi*(t-t_0))]$  (and analogously for S) from MODEL.C / MODEL.S (constant part) and MODEL.variableTerms (from SHREADGFC), at EPOCH (same units/convention as the t0 values in the file -- typically decimal years).

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| model | —    | —    | required | positional |
| epoch | —    | —    | required | positional |

| output | description                                               |
|--------|-----------------------------------------------------------|
| Ct     | (nmax+1 x nmax+1) double cosine coefficients at the epoch |
| St     | (nmax+1 x nmax+1) double sine coefficients at the epoch   |

*Example*

```
[C, S] = shLowLevel.shEvalGFCT(model, 2015.5); % piecewise gfct at an epoch
```

### shLowLevel.readTN13

```
tn = shLowLevel.readTN13(filename)
```

Read a GRACE/GRACE-FO TN-13 geocenter (degree-1) file. TN = shLowLevel.readTN13(FILENAME) parses the official TN-13 product (JPL/CSR/GFZ, e.g. TN-13\_GEOC\_CSR\_RL06.txt): data lines GRCOF2 n m Clm Slm Clm\_sig Slm\_sig begin end with n=1, m=0/1 and begin/end epochs yyymmdd.hhmm. Line pairs (m=0, m=1) sharing the same interval are combined into one monthly record with the interval mid-point as epoch.

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| filename | —    | —    | required | positional |

| output        | description                                                                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| tn            | struct:                                                                                                                                   |
| epoch         | (K,1) double interval mid-points [decimal years]                                                                                          |
| C10, C11, S11 | (K,1) double degree-1 Stokes coefficients sigC10, sigC11, sigS11 (K,1) double 1-sigma t0, t1 (K,1) double interval bounds [decimal years] |

*Example*

```
tn13 = shLowLevel.readTN13("TN-13_GEOC_CSR_RL06.3.txt");
```

### shLowLevel.readTN14

```
tn = shLowLevel.readTN14(filename)
```

Parse a TN-14 style SLR C20/C30 replacement table. TN = shLowLevel.readTN14(FILENAME) reads the CSR/GFZ "TN-14" technical-note text format: an arbitrary text header followed by data lines with 10 whitespace-separated numeric columns MJD\_begin yearfrac\_begin C20 C20-mean\*1e10 sigC20\*1e10 ... C30 C30-mean\*1e10 sigC30\*1e10 MJD\_end yearfrac\_end C30 columns contain 'NaN' for months where no SLR C30 is provided (pre GRACE-FO accelerometer degradation).

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| filename | —    | —    | required | positional |

| output | description                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------|
| tn     | (1,1) struct fields: t (T x 1 double, decimal years), C20/C30 (T x 1 double), sigmaC20/sigmaC30 (T x 1 double) |

*Example*

```
tn14 = shLowLevel.readTN14("TN-14_C30_C20_SLR_GSFC.txt");
```

### shLowLevel.readSINEX

```
snx = shLowLevel.readSINEX(filename, Name=Value)
```

Read gravity-field SINEX files (solutions, covariances, NEQs). SNX = shLowLevel.readSINEX(FILENAME) parses the SINEX blocks used by gravity-field solution providers (ITSG, COST-G, ...): +SOLUTION/ESTIMATE parameter list: type CN/SN, degree, order, value, std +SOLUTION/MATRIX\_ESTIMATE L|U COVA covariance (lower/upper, up to 3 values per line) +SOLUTION/NORMAL\_EQUATION\_MATRIX L|U normal-equation matrix .gz files are gunzipped transparently.

| input    | size  | type                          | default    | kind       |
|----------|-------|-------------------------------|------------|------------|
| filename | —     | —                             | required   | positional |
| Index    | —     | struct                        | struct([]) | name-value |
| Output   | 1 x 1 | string ∈ ["raw","covariance"] | "raw"      | name-value |
| Only     | 1 x 1 | string ∈ ["all","estimate"]   | "all"      | name-value |

| output | description                                                                                                                                             |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| snx    | (1,1) struct fields: x (P x 1 double), idx (struct), Cxx (P x P double, [] unless Only="full"), epoch (1,1 double), N/b (normal equations when present) |

#### Example

```
snx = shLowLevel.readSINEX("ITSG-Grace2018_n96_2008-04_head12.snx");
fprintf("%s with %d params\n", snx.kind, numel(snx.x))
```

### shLowLevel.readDDK

`W = shLowLevel.readDDK(source, Name=Value)`

Load a DDK (Kusche) anisotropic decorrelation filter. `W = shLowLevel.readDDK(SOURCE)` loads a DDK filter into the toolbox's order-block container, ready for `shLowLevel.applyDDK` / `g.applyDDK` / `ts.applyDDK`. The DDK filters (Kusche 2007; Kusche et al. 2009, DDK1..DDK8) are the standard published anisotropic alternative to Gaussian smoothing and the usual benchmark against tvANS.

| input  | size | type   | default  | kind       |
|--------|------|--------|----------|------------|
| source | —    | —      | required | positional |
| Nmax   | —    | double | []       | name-value |

| output | description                                                                                                |
|--------|------------------------------------------------------------------------------------------------------------|
| W      | struct: nmax (1 x 1), name, blocks (1 x 241) struct array with m, cs, n (degrees), M (square block matrix) |

#### Example

```
W = shLowLevel.readDDK("DDK3"); % DDK3 ships with the toolbox; others via shLowLevel.fetchDDK
```

### shLowLevel.readMascon

`mas = shLowLevel.readMascon(filename)`

Read a GRACE/GRACE-FO mascon netCDF (JPL / CSR / GSFC style). `MAS = shLowLevel.readMascon(FILENAME)` reads gridded mascon solutions for comparison against the SH chain (base MATLAB: `nread/ncinfo`, no toolboxes). Variable names are auto-detected among the layouts used by JPL RL06 ('lwe\_thickness'), CSR RL06 ('lwe\_thickness'), and generic 'lwe'/'water\_thickness'; time units "days since YYYY-MM-DD" are converted to decimal years.

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| filename | —    | —    | required | positional |

| output | description                                                                                                |
|--------|------------------------------------------------------------------------------------------------------------|
| mas    | struct: lat (I x 1), lon (J x 1), t (T x 1 decimal years), ewh (I x J x T) [m], units/meta from the netCDF |

#### Example

```
M = shLowLevel.readMascon("CSR_..._Mascons.nc"); % lat/lon/time EWH stack
```

### shLowLevel.readLoveNumbers

`[LN, info] = shLowLevel.readLoveNumbers(filename, Name=Value)`

Read load Love numbers from a user-supplied table file. `LN = shLowLevel.readLoveNumbers(FILE)` parses a plain-text load-Love-number table (comment lines starting with %, #, ! or // are skipped; an optional header line naming the columns is recognized). The toolbox never hardcodes Love numbers - this reader turns YOUR loading-model table (PREM, Gutenberg-Bullen, ak135, ...) into the `kn/hn/ln` vectors the synthesis and deformation routines expect.

| input     | size  | type                            | default  | kind       |
|-----------|-------|---------------------------------|----------|------------|
| filename  | —     | —                               | required | positional |
| Columns   | 1 x 1 | string                          | "auto"   | name-value |
| MaxDegree | —     | double                          | []       | name-value |
| Interp    | 1 x 1 | string ∈ ["none", "pchip"]      | "none"   | name-value |
| InFrame   | 1 x 1 | string ∈ ["", "CE", "CF", "CM"] | ""       | name-value |
| OutFrame  | 1 x 1 | string ∈ ["", "CE", "CF", "CM"] | ""       | name-value |

| output | description                                                                                                                                                                                                                                     |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LN     | struct: <code>n</code> (D x 1) degrees, ascending <code>kn</code> (D x 1) potential load Love numbers <code>k'_n</code> <code>hn</code> (D x 1) vertical (NaN if absent from file) <code>ln</code> (D x 1) horizontal (NaN if absent from file) |
| info   | struct: source, columns, frame, interpolated (logical mask)                                                                                                                                                                                     |

#### Example

```
LN = shLowLevel.readLoveNumbers("prem_load.txt", Columns="n h l k", ...
    MaxDegree=96, Interp="pchip", InFrame="CE", OutFrame="CF");
[up, no, ea] = g.deformation(latPts, lonPts, kn=LN.kn, hn=LN.hn, ...
    ln=LN.ln, Mode="points");
```

### shLowLevel.listICGEM

`[T, info] = shLowLevel.listICGEM(Name=Value)`

List gravity field models available at ICGEM (GFZ). `T = shLowLevel.listICGEM()` returns a table of the STATIC global models from [https://icgem.gfz.de/tom\\_longtime](https://icgem.gfz.de/tom_longtime) (name, year, max degree, data sources, direct .gfc download URL) - feed a row's name into `shLowLevel.fetchICGEM` to download.

| input   | size  | type                            | default  | kind       |
|---------|-------|---------------------------------|----------|------------|
| Type    | 1 x 1 | string ∈ ["static", "temporal"] | "static" | name-value |
| Series  | 1 x 1 | string                          | ""       | name-value |
| Source  | 1 x 1 | string                          | ""       | name-value |
| Timeout | 1 x 1 | double                          | 60       | name-value |

| output | description                                                                                                                |
|--------|----------------------------------------------------------------------------------------------------------------------------|
| T      | table: static: name, year, degree, data, url temporal: group, center, series, path, url, zip<br>temporal+Series: name, url |
| info   | struct: source, retrieved, note                                                                                            |

#### Example

```
T = shLowLevel.listICGEM(Type = "temporal", Series = "ITSG-Grace2018");
```

### shLowLevel.fetchITSG

`[files, info] = shLowLevel.fetchITSG(months, Name=Value)`

Download ITSG monthly OR daily solutions from TU Graz. `FILES = shLowLevel.fetchITSG(2019:2020)` downloads all available monthly GSM solutions for the given years into `dataFolder/itsg_series` (websave, base MATLAB). Already-present files are skipped unless `Update (false)=true`, which re-downloads them with a safe swap (the fresh file is parse-verified before it replaces the old one); months that do not exist on the server (mission gap 2017-07..2018-05, intra-mission dropouts) are reported in `INFO.missing`, not errors.

| input   | size  | type                          | default                                | kind       |
|---------|-------|-------------------------------|----------------------------------------|------------|
| months  | —     | —                             | ""                                     | positional |
| Dest    | 1 x 1 | string                        | ""                                     | name-value |
| Nmax    | 1 x 1 | double                        | NaN                                    | name-value |
| Product | 1 x 1 | string ∈ ["monthly", "daily"] | "monthly"                              | name-value |
| Timeout | 1 x 1 | double                        | 60                                     | name-value |
| Proxy   | 1 x 1 | string                        | ""                                     | name-value |
| Update  | 1 x 1 | logical                       | false                                  | name-value |
| Quiet   | 1 x 1 | logical                       | false                                  | name-value |
| Release | 1 x 1 | string                        | ""                                     | name-value |
| BaseURL | 1 x 1 | string                        | "https://ftp.tugraz.at/pub/ITSG/GRACE" | name-value |
| Catalog | 1 x : | double                        | []                                     | name-value |

| output | description                                                     |
|--------|-----------------------------------------------------------------|
| files  | (1,:) string paths of all present files (new + existing)        |
| info   | struct: fetched, updated, skipped, missing (string arrays), url |

### Example

```
shLowLevel.fetchITSG(["2008-04", "2025-12"], Nmax = 96);
```

### shLowLevel.fetchICGEM

```
[file, info] = shLowLevel.fetchICGEM(model, Name=Value)
```

Download a static gravity field model from ICGEM. FILE = shLowLevel.fetchICGEM("EGM2008") resolves the model name against shLowLevel.listICGEM (case-insensitive exact match, else unique prefix) and downloads the .gfc into <dataFolder>/icgem. Existing files are skipped. Load with shCoefficients.read(FILE).

| input    | size  | type                                  | default  | kind       |
|----------|-------|---------------------------------------|----------|------------|
| model    | —     | —                                     | required | positional |
| Dest     | 1 x 1 | string                                | ""       | name-value |
| Timeout  | 1 x 1 | double                                | 300      | name-value |
| List     | —     | table                                 | table()  | name-value |
| Proxy    | 1 x 1 | string                                | ""       | name-value |
| Update   | 1 x 1 | logical                               | false    | name-value |
| Quiet    | 1 x 1 | logical                               | false    | name-value |
| Pause    | 1 x 1 | double                                | 3        | name-value |
| Retries  | 1 x 1 | double                                | 3        | name-value |
| Type     | 1 x 1 | string ∈ ["static", "temporal"]       | "static" | name-value |
| Files    | 1 x 1 | string                                | "*.gfc"  | name-value |
| Mode     | 1 x 1 | string ∈ ["auto", "archive", "files"] | "auto"   | name-value |
| FileList | —     | table                                 | table()  | name-value |

| output | description                                                                                                                                                                                                                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| file   | string local path(s): one per model, or every file of the series in temporal mode (bulk selections are concatenated)                                                                                                                                                                                             |
| info   | struct: url, skipped (true if already present) and mode (1,1) string reporting the path that actually ran - "model" (single static model)   "archive" (whole-series ZIP in one request)   "files" (per-file fallback)   "present" (nothing to do); a script can tell a fresh download from a no-op by this field |

### Example

```
f = shLowLevel.fetchICGEM("G0C006s");           % static model by name
```

### shLowLevel.fetchDDK

```
[files, info] = shLowLevel.fetchDDK(which, Name=Value)
```

Download DDK filter matrices (Wbd binaries) on demand. FILES = shLowLevel.fetchDDK(1:8) fetches the released anisotropic DDK decorrelation filters DDK1..DDK8 (Kusche 2007; Kusche et al. 2009; ~9.8 MB each) from the MIT-licensed strawpants/GRACE-filter repository into <dataFolder>/DDK. Existing files are skipped unless Update (false)=true, which re-downloads them with a safe swap (the fresh file is parse-verified by shLowLevel.readDDK before replacing the old one). DDK3 additionally ships inside the toolbox (test...

| input   | size  | type    | default  | kind       |
|---------|-------|---------|----------|------------|
| which   | 1 x : | double  | required | positional |
| Dest    | 1 x 1 | string  | ''       | name-value |
| Timeout | 1 x 1 | double  | 120      | name-value |
| Proxy   | 1 x 1 | string  | ''       | name-value |
| Update  | 1 x 1 | logical | false    | name-value |
| Quiet   | 1 x 1 | logical | false    | name-value |

| output | description                                      |
|--------|--------------------------------------------------|
| files  | (1,:) string present file paths (new + existing) |
| info   | struct: fetched, updated, skipped, names, url    |

### Example

```
shLowLevel.fetchDDK(1:8);                       % ~80 MB, skips existing
```

### shLowLevel.fetchTN

```
[files, info] = shLowLevel.fetchTN(Name=Value)
```

Download the TN-13/TN-14 low-degree completion files on demand. FILES = shLowLevel.fetchTN() fetches the GRACE/GRACE-FO technical notes that complete the standard processing chain into <dataFolder>/TN: TN-14 GSFC SLR C20/C30 replacement (shLowLevel.readTN14/applyTN14) TN-13 geocenter (degree 1) per provider (shLowLevel.readTN13/addDegree1) Source: the GFZ ISDC open document server (URLs verified 2026-08-07; the CSR/JPL/GFZ RL06.3 files and the GSFC TN-14 also ship as test fixtures, so the toolbox works ...

| input     | size  | type                           | default               | kind       |
|-----------|-------|--------------------------------|-----------------------|------------|
| Providers | 1 x : | string ∈ ["GFZ", "CSR", "JPL"] | ["GFZ", "CSR", "JPL"] | name-value |
| TN14      | 1 x 1 | logical                        | true                  | name-value |
| Release   | 1 x 1 | string                         | "RL06.3"              | name-value |
| Update    | 1 x 1 | logical                        | false                 | name-value |
| Dest      | 1 x 1 | string                         | ''                    | name-value |
| BaseURL   | 1 x 1 | string                         | ''                    | name-value |
| Proxy     | 1 x 1 | string                         | ''                    | name-value |
| Timeout   | 1 x 1 | double                         | 60                    | name-value |
| Quiet     | 1 x 1 | logical                        | false                 | name-value |

| output | description                                                                                 |
|--------|---------------------------------------------------------------------------------------------|
| files  | (1,:) string verified present file paths (new + updated + kept existing)                    |
| info   | (1,1) struct fetched / updated / skipped / failed (string arrays of file names), urls, dest |

### Example

```
shLowLevel.fetchTN(Update = true); % monthly refresh, safe swap
tn13 = shLowLevel.readTN13(fullfile(shLowLevel.dataFolder(), "TN", ...
    "TN-13_GEOC_CSR_RL06.3.txt"));
g1 = g.applyTN14(tn14).addDegree1(tn13);
```

### shLowLevel.dataFolder

```
folder = shLowLevel.dataFolder(newFolder)
```

Get or set the persistent user data folder of the toolbox. `F = shLowLevel.dataFolder()` returns the current data folder (created on demand). Default: <toolbox root>/data. `F = shLowLevel.dataFolder(PATH)` sets and persists a user folder (MATLAB preference, survives sessions: `setpref('shAnalysis','dataFolder')`). `F = shLowLevel.dataFolder("reset")` back to the default.

| input     | size  | type   | default | kind       |
|-----------|-------|--------|---------|------------|
| newFolder | 1 x 1 | string | ""      | positional |

| output | description                                            |
|--------|--------------------------------------------------------|
| folder | (1 x 1) string current data folder (created on demand) |

#### Example

```
shLowLevel.dataFolder("D:/grace_data") % shared download location
```

### shLowLevel.ddkNames

```
names = shLowLevel.ddkNames()
```

Released Wbd filenames for DDK1..DDK8 (strong -> weak). `NAMES = shLowLevel.ddkNames()` - single source of truth for the mapping (verified against the strawpants/GRACE-filter repository listing). Claude (Table 5), 2026-08-07 (v2.4.1).

| output | description                                  |
|--------|----------------------------------------------|
| names  | (1 x 8) string Wbd file names for DDK1..DDK8 |

#### Example

```
names = shLowLevel.ddkNames(); % Wbd file names DDK1..8
```

### shLowLevel.parseGraceFilename

```
meta = shLowLevel.parseGraceFilename(filename)
```

Product type and mid-epoch from a GRACE L2 filename. `META = shLowLevel.parseGraceFilename(FILENAME)` parses the standard GRACE / GRACE-FO Level-2 naming convention `PPP-2_YYYYDDD-YYYYDDD_*.gfc[.gz]` e.g. `GSM-2_2024032-2024060_GRFO_UTCSR_BA01_0600.gfc`, where `PPP` is one of `GSM`, `GAA`, `GAB`, `GAC`, `GAD` and `YYYYDDD` are year + day-of-year of the coverage span. Best-effort: fields are `NaN`/`"unknown"` when the pattern does not match (static models, renamed files); callers can always override via explicit options.

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| filename | —    | —    | required | positional |

| output | description                                                                                                   |
|--------|---------------------------------------------------------------------------------------------------------------|
| meta   | (1,1) struct fields: center/product/release (string), epoch/year/month (double), nmax (double, NaN if absent) |

#### Example

```
meta = shLowLevel.parseGraceFilename("GSM-2_2008095-2008120_...gz");
```

### shLowLevel.icgemDate2Year

```
y = shLowLevel.icgemDate2Year(d)
```

Convert ICGEM/GRACE yyyyymmdd(.hhmm) epochs to decimal years. `Y = shLowLevel.icgemDate2Year(D)` converts date codes of the form yyyyymmdd or yyyyymmdd.hhmm (as used in ICGEM gfct files, TN-13/TN-14) to decimal years, using day-of-year / (days in year) with proper leap years. Values that do not look like date codes ( $< 1.8e7$ , e.g. already-decimal years in legacy synthetic files) are returned UNCHANGED - this rule keeps pre-v2.1 files evaluating identically.

| input | size | type   | default  | kind       |
|-------|------|--------|----------|------------|
| d     | —    | double | required | positional |

| output | description                               |
|--------|-------------------------------------------|
| t      | (same size as input) double decimal years |

*Example*

```
t = shLowLevel.icgemDate2Year(20080415.0000); % -> 2008.29
```

### shLowLevel.writeGFC

`shLowLevel.writeGFC(filename, C, S, GM, R, Name=Value)`

Write Stokes coefficients to an ICGEM ASCII .gfc file. `shLowLevel.writeGFC(FILENAME, C, S, GM, R)` writes a static ICGEM-format gravity field file readable by `shLowLevel.shReadGFC` (round-trip tested to full double precision: coefficients are written with %22.14e).

| input      | size  | type    | default             | kind       |
|------------|-------|---------|---------------------|------------|
| filename   | —     | —       | required            | positional |
| C          | —     | double  | required            | positional |
| S          | —     | double  | required            | positional |
| GM         | 1 x 1 | double  | required            | positional |
| R          | 1 x 1 | double  | required            | positional |
| SigmaC     | —     | double  | []                  | name-value |
| SigmaS     | —     | double  | []                  | name-value |
| ModelName  | —     | —       | "shAnalysis_export" | name-value |
| TideSystem | —     | —       | "unknown"           | name-value |
| Comment    | —     | —       | ""                  | name-value |
| Sidecar    | 1 x 1 | logical | true                | name-value |

*Example*

```
shLowLevel.writeGFC("out.gfc", g.C, g.S, g.GM, g.R, Comment = "DDK3 filtered");
```

### shLowLevel.writeGrid

`shLowLevel.writeGrid(filename, grid, latDeg, lonDeg, Name=Value)`

Export gridded fields to CF-style netCDF (base MATLAB). `shLowLevel.writeGrid('ewh.nc', GRID, LAT, LON, Name ("field")="lwe_thickness", Units ("")="m", Epoch=2010.29)` writes a 2-D field or a 3-D stack (nlat x nlon x T with Epochs ([])=) using `nccreate/ncwrite` - directly readable back by `shLowLevel.readMascon` (roundtrip-tested), by GMT, panoply, xarray, etc. Time is stored as "days since 2002-01-01" like the mascon products.

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| filename | —     | —      | required | positional |
| grid     | —     | double | required | positional |
| latDeg   | : x 1 | double | required | positional |
| lonDeg   | : x 1 | double | required | positional |
| Name     | 1 x 1 | string | "field"  | name-value |

| input       | size  | type    | default      | kind       |
|-------------|-------|---------|--------------|------------|
| Units       | 1 x 1 | string  | ""           | name-value |
| Epochs      | : x 1 | double  | []           | name-value |
| LatType     | 1 x 1 | string  | "geocentric" | name-value |
| Description | 1 x 1 | string  | ""           | name-value |
| Sidecar     | 1 x 1 | logical | true         | name-value |

#### Example

```
shLowLevel.writeGrid("ewh.nc", grid, lat, lon); % netCDF/geotiff/ascii
```

### shLowLevel.writeAnimation

shLowLevel.writeAnimation(ts, filename, Name=Value)

Export a monthly series as an MP4 map animation. shLowLevel.writeAnimation(TS, 'ewh.mp4', quantity="ewh", kn ([])=kn) renders one shLowLevel.plotSHMap frame per epoch into an MPEG-4 via VideoWriter (base MATLAB), with a FIXED color scale over all epochs (robust 98th percentile across the whole stack - the single most common animation mistake is a per-frame scale).

| input      | size  | type                         | default  | kind       |
|------------|-------|------------------------------|----------|------------|
| ts         | 1 x 1 | shSeries                     | required | positional |
| filename   | —     | —                            | required | positional |
| quantity   | 1 x 1 | string                       | "ewh"    | name-value |
| kn         | —     | double                       | []       | name-value |
| hn         | —     | double                       | []       | name-value |
| lat        | 1 x : | double                       | -89:2:89 | name-value |
| lon        | 1 x : | double                       | 0:2:358  | name-value |
| FrameRate  | 1 x 1 | double                       | 4        | name-value |
| Sidecar    | 1 x 1 | logical                      | true     | name-value |
| CLim       | —     | double                       | []       | name-value |
| Coast      | 1 x 1 | logical                      | true     | name-value |
| Units      | 1 x 1 | string                       | ""       | name-value |
| Projection | 1 x 1 | string ∈ ["plate", "hammer"] | "plate"  | name-value |

#### Example

```
shLowLevel.writeAnimation(ts, "ewh_2019_2022.avi", quantity = "ewh", kn = kn); % .avi portable, .mp4 on Windows/
```

## Legendre, spectral tools & indexing

### shLowLevel.legendreALF

P = shLowLevel.legendreALF(nmax, lat)

Fully normalized (4-pi) associated Legendre functions Pbar\_nm. P = LEGENDREALF(NMAX, LAT) computes fully normalized associated Legendre functions up to degree/order NMAX for each latitude in LAT (radians, geocentric, scalar or vector), using the scaled forward column recursion of Holmes & Featherstone (2002): the u^m factor of each column is carried separately with a 1e-280 scale factor, which keeps the recursion overflow/underflow-free up to at least degree 2190 (EGM2008/XGM2019e) at all latitudes including nea...

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| nmax  | —    | —    | required | positional |
| lat   | —    | —    | required | positional |

| output | description                                                                                                 |
|--------|-------------------------------------------------------------------------------------------------------------|
| P      | (NMAX+1)x(NMAX+1)xnumel(LAT) double $P(n+1,m+1,k) = Pbar_{\{n,m\}}(\sin(\text{lat}(k)))$ , upper triangle 0 |

*Example*

```
P = shLowLevel.legendreALF(96, deg2rad(-89:89)); % stable to n=2190
```

### shLowLevel.legendreALFDeriv

```
[P, D, D2] = shLowLevel.legendreALFDeriv(nmax, latRad)
```

Fully normalized ALFs and their latitude derivatives.  $[P, D] = \text{shLowLevel.legendreALFDeriv}(NMAX, LATRAD)$  returns the 4pi-fully- normalized associated Legendre functions P (via `shLowLevel.legendreALF`, scaled Holmes-Featherstone, stable to degree 2190) and their exact first derivatives with respect to LATITUDE phi,

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| nmax   | 1 x 1 | double | required | positional |
| latRad | 1 x : | double | required | positional |

| output | description                                                                                                                                                                                                                                                                                                                                                                         |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P      | (nmax+1, nmax+1, nlat) double $Pbar_{nm}(\sin \text{lat})$                                                                                                                                                                                                                                                                                                                          |
| D      | (nmax+1, nmax+1, nlat) double $dPbar_{nm}/d\phi$ [1/rad] D2 (nmax+1, nmax+1, nlat) double $d^2Pbar_{nm}/d\phi^2$ [1/rad^2]: the SAME column stencil applied to D - exact, because the first-derivative identity holds degree-wise as a function of phi and differentiates term by term (Python-validated vs numerical second differences, rel $\sim 2e-7 = \text{FD floor}$ ; v2.4) |

*Example*

```
[P, dP] = shLowLevel.legendreALFDeriv(60, theta); % for deformation/tensor
```

### shLowLevel.legendreCached

```
P = shLowLevel.legendreCached(nmax, latRad)
```

Cached fully normalized Legendre functions.  $P = \text{shLowLevel.legendreCached}(NMAX, LATRAD)$  returns `shLowLevel.legendreALF(NMAX, LATRAD)`, serving repeated requests for the same (NMAX, LATRAD) from a process-wide cache. Cache hits are verified with an EXACT isequal comparison of the stored latitude vector -- a mismatched latVec can never silently return the wrong P (this removes the documented footgun of `shSynthesis`' raw 'P' passthrough).

| input  | size | type | default  | kind       |
|--------|------|------|----------|------------|
| nmax   | —    | —    | required | positional |
| latRad | —    | —    | required | positional |

| output | description                            |
|--------|----------------------------------------|
| P      | (nmax+1)x(nmax+1)xnumel(latRad) double |

*Example*

```
shLowLevel.legendreCached("clear") % drop the synthesis cache
```

### shLowLevel.ylm

```
Y = shLowLevel.ylm(latRad, lonRad, idx)
```

4-pi normalized SH basis vectors at arbitrary points.  $Y = \text{shLowLevel.ylm}(LATRAD, LONRAD, IDX)$  evaluates the basis at nPts points (geocentric latitude/longitude in radians, equal-length vectors), so that  $f(\text{points}) = Y * x$  for a coefficient vector x in IDX ordering. Vectorized over points via one `shLowLevel.legendreALF` call.

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| latRad | : x 1 | double | required | positional |

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| lonRad | : x 1 | double | required | positional |
| idx    | 1 x 1 | struct | required | positional |

| output | description           |
|--------|-----------------------|
| Y      | (nPts x idx.P) double |

*Example*

```
Y = shLowLevel.ylm(deg2rad(latPts), deg2rad(lonPts), idx); % design rows, idx order
```

### shLowLevel.shIndex

```
idx = shLowLevel.shIndex(Lmax, Name=Value)
```

Canonical order-major index for SH coefficient vectors. `idx = shLowLevel.shIndex(Lmax)` builds the index table used by all `shLowLevel` functions: order-major ordering, C before S per (n,m), degrees  $n = \text{MinDegree}..L_{\text{max}}$  (default `MinDegree = 2`; degree 0/1 and the SLR-C20 replacement are assumed to be handled upstream).

| input     | size  | type   | default  | kind       |
|-----------|-------|--------|----------|------------|
| Lmax      | 1 x 1 | double | required | positional |
| MinDegree | 1 x 1 | double | 2        | name-value |

| output | description                                                                                       |
|--------|---------------------------------------------------------------------------------------------------|
| idx    | struct: n/m/cs (P x 1), P (1 x 1), nmax, MinDegree - the canonical vector ordering of the toolbox |

*Example*

```
idx = shLowLevel.shIndex(60); % MinDegree = 2 default
```

### shLowLevel.vecFromCS

```
x = shLowLevel.vecFromCS(cnm, snm, idx)
```

Pack lower-triangular `Cnm/Snm` matrices into a `shLowLevel` vector. `x = shLowLevel.vecFromCS(cnm, snm, idx)` with `cnm`, `snm` of size  $(L_{\text{max}}+1) \times (L_{\text{max}}+1)$ , `cnm(n+1,m+1) = Cnm`, `snm(n+1,m+1) = Snm`.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| cnm   | —     | double | required | positional |
| snm   | —     | double | required | positional |
| idx   | 1 x 1 | struct | required | positional |

| output | description                                          |
|--------|------------------------------------------------------|
| x      | (P x 1) double coefficients gathered in idx ordering |

*Example*

```
x = shLowLevel.vecFromCS(g10.C, g10.S, idx); % triangle -> idx-ordered vector
```

### shLowLevel.csFromVec

```
[cnm, snm] = shLowLevel.csFromVec(x, idx)
```

Unpack a `shLowLevel` vector into lower-triangular `Cnm/Snm` matrices. `[cnm, snm] = shLowLevel.csFromVec(x, idx)`. Coefficients outside the index range ( $n < \text{MinDegree}$ ) are zero.

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| x     | : x 1 | double | required | positional |
| idx   | 1 x 1 | struct | required | positional |

| output | description                                               |
|--------|-----------------------------------------------------------|
| cnm    | (nmax+1 x nmax+1) double cosine coefficients, C(n+1, m+1) |
| snm    | (nmax+1 x nmax+1) double sine coefficients, S(:, 1) = 0   |

*Example*

```
[C, S] = shLowLevel.csFromVec(x, idx);           % vector -> triangle matrices
```

### shLowLevel.shDegreeRMS

```
spec = shLowLevel.shDegreeRMS(C, S, varargin)
```

Spectral (degree-domain) analysis of spherical harmonic coefficients. SPEC = SHDEGREERMS(C, S) computes, per degree  $n = 0..nmax$ :  $degVariance(n+1) = \sum_m (C_{nm}^2 + S_{nm}^2)$   $degRMS(n+1) = \sqrt{degVariance}$   $degAmplitude(n+1) = R * degRMS$  [geoid-height-equivalent, m]  $cumAmplitude(n+1) = R * \sqrt{cumsum(degVariance)}$  [cumulative, m]

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| C        | —    | —    | required | positional |
| S        | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                                                                                                                                                         |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| spec   | struct: degree (nmax+1 x 1), degVariance/degRMS/ degAmplitude, cumVariance/cumRMS/cumAmplitude, R, domain; errVariance/errRMS/errAmplitude when sigmas are supplied |

*Example*

```
spec = shLowLevel.shDegreeRMS(g.C, g.S);           % degree-amplitude spectrum struct
```

### shLowLevel.shOrderRMS

```
spec = shLowLevel.shOrderRMS(C, S, Name=Value)
```

Order-domain spectral diagnostics (striping direction). SPEC = shLowLevel.shOrderRMS(C, S) sums coefficient power per ORDER  $m$  - the complementary view to shLowLevel.shDegreeRMS and the natural axis for GRACE striping (correlated errors live at high orders):

| input  | size  | type   | default   | kind       |
|--------|-------|--------|-----------|------------|
| C      | —     | double | required  | positional |
| S      | —     | double | required  | positional |
| R      | 1 x 1 | double | 6378136.3 | name-value |
| n0     | 1 x 1 | double | 0         | name-value |
| sigmaC | —     | double | []        | name-value |
| sigmaS | —     | double | []        | name-value |

| output | description                                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| spec   | struct: order, ordVariance, ordRMS, ordAmplitude, cumVariance, cumRMS, cumAmplitude, (err*), R, domain = 'order' (consumed by plotSHSpectrum) |

*Example*

```
spec = shLowLevel.shOrderRMS(g.C, g.S);           % stripes live at high orders
```

### shLowLevel.shSpectralCrossover

```
[nCrossover, degreeInterp] = shLowLevel.shSpectralCrossover(spec)
```

Degree at which the error spectrum first exceeds the signal (degree-amplitude) spectrum -- a standard guide for choosing a truncation degree or Gaussian filter radius.

| input | size | type | default  | kind       |
|-------|------|------|----------|------------|
| spec  | —    | —    | required | positional |

| output       | description                                                |
|--------------|------------------------------------------------------------|
| nCrossover   | (1,1) double first degree with err >= signal (NaN if none) |
| degreeInterp | (1,1) double linearly interpolated crossing degree         |

*Example*

```
spec = shLowLevel.shDegreeRMS(g.C, g.S, 'sigmaC', g.sigmaC, 'sigmaS', g.sigmaS);
nc = shLowLevel.shSpectralCrossover(spec); % signal/error crossover degree
```

### shLowLevel.gaussLegendre

[x, w] = shLowLevel.gaussLegendre(n)

Gauss-Legendre nodes and weights on [-1, 1]. [X, W] = shLowLevel.gaussLegendre(N) via Golub-Welsch (symmetric eigenvalue problem of the Jacobi matrix). Exact for polynomials up to degree 2N-1: integral f(t) dt = sum(W .\* f(X)).

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| n     | 1 x 1 | double | required | positional |

| output | description                      |
|--------|----------------------------------|
| x      | (n,1) double nodes, ascending    |
| w      | (n,1) double weights, sum(w) = 2 |

*Example*

```
[xNodes, w] = shLowLevel.gaussLegendre(61); % quadrature nodes/weights
```

## Synthesis & analysis

### shLowLevel.shSynthesis

[grid, lat, lon, P] = shLowLevel.shSynthesis(C, S, GM, R, latVec, lonVec, varargin)

Spherical harmonic synthesis of a global/regional grid. [GRID, LAT, LON] = SHSYNTHESIS(C, S, GM, R, LATVEC, LONVEC) synthesizes the disturbing-potential-derived quantity (default: geoid height) on the grid LATVEC x LONVEC (degrees) from fully normalized (ICGEM) Stokes coefficients C, S at the reference sphere r = R.

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| C        | —    | —    | required | positional |
| S        | —    | —    | required | positional |
| GM       | —    | —    | required | positional |
| R        | —    | —    | required | positional |
| latVec   | —    | —    | required | positional |
| lonVec   | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                               |
|--------|-------------------------------------------|
| grid   | (nlat x nlon) double synthesized quantity |
| lat    | (1, nlat) double latitude grid [deg]      |
| lon    | (1, nlon) double longitude grid [deg]     |

| output | description                                                     |
|--------|-----------------------------------------------------------------|
| P      | (1,1) struct cached Legendre functions (pass back in for reuse) |

*Example*

```
grid = shLowLevel.shSynthesis(g.C, g.S, g.GM, g.R, -89:89, 0:359);
```

### shLowLevel.synthesisMatrix

```
[Y, w, grid] = shLowLevel.synthesisMatrix(idx, Name=Value)
```

Dense SH synthesis matrix on a Gauss-Legendre grid. [Y, W, GRID] = shLowLevel.synthesisMatrix(IDX) returns Y (Ngrid x P) with  $f = Y \cdot x$  for coefficient vectors  $x$  in IDX ordering (shLowLevel.shIndex), and quadrature weights W (Ngrid x 1), normalized such that the discrete analysis  $A = Y' \cdot \text{diag}(W)$  is EXACT for band-limited fields:  $A \cdot Y = \text{eye}(P)$  to machine precision (asserted in the test suite).

| input | size  | type   | default        | kind       |
|-------|-------|--------|----------------|------------|
| idx   | 1 x 1 | struct | required       | positional |
| NLat  | 1 x 1 | double | idx.Lmax + 1   | name-value |
| NLon  | 1 x 1 | double | 2*idx.Lmax + 2 | name-value |

| output | description                                                       |
|--------|-------------------------------------------------------------------|
| Y      | (NLat*NLon x idx.P) double synthesis matrix                       |
| w      | (NLat*NLon x 1) double quadrature weights, sum(w) = 1             |
| grid   | struct: theta, lambda [rad], latDeg (NLat x 1), lonDeg (NLon x 1) |

*Example*

```
[Y, w] = shLowLevel.synthesisMatrix(idx); % quadrature-consistent pair
```

### shLowLevel.shSynthesisDeformation

```
[up, north, east] = shLowLevel.shSynthesisDeformation(C, S, R, latDeg, lonDeg, Name=Value)
```

Elastic load deformation: up / north / east. [UP, NORTH, EAST] = shLowLevel.shSynthesisDeformation(C, S, R, LAT, LON, kn ("")=..., hn ("")=..., ln ("")=...) synthesizes the three components of the elastic surface deformation caused by the load expressed in the (residual) Stokes coefficients C, S (Wahr et al. 1998; Kusche & Schrama 2005 - the GRACE <-> GNSS loading comparison):

| input  | size  | type                       | default  | kind       |
|--------|-------|----------------------------|----------|------------|
| C      | —     | double                     | required | positional |
| S      | —     | double                     | required | positional |
| R      | 1 x 1 | double                     | required | positional |
| latDeg | 1 x : | double                     | required | positional |
| lonDeg | 1 x : | double                     | required | positional |
| kn     | —     | double                     | required | name-value |
| hn     | —     | double                     | required | name-value |
| ln     | —     | double                     | required | name-value |
| nmin   | 1 x 1 | double                     | 1        | name-value |
| Mode   | 1 x 1 | string ∈ ["grid","points"] | "grid"   | name-value |

| output | description                                                  |
|--------|--------------------------------------------------------------|
| up     | (nlat x nlon   npts) double vertical elastic deformation [m] |
| north  | same size horizontal north component [m]                     |
| east   | same size horizontal east component [m]                      |

### Example

```
[up, no, ea] = shLowLevel.shSynthesisDeformation(g.C, g.S, g.R, latPts, lonPts, kn = LN.kn, hn = LN.hn, ln = LN.ln)
```

### shLowLevel.shSynthesisGradientTensor

```
[comps, info] = shLowLevel.shSynthesisGradientTensor(C, S, GM, R, latDeg, lonDeg, Name=Value)
```

Full gravity gradient tensor in the local NEU frame. [COMPS, INFO] = shLowLevel.shSynthesisGradientTensor(C, S, GM, R, LAT, LON, Height (250e3)=250e3) synthesizes all six independent components of the disturbing gravity gradient tensor (GOCE-style diagnostics) in the local north-east-up frame at radius  $r = R + \text{Height}$ :

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| C      | —     | double | required | positional |
| S      | —     | double | required | positional |
| GM     | 1 × 1 | double | required | positional |
| R      | 1 × 1 | double | required | positional |
| latDeg | 1 × : | double | required | positional |
| lonDeg | 1 × : | double | required | positional |
| Height | 1 × 1 | double | 250e3    | name-value |
| nmin   | 1 × 1 | double | 2        | name-value |

| output | description                                                                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| comps  | struct of (nlat,nlon) double [1/s <sup>2</sup> ]: uu, nn, ee, un, ue, ne (symmetric tensor; 1 Eotvos = 1e-9 1/s <sup>2</sup> ). Components involving east are NaN at  lat  = 90. |
| info   | struct: maxTraceResidual (max  trace /max G  over the grid, excluding poles) - the built-in Laplace self-check                                                                   |

### Example

```
T = shLowLevel.shSynthesisGradientTensor(g.C, g.S, g.GM, g.R, lat, lon, Height = 450e3);
```

### shLowLevel.shAnalysisGrid

```
[C, S, info] = shLowLevel.shAnalysisGrid(grid, latVec, lonVec, nmax, Name=Value)
```

Spherical harmonic analysis: estimate Stokes coefficients from gridded or scattered data (the inverse of shLowLevel.shSynthesis).

| input     | size  | type                           | default        | kind       |
|-----------|-------|--------------------------------|----------------|------------|
| grid      | —     | double                         | required       | positional |
| latVec    | —     | double                         | required       | positional |
| lonVec    | —     | double                         | required       | positional |
| nmax      | 1 × 1 | double                         | required       | positional |
| Method    | 1 × 1 | string ∈ ["auto","rings","ls"] | "auto"         | name-value |
| quantity  | —     | —                              | 'geoid'        | name-value |
| GM        | 1 × 1 | double                         | 3.986004415e14 | name-value |
| R         | 1 × 1 | double                         | 6378136.3      | name-value |
| kn        | —     | double                         | []             | name-value |
| hn        | —     | double                         | []             | name-value |
| rho_ave   | 1 × 1 | double                         | 5517           | name-value |
| rho_water | 1 × 1 | double                         | 1000           | name-value |
| Weights   | 1 × 1 | string ∈ ["none","coslat"]     | "none"         | name-value |
| Kaula     | 1 × 1 | double                         | 0              | name-value |

| input      | size  | type                                   | default         | kind       |
|------------|-------|----------------------------------------|-----------------|------------|
| ChunkSize  | 1 x 1 | double                                 | 2000            | name-value |
| LatType    | 1 x 1 | string $\in$ ["geocentric","geodetic"] | "geocentric"    | name-value |
| Flattening | 1 x 1 | double                                 | 1/298.257223563 | name-value |

| output | description                                                                                              |
|--------|----------------------------------------------------------------------------------------------------------|
| C      | (nmax+1 x nmax+1) double estimated cosine coefficients                                                   |
| S      | (nmax+1 x nmax+1) double estimated sine coefficients                                                     |
| info   | (1,1) struct fields: condest (1,1 double), nObs (1,1 double), kaula (1,1 double, NaN when unregularized) |

#### Example

```
g2 = shLowLevel.shAnalysisGrid(grid, lat, lon, 10); % exact on ring grids
```

### shLowLevel.kernelFactors

```
kernel = shLowLevel.kernelFactors(quantity, nmax, GM, R, Name=Value)
```

Degree-dependent spectral factors for SH synthesis/analysis. `KERNEL = shLowLevel.kernelFactors(QUANTITY, NMAX, GM, R, kn ([])=..., ...)` returns the (NMAX+1)x1 factor  $f_n$  such that a field of QUANTITY is  $q(lat,lon) = \sum_n f_n * \sum_m Pbar_{nm} (C_{nm} \cos + S_{nm} \sin)$ . Shared by `shLowLevel.shSynthesis` (multiply) and `shLowLevel.shAnalysisGrid` (divide); the single source of truth for the quantity definitions.

| input     | size  | type   | default  | kind       |
|-----------|-------|--------|----------|------------|
| quantity  | —     | —      | required | positional |
| nmax      | 1 x 1 | double | required | positional |
| GM        | 1 x 1 | double | required | positional |
| R         | 1 x 1 | double | required | positional |
| kn        | —     | double | []       | name-value |
| hn        | —     | double | []       | name-value |
| rho_ave   | 1 x 1 | double | 5517     | name-value |
| rho_water | 1 x 1 | double | 1000     | name-value |
| Height    | 1 x 1 | double | 0        | name-value |

| output | description                                                                                       |
|--------|---------------------------------------------------------------------------------------------------|
| kernel | (nmax+1,1) double (zeros where the quantity carries no information, e.g. n=1 for gravity_anomaly) |

#### Example

```
kf = shLowLevel.kernelFactors("ewh", 60, GM, R, kn = kn);
```

### shLowLevel.normalFieldCS

```
[Cn, info] = shLowLevel.normalFieldCS(nmax, Name=Value)
```

Even zonal harmonics of an ellipsoidal normal field. `[CN, INFO] = shLowLevel.normalFieldCS(NMAX, System="WGS84")` returns the fully normalized even zonal coefficients  $Cbar_{\{2n,0\}}$  of the normal gravity field, COMPUTED from the defining constants (GM (NaN), a (NaN), f (NaN), omega (NaN)) via the closed-form theory (Heiskanen & Moritz 1967, sect. 2-9) - no coefficient tables anywhere:

| input  | size  | type                           | default  | kind       |
|--------|-------|--------------------------------|----------|------------|
| nmax   | 1 x 1 | double                         | required | positional |
| System | 1 x 1 | string $\in$ ["WGS84","GRS80"] | "WGS84"  | name-value |
| GM     | 1 x 1 | double                         | NaN      | name-value |
| a      | 1 x 1 | double                         | NaN      | name-value |

| input | size  | type   | default | kind       |
|-------|-------|--------|---------|------------|
| f     | 1 x 1 | double | NaN     | name-value |
| omega | 1 x 1 | double | NaN     | name-value |

| output | description                                                                                                                                                                                       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cn     | (nmax+1,1) double Cbar_{n,0} (zero at odd/omitted degrees; Cn(1) = 1 represents the central GM/r term - subtract it ONLY together with a matching degree-0 of the model, see subtractNormalField) |
| info   | struct: GM, a, f, omega, J2, e2, m, q0, system                                                                                                                                                    |

*Example*

```
[Cn, info] = shLowLevel.normalFieldCS(96, System = "WGS84");
```

### shLowLevel.rescaleGMR

```
[C2, S2, sig] = shLowLevel.rescaleGMR(C, S, GM1, R1, GM2, R2, sigmaC, sigmaS)
```

Convert Stokes coefficients between (GM, R) reference values. [C2, S2] = shLowLevel.rescaleGMR(C, S, GM1, R1, GM2, R2) re-expresses coefficients given w.r.t. (GM1, R1) in the reference (GM2, R2):

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| C      | —     | double | required | positional |
| S      | —     | double | required | positional |
| GM1    | 1 x 1 | double | required | positional |
| R1     | 1 x 1 | double | required | positional |
| GM2    | 1 x 1 | double | required | positional |
| R2     | 1 x 1 | double | required | positional |
| sigmaC | —     | double | []       | positional |
| sigmaS | —     | double | []       | positional |

| output | description                                        |
|--------|----------------------------------------------------|
| C2, S2 | (n1,n1) double                                     |
| sig    | struct with fields C, S (empty if no sigmas given) |

*Example*

```
[C2, S2] = shLowLevel.rescaleGMR(g.C, g.S, g.GM, g.R, GMref, Rref);
```

### shLowLevel.geodetic2geocentric

```
latGC = shLowLevel.geodetic2geocentric(latGD, Name=Value)
```

Convert geodetic to geocentric latitude. LATGC = shLowLevel.geodetic2geocentric(LATGD) converts geodetic (ellipsoidal) latitudes to geocentric latitudes via  $\tan(\text{latGC}) = (1 - f)^2 * \tan(\text{latGD})$ . All spherical-harmonic routines in this toolbox (synthesis, analysis, Legendre functions) expect GEOCENTRIC latitude; user grids from maps, GIS or mascon products are almost always GEODETIC. Forgetting this conversion biases mid-latitude values by up to ~0.19 deg of latitude (~21 km) - a classic silent error source.

| input      | size  | type   | default         | kind       |
|------------|-------|--------|-----------------|------------|
| latGD      | —     | double | required        | positional |
| Flattening | 1 x 1 | double | 1/298.257223563 | name-value |

| output | description                                                  |
|--------|--------------------------------------------------------------|
| latGC  | double array geocentric latitudes [deg], poles/equator exact |

*Example*

```
latGc = shLowLevel.geodetic2geocentric(latGd); % before synthesis
```

### shLowLevel.geocentric2geodetic

```
latGD = shLowLevel.geocentric2geodetic(latGC, Name=Value)
```

Convert geocentric to geodetic latitude. LATGD = shLowLevel.geocentric2geodetic(LATGC) inverts shLowLevel.geodetic2geocentric:  $\tan(\text{latGD}) = \tan(\text{latGC}) / (1 - f)^2$ .

| input      | size  | type   | default         | kind       |
|------------|-------|--------|-----------------|------------|
| latGC      | —     | double | required        | positional |
| Flattening | 1 x 1 | double | 1/298.257223563 | name-value |

| output | description                           |
|--------|---------------------------------------|
| latGD  | double array geodetic latitudes [deg] |

*Example*

```
latGd = shLowLevel.geocentric2geodetic(latGc); % WGS84
```

### shLowLevel.evalMask

```
[mask, grid] = shLowLevel.evalMask(idx, region, Name=Value)
```

Evaluate a region definition on the toolbox quadrature grid. [MASK, GRID] = shLowLevel.evalMask(IDX, REGION) evaluates REGION on the Gauss-Legendre quadrature grid of shLowLevel.synthesisMatrix(IDX) and returns MASK (Ngrid x 1, in [0,1]) plus the GRID struct. Shared by shLowLevel.basinKernel and shLowLevel.slepianBasis.

| input      | size  | type   | default   | kind       |
|------------|-------|--------|-----------|------------|
| idx        | 1 x 1 | struct | required  | positional |
| region     | —     | —      | required  | positional |
| BufferKm   | 1 x 1 | double | 0         | name-value |
| R          | 1 x 1 | double | 6378136.3 | name-value |
| OverSample | 1 x 1 | double | 2         | name-value |

| output | description                                                      |
|--------|------------------------------------------------------------------|
| mask   | (Ngrid,1) double in [0,1]                                        |
| grid   | struct from shLowLevel.synthesisMatrix (latDeg, lonDeg per ring) |

*Example*

```
[mask, area] = shLowLevel.evalMask(idx, capSpec); % quadrature-grid mask
```

## Filtering & operators

### shLowLevel.shGaussianWeights

```
Wn = shLowLevel.shGaussianWeights(nmax, radius_km, varargin)
```

Degree-domain weights for isotropic Gaussian smoothing. WN = SHGAUSSIANWEIGHTS(NMAX, RADIUS\_KM) computes the Jekeli (1981) Gaussian averaging function weights Wn, n = 0..NMAX, for an averaging radius RADIUS\_KM [km] (half-width at which the spatial weighting function drops to 1/2). Standard GRACE/GRACE-FO mass-change smoothing.

| input     | size | type | default  | kind       |
|-----------|------|------|----------|------------|
| nmax      | —    | —    | required | positional |
| radius_km | —    | —    | required | positional |
| varargin  | —    | —    | required | positional |

| output | description                                                                  |
|--------|------------------------------------------------------------------------------|
| Wn     | (nmax+1 x 1) double Jekeli (1981) weights, W(1) = 1, monotone non-increasing |

*Example*

```
w = shLowLevel.shGaussianWeights(60, 350); % Jekeli recursion
```

### shLowLevel.shGaussianFilter

```
[Cf, Sf, Wn] = shLowLevel.shGaussianFilter(C, S, radius_km, varargin)
```

Apply isotropic Gaussian smoothing to Stokes coefficients. [CF, SF] = SHGAUSSIANFILTER(C, S, RADIUS\_KM) multiplies each degree n row of C, S by the Jekeli (1981) weight Wn(n), see SHGAUSSIANWEIGHTS. Standard pre-processing step for GRACE/GRACE-FO mass-change maps before spatial synthesis, used to suppress high-degree noise.

| input     | size | type | default  | kind       |
|-----------|------|------|----------|------------|
| C         | —    | —    | required | positional |
| S         | —    | —    | required | positional |
| radius_km | —    | —    | required | positional |
| varargin  | —    | —    | required | positional |

| output | description                                           |
|--------|-------------------------------------------------------|
| Cf     | (nmax+1 x nmax+1) double smoothed cosine coefficients |
| Sf     | (nmax+1 x nmax+1) double smoothed sine coefficients   |
| Wn     | (nmax+1 x 1) double Jekeli degree weights applied     |

*Example*

```
[Cf, Sf] = shLowLevel.shGaussianFilter(g.C, g.S, 350);
```

### shLowLevel.shFanFilter

```
[Cf, Sf] = shLowLevel.shFanFilter(C, S, rDegKm, rOrdKm, Name=Value)
```

Han fan filter: separable degree x order Gaussian smoothing. [CF, SF] = shLowLevel.shFanFilter(C, S, RDEGKM, RORDKM) applies the fan filter of Han et al. (2005): the product of a Jekeli Gaussian in DEGREE and one in ORDER,  $W_{nm} = W_n(rDeg) * W_m(rOrd)$ , damping high orders (the striping direction) harder than an isotropic Gaussian of equal degree radius - the cheap anisotropic benchmark between Jekeli and DDK/tvANS. Common choice: rOrd = rDeg (original fan) or rOrd < rDeg for stronger destriping.

| input  | size  | type   | default   | kind       |
|--------|-------|--------|-----------|------------|
| C      | —     | double | required  | positional |
| S      | —     | double | required  | positional |
| rDegKm | 1 x 1 | double | required  | positional |
| rOrdKm | 1 x 1 | double | required  | positional |
| R      | 1 x 1 | double | 6378136.3 | name-value |

| output | description                          |
|--------|--------------------------------------|
| Cf, Sf | (n1,n1) double filtered coefficients |

*Example*

```
[C2, S2] = shLowLevel.shFanFilter(g.C, g.S, 350, 200);
```

### shLowLevel.shDestripe

```
[Cf, Sf] = shLowLevel.shDestripe(C, S, varargin)
```

Remove correlated (north-south striping) noise from Stokes coefficients, following Swenson & Wahr (2006).

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| C        | —    | —    | required | positional |
| S        | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                                            |
|--------|--------------------------------------------------------|
| Cf     | (nmax+1 x nmax+1) double destriped cosine coefficients |
| Sf     | (nmax+1 x nmax+1) double destriped sine coefficients   |

#### Example

```
[Cf, Sf] = shLowLevel.shDestripe(g.C, g.S, minOrder = 6);
```

### shLowLevel.applyDDK

```
[Cf, Sf] = shLowLevel.applyDDK(C, S, W)
```

Apply an order-block-diagonal (DDK-style) filter to coefficients. [CF, SF] = shLowLevel.applyDDK(C, S, W) applies the filter container from shLowLevel.readDDK: within each (order m, cs) block, the degree vector c(n1..n2) is replaced by W.blocks(k).M \* c. Degrees present in C but not covered by any block pass through UNCHANGED (typically n < 2). User-facing wrappers: g.applyDDK(W), ts.applyDDK(W).

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| C     | —     | double | required | positional |
| S     | —     | double | required | positional |
| W     | 1 x 1 | struct | required | positional |

| output | description                                               |
|--------|-----------------------------------------------------------|
| Cf     | (nmax+1 x nmax+1) double DDK-filtered cosine coefficients |
| Sf     | (nmax+1 x nmax+1) double DDK-filtered sine coefficients   |

#### Example

```
W = shLowLevel.readDDK("DDK3", Nmax = 60);
[Cf, Sf] = shLowLevel.applyDDK(g.C, g.S, W);
```

### shLowLevel.tvANSFilter

```
[Xf, op, info] = shLowLevel.tvANSFilter(X, tYears, idx, Name=Value)
```

Time-variable anisotropic Wiener filtering of a SH series. [XF, OP, INFO] = shLowLevel.tvANSFilter(X, TYEARS, IDX) runs the tvANS chain on the coefficient series X (P x T, IDX ordering, shLowLevel.shIndex):

| input        | size  | type                              | default               | kind       |
|--------------|-------|-----------------------------------|-----------------------|------------|
| X            | —     | double                            | required              | positional |
| tYears       | : x 1 | double                            | required              | positional |
| idx          | 1 x 1 | struct                            | required              | positional |
| NoiseCov     | —     | double                            | []                    | name-value |
| Constraints  | —     | double                            | []                    | name-value |
| SignalMode   | —     | — ∈ {'isotropic','inhomogeneous'} | 'isotropic'           | name-value |
| NIterSignal  | 1 x 1 | double                            | 3                     | name-value |
| Robust       | 1 x 1 | logical                           | false                 | name-value |
| Shrinkage    | 1 x 1 | double                            | 0.1                   | name-value |
| VCEMinDegree | 1 x 1 | double                            | round(2/3 * idx.Lmax) | name-value |
| VCEBands     | 1 x : | double                            | []                    | name-value |

| input  | size | type                                                 | default | kind       |
|--------|------|------------------------------------------------------|---------|------------|
| Blocks | —    | — $\in \{\text{'auto'}, \text{'on'}, \text{'off'}\}$ | 'auto'  | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Xf     | (P,T) double filtered series (deterministic part restored)                                                                                                                                                                                                                                                                                                                                                                                              |
| op     | struct filter operator: Ut, V, lam, s, Ac, SA, M, idx, tYears, model, Xres, Xfres (use with shLowLevel.opApply / shLowLevel.basinDeconvolve / shLowLevel.resolutionMap)                                                                                                                                                                                                                                                                                 |
| info   | struct noise/signal/VCE diagnostics; additionally info.sigmaXfres (P x T): 1-sigma posterior uncertainty of the filtered residual per coefficient/month, $\text{diag}((I-W_t)S) = (V.^2)*(s_t*\text{lam}./(\text{lam}+s_t))$ (validated to 3.5e-16 against the direct matrix product). With constraints (v2.5) this is the EXACT diagonal of $(W-I)S(W-I)' + s*W*N*W'$ - the earlier unconstrained formula underestimated along constrained directions. |

#### Example

```
[Xf, op, info] = shLowLevel.tvANSFilter(X, tYears, idx);
sigma = info.sigmaXfres; % exact posterior (v2.5 incl. constraints)
```

### shLowLevel.buildNoiseCov

```
[N, info] = shLowLevel.buildNoiseCov(Xres, idx, Name=Value)
```

Monthly-solution noise covariance (shape matrix). [N, info] = shLowLevel.buildNoiseCov(Xres, idx) estimates the noise covariance from the residual coefficient series Xres (P x T, residuals about the deterministic model). Two modes:

| input     | size  | type                                          | default     | kind       |
|-----------|-------|-----------------------------------------------|-------------|------------|
| Xres      | —     | double                                        | required    | positional |
| idx       | 1 x 1 | struct                                        | required    | positional |
| Mode      | —     | — $\in \{\text{'empirical'}, \text{'full'}\}$ | 'empirical' | name-value |
| FullCov   | —     | double                                        | []          | name-value |
| Shrinkage | 1 x 1 | double                                        | 0.1         | name-value |
| Assemble  | —     | — $\in \{\text{'full'}, \text{'blocks'}\}$    | 'full'      | name-value |

| output | description                                                                                                  |
|--------|--------------------------------------------------------------------------------------------------------------|
| N      | (P x P) double or block struct noise covariance (order/parity block-diagonal for Assemble ('full')='blocks') |
| info   | struct: mode, shrinkage, blocks metadata                                                                     |

#### Example

```
N = shLowLevel.buildNoiseCov(Xres, idx); % empirical order/parity blocks
```

### shLowLevel.buildSignalCov

```
[S, info] = shLowLevel.buildSignalCov(Xres, N, idx, Name=Value)
```

Iterative, data-driven signal covariance. [S, info] = shLowLevel.buildSignalCov(Xres, N, idx) estimates the signal covariance from the residual series itself - no hydrology prior.

| input    | size  | type                                                   | default     | kind       |
|----------|-------|--------------------------------------------------------|-------------|------------|
| Xres     | —     | double                                                 | required    | positional |
| N        | —     | —                                                      | required    | positional |
| idx      | 1 x 1 | struct                                                 | required    | positional |
| Mode     | —     | — $\in \{\text{'isotropic'}, \text{'inhomogeneous'}\}$ | 'isotropic' | name-value |
| Nlter    | 1 x 1 | double                                                 | 3           | name-value |
| FloorRel | 1 x 1 | double                                                 | 1e-8        | name-value |

| input     | size  | type    | default | kind       |
|-----------|-------|---------|---------|------------|
| MapSmooth | 1 x 1 | logical | true    | name-value |

| output | description                                      |
|--------|--------------------------------------------------|
| S      | (P x P) double signal covariance in idx ordering |
| info   | struct: mode, iterations, degree-variance model  |

*Example*

```
S = shLowLevel.buildSignalCov(Xres, N, idx, Mode = "isotropic");
```

### shLowLevel.vceRescale

```
s = shLowLevel.vceRescale(N, Xres, idx, Name=Value)
```

Per-month variance factors for the noise covariance shape.  $s = \text{shLowLevel.vceRescale}(N, Xres, idx)$  estimates one variance factor per month such that  $N_t = s(t) * N$ . The factor is the robust (median-based, chi-square(1) corrected) ratio between the observed squared residuals and the formal variances, evaluated over noise-dominated coefficients ( $n \geq \text{MinDegree}$ , default  $\text{round}(2/3 * \text{idx.Lmax})$ ). This is the single-component Helmert VCE specialization; it absorbs the month-to-month noise level changes (orbit geometry,...

| input     | size  | type    | default                               | kind       |
|-----------|-------|---------|---------------------------------------|------------|
| N         | —     | —       | required                              | positional |
| Xres      | —     | double  | required                              | positional |
| idx       | 1 x 1 | struct  | required                              | positional |
| MinDegree | 1 x 1 | double  | $\text{round}(2/3 * \text{idx.Lmax})$ | name-value |
| Rows      | —     | logical | true(0, 1)                            | name-value |

| output | description                                                                   |
|--------|-------------------------------------------------------------------------------|
| s      | (T x 1) or (nBands x T) double monthly (band-wise) VCE noise variance factors |

*Example*

```
s = shLowLevel.vceRescale(N, Xres, idx);           % monthly noise factors
```

### shLowLevel.opApply

```
Yout = shLowLevel.opApply(op, Z, t, mode)
```

Apply the month-t filter operator  $W_t$  (or  $W_t'$ ) to columns of Z.  $Yout = \text{shLowLevel.opApply}(op, Z, t)$  computes  $W_t * Z$   $Yout = \text{shLowLevel.opApply}(op, Z, t, 'transp')$  computes  $W_t' * Z$

| input | size  | type                                        | default   | kind       |
|-------|-------|---------------------------------------------|-----------|------------|
| op    | 1 x 1 | struct                                      | required  | positional |
| Z     | —     | double                                      | required  | positional |
| t     | 1 x 1 | double                                      | required  | positional |
| mode  | —     | $\in \{\text{'forward'}, \text{'transp'}\}$ | 'forward' | positional |

| output | description                                          |
|--------|------------------------------------------------------|
| Yout   | (P x T) double operator applied to every column of Y |

*Example*

```
xf = shLowLevel.opApply(op, op.Xres(:, 1), 1);    %  $W_1 * x$  without the P x P matrix
```

### shLowLevel.resolutionMap

```
res = shLowLevel.resolutionMap(op, t, latDeg, lonDeg, Name=Value)
```

Filter-kernel half-widths: per-point, per-azimuth resolution.  $RES = shLowLevel.resolutionMap(OP, T, LATDEG, LONDEG)$  evaluates, for each query point, the effective smoothing kernel of month  $T$ :  $xf(P) = y_P' * W_t * x \Rightarrow$  kernel coefficients  $k = W_t' * y_P$  and finds the great-circle distance  $\psi_{1/2}$  at which the kernel drops to half its peak, along  $NAz$  (8) azimuths. Replaces the single "equivalent Gaussian radius" with a spatially and azimuthally resolved resolution product (N-S vs E-W half-width ratio = stripin...

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| op     | 1 x 1 | struct | required | positional |
| t      | 1 x 1 | double | required | positional |
| latDeg | : x 1 | double | required | positional |
| lonDeg | : x 1 | double | required | positional |
| NAz    | 1 x 1 | double | 8        | name-value |
| PsiMax | 1 x 1 | double | 20       | name-value |
| NPsi   | 1 x 1 | double | 120      | name-value |

| output | description                                              |
|--------|----------------------------------------------------------|
| res    | (nlat x nlon) double local resolution [km] of the filter |

*Example*

```
res = shLowLevel.resolutionMap(op, 1, -60:30:60, 0:60:300); % epoch 1
```

## Basins, uncertainty & analysis tools

### shLowLevel.basinKernel

```
[b, info] = shLowLevel.basinKernel(idx, region, Name=Value)
```

Band-limited basin kernel(s) from a region definition.  $[B, INFO] = shLowLevel.basinKernel(IDX, REGION)$  builds the SH coefficient vector of the band-limited indicator function of  $REGION$  by exact Gauss-Legendre quadrature,  $b = Y' * (w .* mask)$ , ready for  $shSeries.basinAverage$  /  $shLowLevel.basinDeconvolve$  ( $P \times 1$ ,  $IDX$  ordering). Replaces the error-prone manual  $Y'*(w.*mask)$  construction.

| input      | size  | type   | default   | kind       |
|------------|-------|--------|-----------|------------|
| idx        | 1 x 1 | struct | required  | positional |
| region     | —     | —      | required  | positional |
| BufferKm   | 1 x 1 | double | 0         | name-value |
| TaperKm    | 1 x 1 | double | 0         | name-value |
| R          | 1 x 1 | double | 6378136.3 | name-value |
| OverSample | 1 x 1 | double | 2         | name-value |

| output | description                                                                                       |
|--------|---------------------------------------------------------------------------------------------------|
| b      | (P,1) double kernel in $IDX$ ordering                                                             |
| info   | struct: areaFraction (quadrature area of the mask / $4\pi$ ), nGridPoints, buffered/tapered flags |

*Example*

```
b = shLowLevel.basinKernel(idx, maskAmazon, TaperKm = 200);
```

### shLowLevel.basinDeconvolve

```
[avgHat, out] = shLowLevel.basinDeconvolve(B, op, Name=Value)
```

Unbiased basin averages by exact operator deconvolution.  $[avgHat, out] = shLowLevel.basinDeconvolve(B, op)$  removes filter attenuation and inter-basin leakage from basin averages, exploiting that  $W_t$  is a KNOWN linear operator (no hydrology-model scale factors).

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| B     | —     | double | required | positional |
| op    | 1 x 1 | struct | required | positional |
| Ridge | 1 x 1 | double | 0        | name-value |

| output | description                                                                                                              |
|--------|--------------------------------------------------------------------------------------------------------------------------|
| avg    | (K x T) double deconvolved basin averages                                                                                |
| out    | struct: c (K x T) deconvolved coefficients, sigma (K x T) 1-sigma incl. deterministic term (v2.5), avgNaive, attn, condA |

#### Example

```
[avg, out] = shLowLevel.basinDeconvolve(B, op); % op from ts.filter("tvANS")
errorbar(op.tYears, avg(1,:), out.sigma(1,:))
```

### shLowLevel.basinScaling

```
[k, info] = shLowLevel.basinScaling(op, b, tsModel, Name=Value)
```

Leakage/attenuation scaling factors by forward modelling. [K, INFO] = shLowLevel.basinScaling(OP, B, TSMODEL) estimates the classic basin scaling (gain) factor: push a MODEL series (hydrology model, mascon-derived field, synthetic pattern) through the SAME filter operator OP that processed the data, and regress true against filtered basin averages:  $k = \text{sum}(a_{\text{True}} .* a_{\text{Filt}}) / \text{sum}(a_{\text{Filt}}.^2)$  Scaled data series:  $k * (b' \times \text{Filtered}) / (b'b)$ . Complements shLowLevel.basinDeconvolve: deconvolution inverts the operator on a ...

| input    | size  | type     | default  | kind       |
|----------|-------|----------|----------|------------|
| op       | —     | —        | required | positional |
| b        | : x 1 | double   | required | positional |
| tsModel  | 1 x 1 | shSeries | required | positional |
| idx      | —     | struct   | struct() | name-value |
| tYears   | 1 x : | double   | []       | name-value |
| PerMonth | 1 x 1 | logical  | false    | name-value |

| output | description                                                                                               |
|--------|-----------------------------------------------------------------------------------------------------------|
| k      | (1,1) double regression scaling factor                                                                    |
| info   | struct: sigmaK (regression 1-sigma), kMonthly (T x 1, if PerMonth), matched (T x 1 logical), aTrue, aFilt |

#### Example

```
k = shLowLevel.basinScaling(op, b, tsHydModel); % scale factor vs a model series
```

### shLowLevel.fitDeterministicModel

```
[model, Xres, coef, A, coefSigma, resVar, ar1] = shLowLevel.fitDeterministicModel(X, tYears, Name=Value)
```

Bias/trend/(semi-)annual (+extra periods) per SH coefficient. [MODEL, XRES, COEF, A, COEFSIGMA, RESVAR] = shLowLevel.fitDeterministicModel(X, TYEARS, ...) fits, for each coefficient time series (rows of X, P x T), the model  $x(t) = a + b*tc + c*\cos(2*\pi*tc) + d*\sin(2*\pi*tc) + e*\cos(4*\pi*tc) + f*\sin(4*\pi*tc) + \text{sum\_k} [g\_k*\cos(2*\pi*tc/p\_k) + h\_k*\sin(2*\pi*tc/p\_k)]$  with  $tc = t - T_0$  (NaN) and optional extra periods  $p\_k$  [years] - e.g. the GRACE tidal alias periods  $S_2 = 161/365.25$ ,  $K_2 = 3.66$ ,  $K_1 = 7.48$ .

| input  | size  | type    | default  | kind       |
|--------|-------|---------|----------|------------|
| X      | —     | double  | required | positional |
| tYears | : x 1 | double  | required | positional |
| Robust | 1 x 1 | logical | false    | name-value |

| input     | size  | type    | default           | kind       |
|-----------|-------|---------|-------------------|------------|
| HuberK    | 1 x 1 | double  | 1.5               | name-value |
| MaxIter   | 1 x 1 | double  | 10                | name-value |
| T0        | 1 x 1 | double  | NaN               | name-value |
| Periods   | 1 x : | double  | double.empty(1,0) | name-value |
| Weights   | : x 1 | double  | []                | name-value |
| ARCorrect | 1 x 1 | logical | false             | name-value |
| Breaks    | 1 x : | double  | double.empty(1,0) | name-value |

| output    | description                                                                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| model     | P x T fitted deterministic part                                                                                                                                                                           |
| Xres      | P x T X - model                                                                                                                                                                                           |
| coef      | ncol x P [bias; trend; cosA; sinA; cosSA; sinSA; cosP1; sinP1; ...], ncol = 6 + 2*numel(Periods)                                                                                                          |
| A         | T x ncol design matrix (unweighted)                                                                                                                                                                       |
| coefSigma | ncol x P 1-sigma per coefficient: sqrt(resVar_p * diag(inv(A'WA))); OLS formula, validated by Monte Carlo (ratios 0.99-1.02). For Robust=true the final IRLS weights enter W (approximation, documented). |
| resVar    | 1 x P residual variance rss/(T - ncol)                                                                                                                                                                    |
| ar1       | 1 x P bias-corrected lag-1 residual auto- correlation actually applied (clamped to [0, 0.99]; 0 when ARCorrect=false)                                                                                     |

#### Example

```
[model, Xres, coef, A, sig] = shLowLevel.fitDeterministicModel(X, tYears, ARCorrect = true);
```

### shLowLevel.mcPropagate

```
out = shLowLevel.mcPropagate(fun, g, Name=Value)
```

Monte-Carlo uncertainty propagation through arbitrary chains. OUT = shLowLevel.mcPropagate(FUN, G) draws coefficient samples consistent with the formal errors of the shCoefficients G, pushes each through FUN, and returns empirical statistics - uncertainty propagation for ANY functional (filtered grids, basin averages, amplitude maps, ...) without deriving analytic formulas, and the standard cross-check for every analytic sigma in this toolbox.

| input       | size  | type            | default    | kind       |
|-------------|-------|-----------------|------------|------------|
| fun         | 1 x 1 | function_handle | required   | positional |
| g           | 1 x 1 | shCoefficients  | required   | positional |
| N           | 1 x 1 | double          | 500        | name-value |
| Cov         | —     | double          | []         | name-value |
| Idx         | —     | struct          | struct([]) | name-value |
| Seed        | —     | double          | []         | name-value |
| KeepSamples | 1 x 1 | logical         | false      | name-value |

| output | description                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------------------|
| out    | (1,1) struct fields: samples (K x nSamples double), mean, sigma (K x 1 double) Monte-Carlo moments of the functional |

#### Example

```
out = shLowLevel.mcPropagate(@(gs) gs.C(3, 1), g, N = 200); % any functional of g
```

### shLowLevel.errorMap

```
[sig, latDeg, lonDeg] = shLowLevel.errorMap(M, idx, latDeg, lonDeg, Name=Value)
```

Analytic formal-error maps from a full coefficient covariance. [SIG, LAT, LON] = shLowLevel.errorMap(M, IDX, LAT, LON, quantity ("geoid")="ewh", kn ([])=kn) computes the pointwise 1-sigma of a synthesized field from a full P x P covariance (e.g. shLowLevel.readSINEX(...,

| input    | size  | type   | default        | kind       |
|----------|-------|--------|----------------|------------|
| M        | —     | double | required       | positional |
| idx      | 1 x 1 | struct | required       | positional |
| latDeg   | 1 x : | double | required       | positional |
| lonDeg   | 1 x : | double | required       | positional |
| quantity | 1 x 1 | string | "geoid"        | name-value |
| kn       | —     | double | []             | name-value |
| hn       | —     | double | []             | name-value |
| GM       | 1 x 1 | double | 3.986004415e14 | name-value |
| R        | 1 x 1 | double | 6378136.3      | name-value |
| Height   | 1 x 1 | double | 0              | name-value |

| output | description                                              |
|--------|----------------------------------------------------------|
| sig    | (nlat x nlon) double 1-sigma of the synthesized quantity |
| latDeg | (1, nlat) double latitude grid used [deg]                |
| lonDeg | (1, nlon) double longitude grid used [deg]               |

#### Example

```
sig = shLowLevel.errorMap(M, idx, -89:4:89, 0:4:356, quantity = "ewh", kn = kn);
```

### shLowLevel.combineCenters

```
[tsComb, info] = shLowLevel.combineCenters(tsCell, Name=Value)
```

VCE-weighted combination of multi-center monthly series. [TSCOMB, INFO] = shLowLevel.combineCenters({tsITSG, tsCSR, tsGFZ}) combines monthly solutions from several processing centers into one series with one variance factor per (center, month), estimated by Foerstner variance-component iteration with PARTIAL REDUNDANCIES:

| input        | size  | type    | default  | kind       |
|--------------|-------|---------|----------|------------|
| tsCell       | 1 x : | cell    | required | positional |
| Tolerance    | 1 x 1 | double  | 0.05     | name-value |
| AllowMissing | 1 x 1 | logical | true     | name-value |
| Robust       | 1 x 1 | logical | false    | name-value |
| MaxIter      | 1 x 1 | double  | 6        | name-value |
| Tol          | 1 x 1 | double  | 1e-3     | name-value |

| output | description                                                                                                                                                                                                                                                                                            |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tsComb | shSeries combined stacks on the reference epochs (first center), sigmaCs/Ss = posterior sqrt(diag(H)) per month, provenance in history                                                                                                                                                                 |
| info   | struct: s2 (C x T variance factors, NaN where missing), weights (C x T, normalized 1/s2), redundancy (C x T), interCenterCorr (C x C, empirical residual correlation - the honesty diagnostic: VCE assumes independence, GRACE centers share data and background models), nIter, epochs, centers, note |

#### Example

```
tsC = shLowLevel.combineCenters({tsCSR, tsGFZ, tsJPL}); % VCE-weighted
```

### shLowLevel.eofAnalysis

```
[modes, pcs, varExplained, info] = shLowLevel.eofAnalysis(ts, Name=Value)
```

Empirical orthogonal functions of a coefficient series. [MODES, PCS, VAREXPLAINED, INFO] = shLowLevel.eofAnalysis(TS) decomposes the (typically residual) series into spatial modes and principal components via SVD of the flattened, time-centered coefficient stack:  $X = U S V'$ ; MODES{k} carries  $U(:,k) \cdot S(k,k) / \sqrt{T-1}$  as an shCoefficients (synthesize for the spatial pattern),  $PCS(:,k) = \sqrt{T-1} \cdot V(:,k)$  is the unit-variance temporal amplitude, so mode-k field \* pc reconstructs the mode's contribution exactly.

| input  | size  | type     | default  | kind       |
|--------|-------|----------|----------|------------|
| ts     | 1 x 1 | shSeries | required | positional |
| NModes | 1 x 1 | double   | 6        | name-value |
| Center | 1 x 1 | logical  | true     | name-value |

| output       | description                                     |
|--------------|-------------------------------------------------|
| modes        | 1 x K cell of shCoefficients (spatial patterns) |
| pcs          | (T,K) double, unit variance columns             |
| varExplained | (K,1) double, fraction in [0,1]                 |
| info         | struct: singular values, total variance, T      |

#### Example

```
[modes, pcs, ve] = shLowLevel.eofAnalysis(resid);
plot(resid.epochs, pcs(:, 1)) % leading principal component
```

### shLowLevel.slepianBasis

```
[G, lam, info] = shLowLevel.slepianBasis(idx, region, Name=Value)
```

Spatially concentrated (Slepian) basis for a region. [G, LAM, INFO] = shLowLevel.slepianBasis(IDX, REGION) solves the spherical concentration problem of Simons et al. (2006): find band-limited functions  $g = Y \cdot x$  maximizing the energy fraction inside REGION,  $\lambda = (x' K x) / (x' x)$ ,  $K = Y' \text{diag}(w \cdot \text{mask}) Y$ , by symmetric eigendecomposition of the localization kernel K (exact Gauss-Legendre quadrature). Columns of G are the Slepian taper coefficient vectors, orthonormal ( $G'G = I$ ), sorted by concentration LAM (...)

| input      | size  | type   | default   | kind       |
|------------|-------|--------|-----------|------------|
| idx        | 1 x 1 | struct | required  | positional |
| region     | —     | —      | required  | positional |
| NKeep      | —     | double | []        | name-value |
| BufferKm   | 1 x 1 | double | 0         | name-value |
| R          | 1 x 1 | double | 6378136.3 | name-value |
| OverSample | 1 x 1 | double | 2         | name-value |

| output | description                                                                                     |
|--------|-------------------------------------------------------------------------------------------------|
| G      | (P,NKeep) double taper coefficient vectors, $G'G = I$                                           |
| lam    | (NKeep,1) double concentrations, descending in [0,1]                                            |
| info   | struct: shannon (= areaFraction*P, exact by the addition theorem), areaFraction, lamAll (P x 1) |

#### Example

```
[G, lam] = shLowLevel.slepianBasis(idx, capSpec); % concentration basis
```

### shLowLevel.seaLevelFingerprint

```
[S, grid, info] = shLowLevel.seaLevelFingerprint(loadRegion, ocean, idx, Name=Value)
```

Elastic sea-level fingerprint (sea-level equation). [S, GRID, INFO] = shLowLevel.seaLevelFingerprint(LOADREGION, OCEAN, IDX, kn('')=kn, hn('')=hn) solves the elastic sea-level equation for a land load: the ocean does NOT rise uniformly - it follows the perturbed geoid minus the deformed

crust under global mass conservation (Farrell & Clark 1976, elastic limit):

| input      | size  | type   | default   | kind       |
|------------|-------|--------|-----------|------------|
| loadRegion | —     | —      | required  | positional |
| ocean      | —     | —      | required  | positional |
| idx        | 1 x 1 | struct | required  | positional |
| kn         | —     | double | required  | name-value |
| hn         | —     | double | required  | name-value |
| LoadValue  | 1 x 1 | double | 1         | name-value |
| rho_water  | 1 x 1 | double | 1000      | name-value |
| rho_ave    | 1 x 1 | double | 5517      | name-value |
| R          | 1 x 1 | double | 6378136.3 | name-value |
| OverSample | 1 x 1 | double | 2         | name-value |
| MaxIter    | 1 x 1 | double | 50        | name-value |
| Tol        | 1 x 1 | double | 1e-8      | name-value |

| output | description                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------------------|
| S      | (Ngrid,1) double relative sea-level change over the ocean [m per unit load], 0 on land                               |
| grid   | struct latDeg/lonDeg rings of the solver grid                                                                        |
| info   | struct: iterations, dS, massResidual [kg/m^2 sphere mean], eustatic [m], kappa, areaFractionOcean, S2D (nlat x nlon) |

*Example*

```
S = shLowLevel.seaLevelFingerprint(gLoad, oceanMask, idx, kn = LN.kn, hn = LN.hn);
```

### shLowLevel.pctile

```
v = shLowLevel.pctile(x, p)
```

Percentile without the Statistics Toolbox (base MATLAB only).  $V = \text{shLowLevel.pctile}(X, P)$  returns the  $P$ -th percentile(s) (0..100) of the finite values of  $X(:)$ , using the same linear interpolation between order statistics as the Statistics Toolbox `pctile`: sample  $k$  maps to percentile  $100 \cdot (k-0.5)/n$ .  $P$  may be a vector (v2.5.1);  $V$  has the same size as  $P$ . NaN/Inf are ignored; empty input yields NaN(size(P)).

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| x     | —     | double | required | positional |
| p     | 1 x : | double | required | positional |

| output | description                                       |
|--------|---------------------------------------------------|
| v      | (size(p)) double percentile value per requested p |

*Example*

```
q = shLowLevel.pctile(res, [16 50 84]); % base-MATLAB percentiles
```

## Comparison & validation metrics (v2.6.0)

### shLowLevel.compareSolutions

```
[rep, h] = shLowLevel.compareSolutions(g1, g2, Name=Value)
```

Full spectral + spatial comparison of two solutions.  $REP = \text{shLowLevel.compareSolutions}(G1, G2)$  compares two shCoefficients on a COMMON BASIS - both are truncated to the smaller nmax and G2 is rescaled to G1's GM/R if they differ (otherwise the comparison measures processing chains, not solutions) - and returns the standard metric set: spectral: difference degree amplitude vs both signal spectra, per-degree correlation, crossover degree of agreement (shLowLevel.diffSpectrum) spatial: area-weighted bias/RMSD/cent...

| input    | size  | type           | default                      | kind       |
|----------|-------|----------------|------------------------------|------------|
| g1       | 1 x 1 | shCoefficients | required                     | positional |
| g2       | 1 x 1 | shCoefficients | required                     | positional |
| LatDeg   | 1 x : | double         | -89:2:89                     | name-value |
| LonDeg   | 1 x : | double         | 0:2:358                      | name-value |
| Quantity | 1 x 1 | string         | "geoid"                      | name-value |
| kn       | —     | double         | []                           | name-value |
| Mask     | —     | —              | []                           | name-value |
| Names    | 1 x 2 | string         | ["solution 1", "solution 2"] | name-value |
| Plot     | 1 x 1 | logical        | false                        | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                                                          |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rep    | (1,1) struct fields: .spectral (1,1) struct shLowLevel.diffSpectrum output .spatial (1,1) struct shLowLevel.spatialStats output .chi2dof (1,1) double error-realism statistic (NaN when either solution lacks sigmas) .nmax (1,1) double common maximum degree used .quantity (1,1) string compared quantity .names (1,2) string solution labels .rescaled (1,1) logical true when GM/R were unified |
| h      | (1,1) graphics handle figure handle (Plot = true only)                                                                                                                                                                                                                                                                                                                                               |

### Example

```
rep = shLowLevel.compareSolutions(g, g.gaussian(350), Names = ["raw", "G350"]);
fprintf("crossover n = %d, spatial corr = %.3f\n", ...
    rep.spectral.ncross, rep.spatial.corr)
```

### shLowLevel.compareSeries

```
[rep, h] = shLowLevel.compareSeries(tsList, Name=Value)
```

Temporal + component comparison of N solution series. REP = shLowLevel.compareSeries({TS1, TS2, ...}) compares N >= 2 shSeries against the FIRST (the reference) on a COMMON BASIS: epochs are matched to the reference within MatchTolerance (0.02) (unmatched epochs are dropped and reported - never interpolated across gaps), all series are truncated to the smallest nmax, and GM/R are unified. Per solution k >= 2 the standard temporal metric set is computed on a scalar comparison series y\_k(t) - a basin average when ...

| input          | size  | type    | default       | kind       |
|----------------|-------|---------|---------------|------------|
| tsList         | 1 x : | cell    | required      | positional |
| Names          | —     | string  | strings(1, 0) | name-value |
| MatchTolerance | 1 x 1 | double  | 0.02          | name-value |
| Basin          | —     | double  | []            | name-value |
| Idx            | —     | —       | []            | name-value |
| LatDeg         | 1 x : | double  | -88:4:88      | name-value |
| LonDeg         | 1 x : | double  | 0:6:354       | name-value |
| Quantity       | 1 x 1 | string  | "geoid"       | name-value |
| kn             | —     | double  | []            | name-value |
| Mask           | —     | —       | []            | name-value |
| Plot           | 1 x 1 | logical | false         | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rep    | (1,1) struct fields: .names (1,N) string .epochs (T,1) double common epochs used .nDropped (1,N) double reference epochs unmatched per series .y (T,N) double comparison series (col 1 = reference) .nse (1,N) double NSE vs reference (NaN for col 1) .corr (1,N) struct-array-free: Pearson r per series .pcorr (1,N) double AR(1)-corrected two-sided p .neff (1,N) double effective sample sizes .trendDiff (1,N) double trend - reference trend [units/yr] .trendSigma (1,N) double combined 1-sigma of trendDiff .trendZ (1,N) double significance z of trendDiff .ampRatio (1,N) double annual amplitude / reference .phaseLagDays (1,N) double annual phase lag vs reference .chi2dof (1,N) double error realism vs reference .trendDiffFields (1,N) cell shCoefficients (empty for col 1) .tch (1,1) struct shLowLevel.threeCorneredHat output (N >= 3) .nmax (1,1) double common maximum degree |
| h      | (1,1) graphics handle figure handle (Plot = true only)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

#### Example

```
rep = shLowLevel.compareSeries({tsCSR, tsGFZ, tsJPL}, ...
    Names = ["CSR", "GFZ", "JPL"]);
fprintf("TCH noise: %s\n", sprintf("%.3g ", rep.tch.sigma))
```

### shLowLevel.diffSpectrum

```
spec = shLowLevel.diffSpectrum(C1, S1, C2, S2)
```

Spectral comparison of two coefficient sets. SPEC = shLowLevel.diffSpectrum(C1, S1, C2, S2) compares two solutions in the spectral domain: the difference degree amplitude localizes the disagreement in wavelength, the degree correlation separates "same pattern, different amplitude" from genuine disagreement, and the crossover of the difference against the signal spectrum gives the effective resolution of agreement as a single scalar. Inputs must share the same size; truncate beforehand for mixed degrees (the shLo...

| input | size | type   | default  | kind       |
|-------|------|--------|----------|------------|
| C1    | —    | double | required | positional |
| S1    | —    | double | required | positional |
| C2    | —    | double | required | positional |
| S2    | —    | double | required | positional |

| output | description                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| spec   | (1,1) struct fields: .n (nmax+1 x 1) double degree axis 0..nmax .diffAmp (nmax+1 x 1) double degree amplitude of (1)-(2) .amp1 (nmax+1 x 1) double signal degree amplitude of (1) .amp2 (nmax+1 x 1) double signal degree amplitude of (2) .degCorr (nmax+1 x 1) double per-degree correlation (NaN where a degree is all-zero) .ncross (1,1) double first degree where diffAmp exceeds amp1 (NaN if never; agreement to nmax) |

#### Example

```
gF = g.gaussian(350);
spec = shLowLevel.diffSpectrum(g.C, g.S, gF.C, gF.S);
semilogy(spec.n, [spec.amp1, spec.diffAmp]); grid on
```

### shLowLevel.spatialStats

```
stats = shLowLevel.spatialStats(A, B, latDeg, lonDeg, Name=Value)
```

Area-weighted spatial comparison of two grids. STATS = shLowLevel.spatialStats(A, B, LATDEG, LONDEG) compares two nlat x nlon grids with proper cos(latitude) area weighting - unweighted RMS on a lat/lon grid over-counts the poles and is simply wrong. Returns bias, RMSD, the CENTERED pattern statistics (centered RMSD, weighted correlation, standard deviations) that feed a Taylor diagram, and verifies the Taylor identity  $\text{crmsd}^2 = \text{stdA}^2 + \text{stdB}^2 - 2 \text{stdA} \text{stdB} \text{corr}$  internally.

| input  | size  | type   | default  | kind       |
|--------|-------|--------|----------|------------|
| A      | —     | double | required | positional |
| B      | —     | double | required | positional |
| latDeg | 1 x : | double | required | positional |

| input   | size  | type   | default  | kind       |
|---------|-------|--------|----------|------------|
| lonDeg  | 1 x : | double | required | positional |
| Mask    | —     | —      | []       | name-value |
| Weights | —     | double | []       | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| stats  | (1,1) struct fields: .bias (1,1) double weighted mean of A - B .rmsd (1,1) double weighted RMS of A - B .crmsd (1,1) double centered (bias-free) RMSD .corr (1,1) double weighted pattern correlation .stdA (1,1) double weighted std of A .stdB (1,1) double weighted std of B .ratio (1,1) double stdB / stdA .nUsed (1,1) double number of grid points used |

#### Example

```
A1 = g.synthesis(lat, lon); A2 = gF.synthesis(lat, lon);
st = shLowLevel.spatialStats(A1, A2, lat, lon);
fprintf("rmsd %.3g, corr %.3f\n", st.rmsd, st.corr)
```

### shLowLevel.nashSutcliffe

```
nse = shLowLevel.nashSutcliffe(ref, model)
```

Nash-Sutcliffe efficiency of a model against a reference. NSE = shLowLevel.nashSutcliffe(REF, MODEL) computes the hydrology-standard skill score  $NSE = 1 - \text{sum}((\text{model} - \text{ref})^2) / \text{sum}((\text{ref} - \text{mean}(\text{ref}))^2)$  column-wise. NSE = 1 is perfect, 0 means "no better than the reference mean", negative is worse than the mean. More informative than correlation because bias and amplitude errors are punished. NaN pairs are removed per column.

| input | size | type   | default  | kind       |
|-------|------|--------|----------|------------|
| ref   | —    | double | required | positional |
| model | —    | double | required | positional |

| output | description                                            |
|--------|--------------------------------------------------------|
| nse    | (1,K) double efficiency per column (K = 1 for vectors) |

#### Example

```
nse = shLowLevel.nashSutcliffe(avg(1,:), avg(2,:)); % e.g. SH vs mascon basin
```

### shLowLevel.effectiveCorr

```
out = shLowLevel.effectiveCorr(x, y)
```

Correlation with AR(1)-corrected significance. OUT = shLowLevel.effectiveCorr(X, Y) computes the Pearson correlation of two time series together with a significance test that accounts for serial correlation: monthly GRACE series are strongly autocorrelated, so the effective sample size  $N_{\text{eff}} = T \cdot (1 - r_1 \cdot r_2) / (1 + r_1 \cdot r_2)$  ( $r_1, r_2 = \text{lag-1}$  autocorrelations) is well below T, and the naive t-test badly over-states significance (Monte-Carlo: ~30% false positives at  $\phi = 0.7$  vs ~6% corrected; validated in Python). The two-si...

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| x     | : x 1 | double | required | positional |
| y     | : x 1 | double | required | positional |

| output | description                                                                                                                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out    | (1,1) struct fields: .r (1,1) double Pearson correlation .neff (1,1) double effective sample size (clamped [4, T]) .t (1,1) double t statistic on neff-2 dof .p (1,1) double two-sided p-value .n (1,1) double finite pairs used |

#### Example

```
ec = shLowLevel.effectiveCorr(avg(1,:)', avg(2,:)'); % two basin series
fprintf("r = %.3f (p = %.3g, Neff = %.0f)\n", ec.r, ec.p, ec.neff)
```

### shLowLevel.threeCorneredHat

out = shLowLevel.threeCorneredHat(X)

Per-solution noise levels from  $N \geq 3$  series.  $OUT = \text{shLowLevel.threeCorneredHat}(X)$  with  $X$  ( $T \times N$ ,  $N \geq 3$ ) estimates the individual noise standard deviation of each series by the classic (generalized) three-cornered hat: pairwise difference variances  $V(i,j) = \text{var}(x_i - x_j)$  cancel the common signal, and under the assumption of mutually independent noises  $V(i,j) = s_i^2 + s_j^2$ ; the  $N$  unknowns are solved from the  $N*(N-1)/2$  pairs by least squares (exact Grubbs solution for  $N = 3$ ). This answers the question pairwise...

| input | size  | type   | default  | kind       |
|-------|-------|--------|----------|------------|
| X     | : x : | double | required | positional |

| output | description                                                                                                                                                                                                                                                |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out    | (1,1) struct fields: .sigma (1,N) double noise standard deviation per series .variance (1,N) double raw LS variance estimates (may be < 0) .pairVar (P,3) double [i j var(x_i - x_j)] per pair .clipped (1,N) logical true where variance was clipped to 0 |

#### Example

```
out = shLowLevel.threeCorneredHat(Y3); % T x 3 basin series (CSR/GFZ/JPL)
fprintf("noise levels: %s\n", sprintf("%.3g ", out.sigma))
```

### shLowLevel.taylorDiagram

h = shLowLevel.taylorDiagram(refStd, stds, corrs, Name=Value)

Taylor diagram (std / correlation / centered RMSD).  $H = \text{shLowLevel.taylorDiagram}(\text{REFSTD}, \text{STDS}, \text{CORRS})$  draws the standard one-figure comparison of solutions against a reference: radius is the (weighted) standard deviation, the azimuth is  $\text{acos}(\text{correlation})$ , and by the law of cosines the distance to the reference point equals the centered RMSD - so pattern amplitude, pattern correlation and pattern error are read off a single plot. Feed it the .stdA/.stdB/.corr fields of shLowLevel.spatialStats, or temporal statis...

| input     | size  | type    | default          | kind       |
|-----------|-------|---------|------------------|------------|
| refStd    | 1 x 1 | double  | required         | positional |
| stds      | 1 x : | double  | required         | positional |
| corrs     | 1 x : | double  | required         | positional |
| Labels    | —     | string  | strings(1, 0)    | name-value |
| Title     | 1 x 1 | string  | "Taylor diagram" | name-value |
| Normalize | 1 x 1 | logical | false            | name-value |

| output | description                           |
|--------|---------------------------------------|
| h      | (1,1) graphics handle the axes handle |

#### Example

```
st = shLowLevel.spatialStats(grid, 0.8 * grid, lat, lon);
shLowLevel.taylorDiagram(st.stdA, st.stdB, st.corr, Labels = "damped");
```

## Plotting

### shLowLevel.plotSHMap

h = shLowLevel.plotSHMap(grid, latDeg, lonDeg, Name=Value)

Global map plot with coastlines and a sane divergent scale.  $H = \text{shLowLevel.plotSHMap}(\text{GRID}, \text{LAT}, \text{LON})$  renders a synthesized field the way a GRACE map should look by default: divergent blue-white-red colormap symmetric about zero, robust color limits (98th percentile of |value|), coastline overlay (base MATLAB 'coastlines'), correct aspect. The convenience method g.map(...) synthesizes and plots in one call.

| input | size | type   | default  | kind       |
|-------|------|--------|----------|------------|
| grid  | —    | double | required | positional |

| input      | size  | type                        | default     | kind       |
|------------|-------|-----------------------------|-------------|------------|
| latDeg     | 1 x : | double                      | required    | positional |
| lonDeg     | 1 x : | double                      | required    | positional |
| Projection | 1 x 1 | string ∈ ["plate","hammer"] | "plate"     | name-value |
| Coast      | 1 x 1 | logical                     | true        | name-value |
| CLim       | —     | double                      | []          | name-value |
| Colormap   | 1 x 1 | string                      | "divergent" | name-value |
| Title      | 1 x 1 | string                      | ""          | name-value |
| Units      | 1 x 1 | string                      | ""          | name-value |
| ax         | —     | —                           | []          | name-value |

| output | description                                |
|--------|--------------------------------------------|
| h      | (1,1) graphics handle axes of the map plot |

*Example*

```
shLowLevel.plotSHMap(grid, lat, lon, Title = "EWH 2008-04 [m]");
```

### shLowLevel.plotSHSpectrum

`h = shLowLevel.plotSHSpectrum(specs, varargin)`

Log-log degree-amplitude spectrum plot. PLOTSHSPECTRUM(SPEC) plots SPEC.degree vs SPEC.degAmplitude (from SHDEGREERMS) on a log-log scale, in mm geoid-height-equivalent.

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| specs    | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                                   |
|--------|-----------------------------------------------|
| h      | (1 x 1) graphics handle log-log spectrum axes |

*Example*

```
spec = shLowLevel.shDegreeRMS(g.C, g.S);
shLowLevel.plotSHSpectrum(spec, Kaula = true, MarkCrossover = true);
```

### shLowLevel.plotSHCoeffTriangle

`h = shLowLevel.plotSHCoeffTriangle(C, S, varargin)`

Coefficient-triangle diagnostic plot. PLOTSHCOEFFTRIANGLE(C, S) shows |C<sub>nm</sub>| (left half) and |S<sub>nm</sub>| (right half) as a single degree/order "butterfly" triangle image on a log color scale -- the standard first diagnostic for spotting striping, leakage, or bad coefficients before spectral/spatial analysis. x-axis: order m (mirrored: S<sub>nm</sub> on the LEFT, C<sub>nm</sub> on the RIGHT), y-axis: degree n with the apex (n=0) at the TOP - the triangle's peak points upwards. The data aspect ratio is fixed to 1:1, so the axes box is 2:1 (x ...

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| C        | —    | —    | required | positional |
| S        | —    | —    | required | positional |
| varargin | —    | —    | required | positional |

| output | description                                                                                |
|--------|--------------------------------------------------------------------------------------------|
| h      | (1 x 1) graphics handle butterfly triangle image (abs log10 or signed diff with RefC/RefS) |

*Example*

```
shLowLevel.plotSHCoeffTriangle(g.C, g.S);
shLowLevel.plotSHCoeffTriangle(gF.C, gF.S, RefC = g.C, RefS = g.S); % signed diff
```

### shLowLevel.plotBasinSeries

```
h = shLowLevel.plotBasinSeries(tYears, c, sigma, Name=Value)
```

Basin time series with 1-sigma band, gaps, trend. `H = shLowLevel.plotBasinSeries(TYEARS, C, SIGMA)` draws the standard basin- average figure: shaded 1-sigma band (fill), data line, the GRACE <-> GRACE-FO gap (or any gap > GapThreshold (0.3) years) hatched grey, and an annotated trend fitted with the toolbox climatology design (bias/ trend/annual/semi-annual, AR(1)-corrected sigma).

| input        | size  | type    | default         | kind       |
|--------------|-------|---------|-----------------|------------|
| tYears       | : x 1 | double  | required        | positional |
| c            | : x 1 | double  | required        | positional |
| sigma        | —     | double  | []              | positional |
| Label        | 1 x 1 | string  | "basin average" | name-value |
| Units        | 1 x 1 | string  | ""              | name-value |
| GapThreshold | 1 x 1 | double  | 0.3             | name-value |
| Trend        | 1 x 1 | logical | true            | name-value |
| ax           | —     | —       | []              | name-value |

| output | description                                         |
|--------|-----------------------------------------------------|
| h      | (1,1) graphics handle axes of the basin series plot |

#### Example

```
shLowLevel.plotBasinSeries(op.tYears, avg(1,:).', out.sigma(1,:).', Units = "m EWH");
```

### shLowLevel.plotCovariance

```
h = shLowLevel.plotCovariance(M, idx, Name=Value)
```

Covariance/correlation image with order-block boundaries. `H = shLowLevel.plotCovariance(M, IDX)` shows a  $P \times P$  covariance (e.g. from `shLowLevel.readSINEX` with `Index=IDX`) as an image in `shIndex` ordering, with the (order, C/S) block boundaries drawn - the one-look check of how good the block-diagonal approximation (tvANS block path, DDK structure) is for THIS matrix.

| input      | size  | type                                      | default       | kind       |
|------------|-------|-------------------------------------------|---------------|------------|
| M          | —     | double                                    | required      | positional |
| idx        | 1 x 1 | struct                                    | required      | positional |
| Type       | 1 x 1 | string $\in$ ["correlation","covariance"] | "correlation" | name-value |
| BlockLines | 1 x 1 | logical                                   | true          | name-value |
| ax         | —     | —                                         | []            | name-value |

| output | description                                                           |
|--------|-----------------------------------------------------------------------|
| h      | (1 x 1) graphics handle image of the covariance/correlation structure |

#### Example

```
shLowLevel.plotCovariance(snx.M, idx); % SINEX covariance structure
```

### Utilities

#### shLowLevel.designFilter

```
[W, info] = shLowLevel.designFilter(SigmaC, SigmaS, Name=Value)
```

Build a DDK-class anisotropic filter from user covariances.  $W = \text{shLowLevel.designFilter}(\text{SIGMAC}, \text{SIGMAS})$  constructs the Wiener- type decorrelation filter  $W = (N + \text{Alpha} * \text{inv}(S))^{-1} * N$  per order/parity block - the same construction as the published DDK filters, but matched to YOUR solution's error information instead of a fixed release. N is the normal (inverse noise covariance) matrix built from the formal sigmas (diagonal) or from a full covariance (Noise=); S is the signal covariance from a Kaula-type degree ...

| input     | size  | type   | default        | kind       |
|-----------|-------|--------|----------------|------------|
| SigmaC    | —     | double | required       | positional |
| SigmaS    | —     | double | required       | positional |
| Alpha     | 1 x 1 | double | 1              | name-value |
| Kaula     | 1 x 1 | double | 1e-5           | name-value |
| DegVar    | —     | double | []             | name-value |
| Noise     | —     | double | []             | name-value |
| Signal    | —     | double | []             | name-value |
| Idx       | —     | —      | []             | name-value |
| MinDegree | 1 x 1 | double | 2              | name-value |
| Name      | 1 x 1 | string | "designFilter" | name-value |

| output | description                                                                                                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| W      | (1,1) struct fields: nmax (1,1 double), name (char), blocks (1,K struct: m (1,1 double), cs ('c' 's'), n (1,B double, degrees), M (B x B double)) - identical layout to shLowLevel.readDDK |
| info   | (1,1) struct fields: alpha (1,1 double), mode (string: "diagonal"   "fullcov"), gainRange (1,2 double, min/max eigenvalue over all blocks)                                                 |

#### Example

```
W = shLowLevel.designFilter(g.sigmaC, g.sigmaS, Alpha = 1);
gF = g.applyDDK(W); % apply like any DDK
```

### shLowLevel.estimateDegree1

```
[d1, info] = shLowLevel.estimateDegree1(ts, ocean, Name=Value)
```

Geocentre coefficients from GRACE itself (Swenson 2008). [D1, INFO] = shLowLevel.estimateDegree1(TS, OCEAN) estimates the degree-1 coefficients C10, C11, S11 for every epoch of the series TS, using the method of Swenson, Chambers & Wahr (2008).

| input      | size  | type     | default  | kind       |
|------------|-------|----------|----------|------------|
| ts         | 1 x 1 | shSeries | required | positional |
| ocean      | —     | —        | required | positional |
| kn         | —     | double   | []       | name-value |
| OceanModel | —     | double   | []       | name-value |
| LatDeg     | 1 x : | double   | -89:2:89 | name-value |
| LonDeg     | 1 x : | double   | 0:2:358  | name-value |
| Nmax       | 1 x 1 | double   | NaN      | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| d1     | (1,1) struct fields: epoch, t0, t1 (T,1 double), C10, C11, S11 (T,1 double) and sigC10, sigC11, sigS11 (T,1 double, from the ocean-fit residual propagated through the least-squares solution). This is exactly the layout shLowLevel.readTN13 returns - including the sigmas, which shCoefficients.addDegree1 requires - so it drops straight into shSeries.addDegree1 |

| output | description                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| info   | (1,1) struct fields: cond (T,1 double, condition number of the column-equilibrated design matrix per epoch (equilibrated so the number reflects the GEOMETRY of the ocean domain, not the units of the columns) - a rising value warns that the ocean domain is too small or too lopsided BEFORE the noise shows it), oceanFraction (1,1 double, area fraction used), nPixels, hasModel (1,1 logical), residualRMS (T,1 double, of the ocean fit in EWH [m]) |

#### Example

```
d1 = shLowLevel.estimateDegree1(ts, isOcean, kn = LN.kn, ...
    OceanModel = obpModel);
ts = ts.addDegree1(d1);           % same interface as a TN-13 table
```

### shLowLevel.fetchLoveNumbers

```
[files, info] = shLowLevel.fetchLoveNumbers(names, Name=Value)
```

Download load/deformation Love number files (GROOPS set). FILES = shLowLevel.fetchLoveNumbers() downloads the standard load Love number file (Gegout97, kn for degrees 0..1024) from the GROOPS data collection hosted by TU Graz into <dataFolder>/loveNumbers, with the usual safe-swap semantics. The toolbox deliberately never hardcodes Love numbers - this fetcher automates obtaining a PUBLISHED set, which you then pass explicitly (kn = ...) where required.

| input   | size  | type    | default                                              | kind       |
|---------|-------|---------|------------------------------------------------------|------------|
| names   | —     | string  | "loadLoveNumbers_Gegout97.txt"                       | positional |
| Dest    | 1 x 1 | string  | ""                                                   | name-value |
| BaseURL | 1 x 1 | string  | "https://ftp.tugraz.at/pub/ITSG/groups/data/loading" | name-value |
| Timeout | 1 x 1 | double  | 30                                                   | name-value |
| Proxy   | 1 x 1 | string  | ""                                                   | name-value |
| Update  | 1 x 1 | logical | false                                                | name-value |
| Quiet   | 1 x 1 | logical | false                                                | name-value |

| output | description                                                                                                                                     |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| files  | (1,K) string local paths                                                                                                                        |
| info   | (1,1) struct fields: fetched/updated/skipped/failed (1,K string); parsed (1,K struct: name, n (N x 1), kn (N x 1); empty for deformation files) |

#### Example

```
[f, inf] = shLowLevel.fetchLoveNumbers();
kn = inf.parsed(1).kn(1:61);           % degrees 0..60 for an n60 field
```

### shLowLevel.gridScaling

```
[k, info] = shLowLevel.gridScaling(model, latDeg, lonDeg, Name=Value)
```

Per-pixel leakage/attenuation scaling factors from a model. [K, INFO] = shLowLevel.gridScaling(MODEL, LATDEG, LONDEG, Filter = ...) computes the classic gridded scaling factors: push a MODEL field series through the SAME processing chain the data saw and regress, per pixel, the true value on the filtered one,

| input     | size  | type   | default    | kind       |
|-----------|-------|--------|------------|------------|
| model     | —     | double | required   | positional |
| latDeg    | —     | double | required   | positional |
| lonDeg    | —     | double | required   | positional |
| Filter    | —     | —      | "gauss300" | name-value |
| Nmax      | 1 x 1 | double | NaN        | name-value |
| MinSignal | 1 x 1 | double | 1e-3       | name-value |

| input | size  | type    | default        | kind       |
|-------|-------|---------|----------------|------------|
| Clip  | —     | double  | []             | name-value |
| GM    | 1 x 1 | double  | 3.986004415e14 | name-value |
| R     | 1 x 1 | double  | 6378136.3      | name-value |
| Quiet | 1 x 1 | logical | false          | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| k      | (nLat x nLon) double scaling factors; NaN where the model carries too little signal to define one                                                                                                                                                                                                                                                                                          |
| info   | (1,1) struct fields: covered (1,1 double, fraction of the grid with a finite k), kMedian / kMin / kMax (1,1 double, over the pixels the MODEL gives mass to - not over all finite pixels, where pure-leakage k = 0 would dominate), onSignal (1,1 double, how many those are), clipped (1,1 double, number of clamped pixels), nmax (1,1 double), filter (1,1 string), epochs (1,1 double) |

### Example

```
k = shLowLevel.gridScaling(modelEWH, -89:89, 0:359, Filter = "gauss300");
corrected = k .* filteredEWH;      % NaN where the model is blind
```

### shLowLevel.leakageCorrect

```
[mCorr, info] = shLowLevel.leakageCorrect(grid, latDeg, lonDeg, Name=Value)
```

Iterative forward modelling of filter leakage/attenuation. [MCORR, INFO] = shLowLevel.leakageCorrect(GRID, LATDEG, LONDEG, Filter = ...) recovers the mass field that, pushed through the SAME processing chain the data saw, reproduces the observed filtered GRID. The chain F is analysis -> filter -> synthesis, and the correction is the fixed-point iteration

| input          | size  | type    | default        | kind       |
|----------------|-------|---------|----------------|------------|
| grid           | —     | double  | required       | positional |
| latDeg         | —     | double  | required       | positional |
| lonDeg         | —     | double  | required       | positional |
| Filter         | —     | —       | "gauss300"     | name-value |
| Mask           | —     | —       | []             | name-value |
| Nmax           | 1 x 1 | double  | NaN            | name-value |
| Gain           | 1 x 1 | double  | 1              | name-value |
| NoiseLevel     | 1 x 1 | double  | 0              | name-value |
| ResidualRegion | —     | —       | []             | name-value |
| Tau            | 1 x 1 | double  | 1.2            | name-value |
| MaxIter        | 1 x 1 | double  | 200            | name-value |
| Tol            | 1 x 1 | double  | 1e-4           | name-value |
| GM             | 1 x 1 | double  | 3.986004415e14 | name-value |
| R              | 1 x 1 | double  | 6378136.3      | name-value |
| Quiet          | 1 x 1 | logical | false          | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mCorr  | (nLat x nLon) double leakage-corrected field, same units and grid as GRID                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| info   | (1,1) struct fields: iterations (1,1 double), residual (1,1 double, final relative residual - with a mask this floors above Tol and that is correct), history (1,K double, the residual per iteration - plot it, a rising curve means Gain is too large), step (1,K double, the relative change of the solution per iteration; this is what Tol tests), stoppedBy (1,1 string: "discrepancy"   "step"   "maxIter"   "zeroField" - always check this, it says whether the result was regularised), residualRMS (1,1 double, over ResidualRegion), residualRMSGlobal (1,1 double, over the whole grid - a large gap between the two means unmodelled signal elsewhere) and noiseLevel (1,1 double, as given), converged (1,1 logical), nmax (1,1 double), filter (1,1 string), masked (1,1 logical) |

### Example

```
ewh = g.synthesis(-89:89, 0:359, quantity = "ewh", kn = kn);  
[m, info] = shLowLevel.leakageCorrect(ewh, -89:89, 0:359, ...  
    Filter = "gauss300", Mask = basinMask, NoiseLevel = oceanRMS);
```

### shLowLevel.listITSG

```
[T, info] = shLowLevel.listITSG(Name=Value)
```

Catalogue of ITSG gravity field series on the TU Graz server. T = shLowLevel.listITSG() enumerates the available ITSG releases and product folders (releases ITSG-Grace2014/2016/2018 and the continuously extended ITSG-Grace\_operational; products monthly at n60/n96/n120 and daily Kalman solutions) by parsing the server directory indices - so mixing files from different releases, the trap of guessing folder names, becomes impossible. The catalogue row number is the selection handle for shLowLevel.fetchITSG(Catalog=...

| input   | size  | type   | default                                | kind       |
|---------|-------|--------|----------------------------------------|------------|
| BaseURL | 1 x 1 | string | "https://ftp.tugraz.at/pub/ITSG/GRACE" | name-value |
| Timeout | 1 x 1 | double | 30                                     | name-value |

| output | description                                                                                                                                                                                                                                                         |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T      | (K x 5) table columns: idx (1,1) double selection number for fetchITSG release (1,1) string e.g. "ITSG-Grace2018" product (1,1) string "monthly"   "daily"   "static" nmax (1,1) double 60/96/120 (NaN for daily/static) url (1,1) string folder URL or mirror path |
| info   | (1,1) struct fields: source (string), nReleases (double)                                                                                                                                                                                                            |

### Example

```
T = shLowLevel.listITSG();  
disp(T) % pick rows, then fetch via Catalog= in fetchITSG
```

### shLowLevel.oceanRMS

```
[rmsVal, info] = shLowLevel.oceanRMS(grid, latDeg, lonDeg, ocean, Name=Value)
```

Open-ocean RMS of a field - the standard GRACE noise metric. RMSVAL = shLowLevel.oceanRMS(GRID, LATDEG, LONDEG, OCEAN) returns the area-weighted RMS of GRID over the OPEN ocean: ocean points more than MinDistanceKm (default 1000) from the nearest non-ocean point.

| input         | size  | type    | default  | kind       |
|---------------|-------|---------|----------|------------|
| grid          | —     | double  | required | positional |
| latDeg        | —     | double  | required | positional |
| lonDeg        | —     | double  | required | positional |
| ocean         | —     | —       | required | positional |
| MinDistanceKm | 1 x 1 | double  | 1000     | name-value |
| Weighted      | 1 x 1 | logical | true     | name-value |
| Subsample     | 1 x 1 | double  | 1        | name-value |

| output | description                                                                                                                                                                                                                          |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rmsVal | (1,1) double open-ocean RMS, in GRID's units. NaN when the erosion leaves no points                                                                                                                                                  |
| info   | (1,1) struct fields: nPixels (1,1 double, points used), fractionUsed (1,1 double, of the whole grid), mask (nLat x nLon logical, the eroded region - plot it once before trusting the number), minDistanceKm, weighted (1,1 logical) |

### Example

```
noise = shLowLevel.oceanRMS(ewh, -89:89, 0:359, isOcean); % > 1000 km offshore  
m = shLowLevel.leakageCorrect(ewh, -89:89, 0:359, Mask = basin, NoiseLevel = noise);
```

## shLowLevel.poleTideConvert

varargout = shLowLevel.poleTideConvert(varargin)

Convert C21/S21 between mean-pole (pole tide) conventions. G2 = shLowLevel.poleTideConvert(G, From="IERS2010", To="IERS2018") adjusts the C21/S21 coefficients of an shCoefficients object (epoch taken from G) from one mean-pole convention to another. Processing centers remove the (solid + ocean) pole tide using a MEAN POLE MODEL, and the IERS 2010 conventional model (cubic until 2010.0, linear after) differs from the 2018 secular-pole update (linear, adopted for GRACE/GRACE-FO RL06) by up to  $\sim 1e-10$  in C21/S21 wit...

| input    | size | type | default  | kind       |
|----------|------|------|----------|------------|
| varargin | —    | —    | required | positional |

| output | description                                                            |
|--------|------------------------------------------------------------------------|
| G2     | (1,1) shCoefficients converted object (object input; history appended) |
| C2, S2 | (nmax+1 x nmax+1) double converted triangles (raw input)               |

### Example

```
g18 = shLowLevel.poleTideConvert(g, From = "IERS2010", To = "IERS2018");  
dC21 = g18.C(3, 2) - g.C(3, 2)      %  $\sim 1e-10$  level, trend-like in t
```

## shLowLevel.readSHM

out = shLowLevel.readSHM(filename, Name=Value)

Read a GRAVIS/GRACE SHM file (GRCOF2 fields, GRDOTA rates). OUT = shLowLevel.readSHM(FILENAME) reads the spherical-harmonic format used by the GRAVIS Level-2B products and by GRACE/GRACE-FO Level-2 files that are not in ICGEM layout:

| input    | size  | type   | default  | kind       |
|----------|-------|--------|----------|------------|
| filename | —     | —      | required | positional |
| Nmax     | 1 x 1 | double | NaN      | name-value |

| output | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out    | (1,1) struct fields: kind (1,1) string "GRCOF2" (field)   "GRDOTA" (rate) C, S (nmax+1 x nmax+1) double coefficients in the C(n+1, m+1) house layout; for GRDOTA these are rates per year sigmaC, sigmaS (nmax+1 x nmax+1) double, or [] when the file carries none (GRDOTA never does) GM (1,1) double $[m^3/s^2]$ from the header R (1,1) double $[m]$ from the header nmax (1,1) double maximum degree actually read header (1,1) string the raw YAML header, kept so provenance is not lost on the way in |

### Example

```
gia = shLowLevel.readSHM("GRAVIS-2B..._GIA_ICE-6G_D_VM5a.gz", Nmax = 96);  
g = shCoefficients(gia.C, gia.S, GM = gia.GM, R = gia.R); % rates [1/yr]
```

## shLowLevel.safeMove

n = shLowLevel.safeMove(src, dst, Name=Value)

Move a file into place, retrying while a scanner still holds it. SAFEMOVE(SRC, DST) renames SRC to DST, overwriting DST, and retries with exponential backoff while the move fails. Every shLowLevel.fetch\* function downloads to "<target>.part", verifies the fresh file by parsing it, and only then swaps it into place - this function is that swap.

| input   | size  | type   | default  | kind       |
|---------|-------|--------|----------|------------|
| src     | 1 x 1 | string | required | positional |
| dst     | 1 x 1 | string | required | positional |
| Retries | 1 x 1 | double | 5        | name-value |
| Pause   | 1 x 1 | double | 0.25     | name-value |

| output | description                                                                                         |
|--------|-----------------------------------------------------------------------------------------------------|
| n      | (1,1) double number of attempts used (1 = the move succeeded immediately, which is the normal case) |

#### Example

```
shLowLevel.shReadGFC(tmpf); % verify BEFORE the swap
shLowLevel.safeMove(tmpf, "ITSG-Grace2018_n60_2008-04.gfc");
```

### shLowLevel.standardChain

```
[ts, rep] = shLowLevel.standardChain(folder, Name=Value)
```

Canonical GRACE/GRACE-FO post-processing in one call. [TS, REP] = shLowLevel.standardChain(FOLDER) encodes the standard chain as a single, correctly ordered entry point: 1. read all monthly solutions from FOLDER (shSeries.fromFolder) 2. replace C20/C30 with the SLR values (TN-14) 3. add the degree-1 (geocenter) coefficients (TN-13) 4. optionally subtract a GIA trend field 5. filter (Gaussian radius, a DDK, or a shLowLevel.designFilter W) The order matters (TN corrections BEFORE filtering, GIA on the corrected se...

| input       | size  | type                                   | default    | kind       |
|-------------|-------|----------------------------------------|------------|------------|
| folder      | 1 x 1 | string                                 | required   | positional |
| Pattern     | 1 x 1 | string                                 | "*.gfc"    | name-value |
| Truncate    | 1 x 1 | double                                 | NaN        | name-value |
| TN14        | 1 x 1 | logical                                | true       | name-value |
| TN14File    | 1 x 1 | string                                 | ""         | name-value |
| Degree1     | 1 x 1 | string ∈ ["GFZ", "CSR", "JPL", "none"] | "GFZ"      | name-value |
| Degree1File | 1 x 1 | string                                 | ""         | name-value |
| GIA         | —     | —                                      | []         | name-value |
| GIAEpoch    | 1 x 1 | double                                 | NaN        | name-value |
| Filter      | —     | —                                      | "gauss300" | name-value |
| Tolerance   | 1 x 1 | double                                 | 0.05       | name-value |
| OnMissing   | 1 x 1 | string ∈ ["drop", "error"]             | "drop"     | name-value |
| Quiet       | 1 x 1 | logical                                | false      | name-value |

| output | description                                                                                                                                                                                                     |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ts     | (1,1) shSeries the processed series (full history)                                                                                                                                                              |
| rep    | (1,1) struct fields: files (1,T string), steps (1,K string, applied stages with parameters), nmax (1,1 double), epochs (T,1 double), version (1,1 string, shLowLevel.version at run time), created (1,1 string) |

#### Example

```
[ts, rep] = shLowLevel.standardChain("D:/grace/monthly", ...
    Filter = "DDK3", Degree1 = "GFZ");
disp(rep.steps')
```

### shLowLevel.synthesisPoints

```
val = shLowLevel.synthesisPoints(C, S, GM, R, latDeg, lonDeg, r, Name=Value)
```

Gravity field quantities at arbitrary 3-D points. VAL = shLowLevel.synthesisPoints(C, S, GM, R, LATDEG, LONDEG, RGEO) evaluates the spherical harmonic expansion POINTWISE at positions given by equal-length vectors of geocentric latitude [deg], longitude [deg] and geocentric radius [m] - satellite altitudes, scattered stations, profiles - including the upward continuation  $(R/r)^n$  that grid synthesis on the sphere cannot provide: potential  $T = GM/r \sum_n (R/r)^n \sum_m P_{nm} (C \cos + S \sin)$  disturbance ...

| input | size | type   | default  | kind       |
|-------|------|--------|----------|------------|
| C     | —    | double | required | positional |

| input     | size  | type                                             | default     | kind       |
|-----------|-------|--------------------------------------------------|-------------|------------|
| S         | —     | double                                           | required    | positional |
| GM        | 1 x 1 | double                                           | required    | positional |
| R         | 1 x 1 | double                                           | required    | positional |
| latDeg    | : x 1 | double                                           | required    | positional |
| lonDeg    | : x 1 | double                                           | required    | positional |
| r         | : x 1 | double                                           | required    | positional |
| Quantity  | 1 x 1 | string ∈ ["potential", "disturbance", "anomaly"] | "potential" | name-value |
| MinDegree | 1 x 1 | double                                           | 2           | name-value |

| output | description                         |
|--------|-------------------------------------|
| val    | (P,1) double quantity at each point |

#### Example

```
dg = shLowLevel.synthesisPoints(g.C, g.S, g.GM, g.R, ...
    [10; 12], [100; 101], (g.R + 450e3) * [1; 1], ...
    Quantity = "disturbance") * 1e5; % mGal at 450 km
```

### shLowLevel.version

`v = shLowLevel.version()`

Toolbox metadata: name, version, date, root path, provenance. `V = shLowLevel.version()` returns the toolbox metadata as a struct. The version string and date are PARSED FROM Contents.m (the MATLAB toolbox convention line "% Version x.y.z ... dd-Mmm-yyyy"), so there is a single source of truth also honoured by MATLAB's own `ver('shAnalysis')`. Called without an output, prints a summary.

| output | description                                                                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v      | (1,1) struct fields: .Name (1,1) string "shAnalysis" .Version (1,1) string e.g. "2.5.1" .Date (1,1) string release date from Contents.m .Root (1,1) string toolbox root folder .Provenance (1,1) string authorship line |

#### Example

```
v = shLowLevel.version();
fprintf("%s %s (%s)\n", v.Name, v.Version, v.Date)
```

## Toolbox root

### setup\_shAnalysis

`summary = setup_shAnalysis(Name=Value)`

One-call setup: path + staged data downloads. `setup_shAnalysis` adds the toolbox to the path for THIS session only `setup_shAnalysis(Permanent (false) = true)` also persists it (savepath) `setup_shAnalysis(Download ("none") = "core")` + TN-14 and TN-13 files `setup_shAnalysis(Download = "filters")` + DDK (3) filter matrices `setup_shAnalysis(Download = "starter")` + ITSG starter months `setup_shAnalysis(DryRun (false) = true, ...)` print/return the plan, do NOTHING (no path change, no f...

| input     | size  | type                                            | default                | kind       |
|-----------|-------|-------------------------------------------------|------------------------|------------|
| Permanent | 1 x 1 | logical                                         | false                  | name-value |
| Download  | 1 x 1 | string ∈ ["none", "core", "filters", "starter"] | "none"                 | name-value |
| DDK       | 1 x : | double                                          | 3                      | name-value |
| Providers | 1 x : | string ∈ ["GFZ", "CSR", "JPL"]                  | ["GFZ", "CSR", "JPL"]  | name-value |
| Months    | 1 x : | string                                          | ["2008-04", "2025-12"] | name-value |

| input      | size  | type                  | default | kind       |
|------------|-------|-----------------------|---------|------------|
| Nmax       | 1 x 1 | double $\in$ [60, 96] | 96      | name-value |
| Docs       | 1 x 1 | logical               | false   | name-value |
| DryRun     | 1 x 1 | logical               | false   | name-value |
| DataFolder | 1 x 1 | string                | ''      | name-value |
| FetchITSG  | 1 x 1 | string                | "none"  | name-value |
| Proxy      | 1 x 1 | string                | ''      | name-value |
| Update     | 1 x 1 | logical               | false   | name-value |
| Quiet      | 1 x 1 | logical               | false   | name-value |

| output     | description                                                                            |
|------------|----------------------------------------------------------------------------------------|
| root       | (1,1) string toolbox root folder                                                       |
| pathAction | (1,1) string "already-on-path"   "added-temporarily"   "added-permanently"   "planned" |
| dataFolder | (1,1) string download target root                                                      |
| plan       | (1,:) string human-readable action list                                                |
| fetched    | (1,:) string files newly downloaded (verified)                                         |
| skipped    | (1,:) string files already present (re-verified)                                       |
| failed     | (1,:) string files that failed (download or parse)                                     |
| ok         | (1,1) logical isempty(failed)                                                          |

#### Example

```
setup_shAnalysis(Permanent = true, Download = "core")
setup_shAnalysis(Download = "filters", DDK = [2 3 5])
s = setup_shAnalysis(DryRun = true, Download = "starter")
```

### runAllTests

results = runAllTests(Name=Value)

Run the complete shAnalysis v2 validation suite. RESULTS = RUNALLTESTS runs, in order:

tests/testCorrectness.m numerics vs. golden/legacy/analytic values tests/testScience.m regression against PUBLISHED values (defining constants, load Love numbers, the closed-form EWH kernel, geocenter amplitudes) - so a consistently wrong formula has somewhere to show up tests/testContract.m error IDs, immutability, dimensions tests/testRobustness.m edge cases, degenerate inputs, file I/O documented patch: kern...

| input           | size  | type    | default | kind       |
|-----------------|-------|---------|---------|------------|
| SkipPerformance | 1 x 1 | logical | false   | name-value |
| LogFile         | —     | —       | ''      | name-value |

| output  | description                                                                                              |
|---------|----------------------------------------------------------------------------------------------------------|
| results | (1,N) matlab.unittest.TestResult full result array (pass/fail/duration per test; log written to LogFile) |

#### Example

```
runAllTests % 169 tests; acceptance gate on R2026a
```

### demo\_shAnalysis

reg = demo\_shAnalysis(cases, Name=Value)

Selectable demonstrations of every toolbox capability. demo\_shAnalysis runs the "core" set (D01 D02 D04 D05 D15) demo\_shAnalysis("all") runs every case demo\_shAnalysis("list") prints the case table, returns registry demo\_shAnalysis(["D04","D15"]) runs selected cases demo\_shAnalysis(..., Visible (true)=false) hidden figures (CI/smoke tests) demo\_shAnalysis(..., OutDir (string(tempdir))=tempdir) target for D16 file exports demo\_shAnalysis(..., StopOnError (false)=true) rethrow ins...

| input       | size  | type    | default         | kind       |
|-------------|-------|---------|-----------------|------------|
| cases       | 1 x : | string  | "core"          | positional |
| Visible     | 1 x 1 | logical | true            | name-value |
| OutDir      | 1 x 1 | string  | string(tempdir) | name-value |
| StopOnError | 1 x 1 | logical | false           | name-value |

| output | description                                           |
|--------|-------------------------------------------------------|
| reg    | (D x 3) table demo registry: id, title, exercised API |

*Example*

```
demo_shAnalysis(["D01", "D06"]) % run selected demos, fail-and-continue
```

## Appendix

### A. Error-identifier conventions

Namespaces: `shLowLevel:<function>:<what>` for package functions, `shCoefficients:/shSeries:/shClimatology:` for class methods. Every guard has a unit test; every message names the offending quantity. Instrumentation asserts (`nonFiniteVCE`, `nonFiniteEig`, `nonFiniteOperator`, `allZeroVariance`) remain in the code as a permanent safety net after the v2.1 debugging campaign.

### B. Known limitations (documented, tested where applicable)

- TN-13 verified against the real GFZ, CSR and JPL RL06.3 files (v2.5; 256 paired months each, cross-provider C10 correlation 0.995); SINEX against ITSG monthly files and synthetic fixtures — COST-G/other SINEX dialects may need parser tweaks (drop files into `tests/test_data`, the discovery tests pick them up).
- DDK: all eight released Wbd binaries parsed and gain-ordering validated (v2.5); a discovery test re-checks whatever subset is present locally. Other BIN dialects untested.
- `readMascon` validated against a synthetic JPL-style fixture; real product files were not openly reachable at build time (auth/503) — verify against yours (units, GIA convention, CRI flag).
- GIA models treated as exact; AR(1) correction is first-order (Kendall-corrected  $r_1$  since v2.5, ~3% residual optimism).
- Multi-center combination: common-mode errors invisible → posterior sigmas a lower bound.
- Basin sigma: deterministic-part errors correlated across epochs (pointwise 1-sigma reported).
- Streaming gz SINEX needs the JVM (fallback: `gunzip` to temp).
- ICGEM catalogue parsers are fixture-tested against real page captures (2026-08-07); a future site redesign would need refreshed fixtures (`shLowLevel:listICGEM:parseFailed` errors loudly).

### C. License & provenance

The toolbox is released under the MIT license (LICENSE in the package root; copyright Matthias Weigelt). The bundled DDK3 Wbd file carries its own MIT license from its upstream repository; ITSG/GFZ files in `tests/test_data` remain the property of their providers and are included as test fixtures only.

All code, tests, documentation and this guide: developed by Matthias Weigelt with the help of Claude (Fable 5), 2026-08-07 (v1 through v2.4.2 in working sessions the same week). Every numerical method was independently validated in Python before MATLAB implementation; MATLAB execution itself happens on the user's side with `runAllTests` as the acceptance gate (118/118 green at v2.1; 134 at v2.2.2; 167 at v2.4.1; 168 at v2.4.2; 179 at v2.5 — v2.5 adds the exact constrained posterior, the basin deterministic-sigma term, the Kendall AR(1) correction, `shLowLevel.readLoveNumbers`, `setup_shAnalysis/shLowLevel.fetchTN` with verified staged downloads, real CSR/JPL TN-13 + GSFC TN-14 fixtures and the DDK-pack validation).